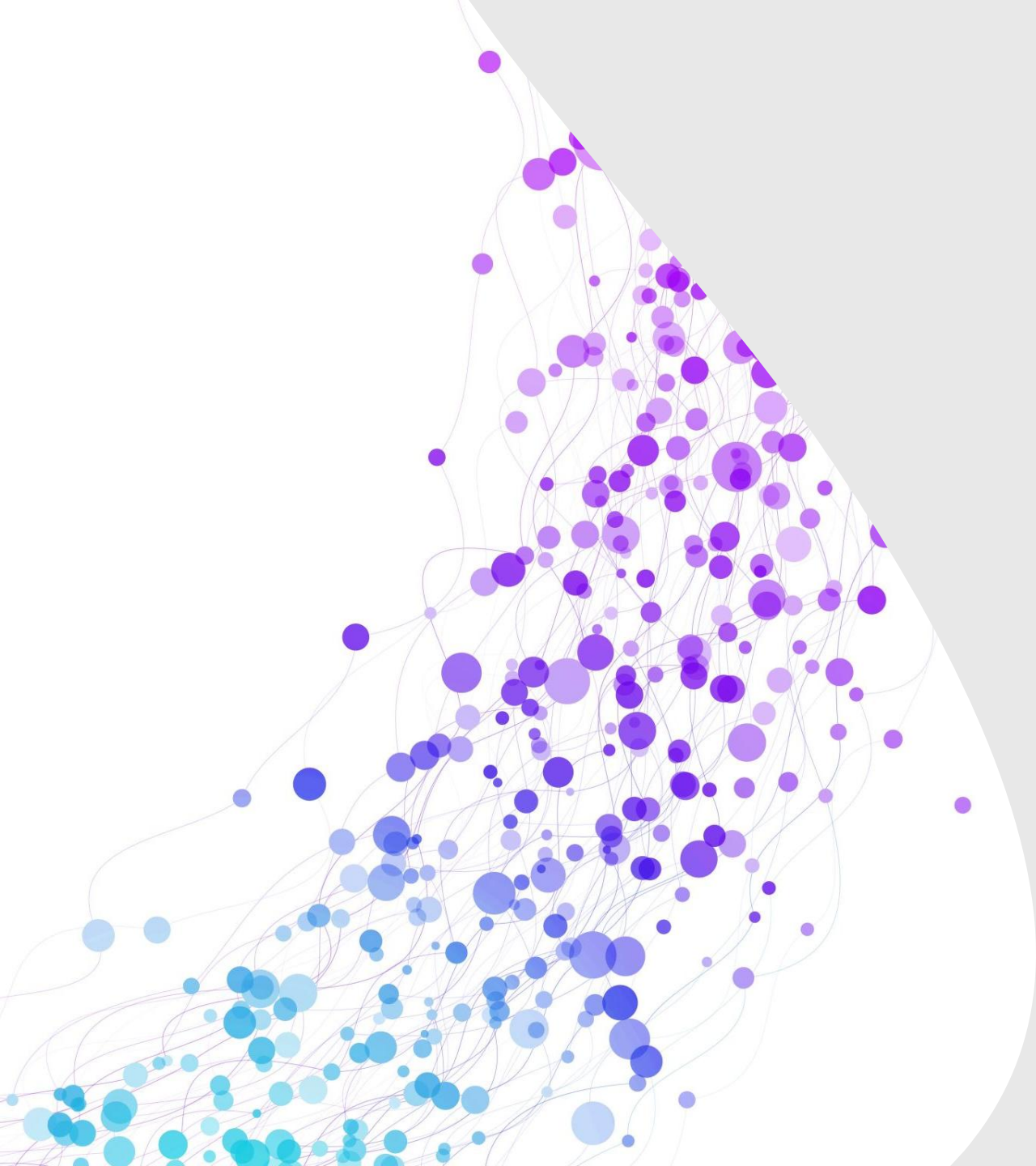
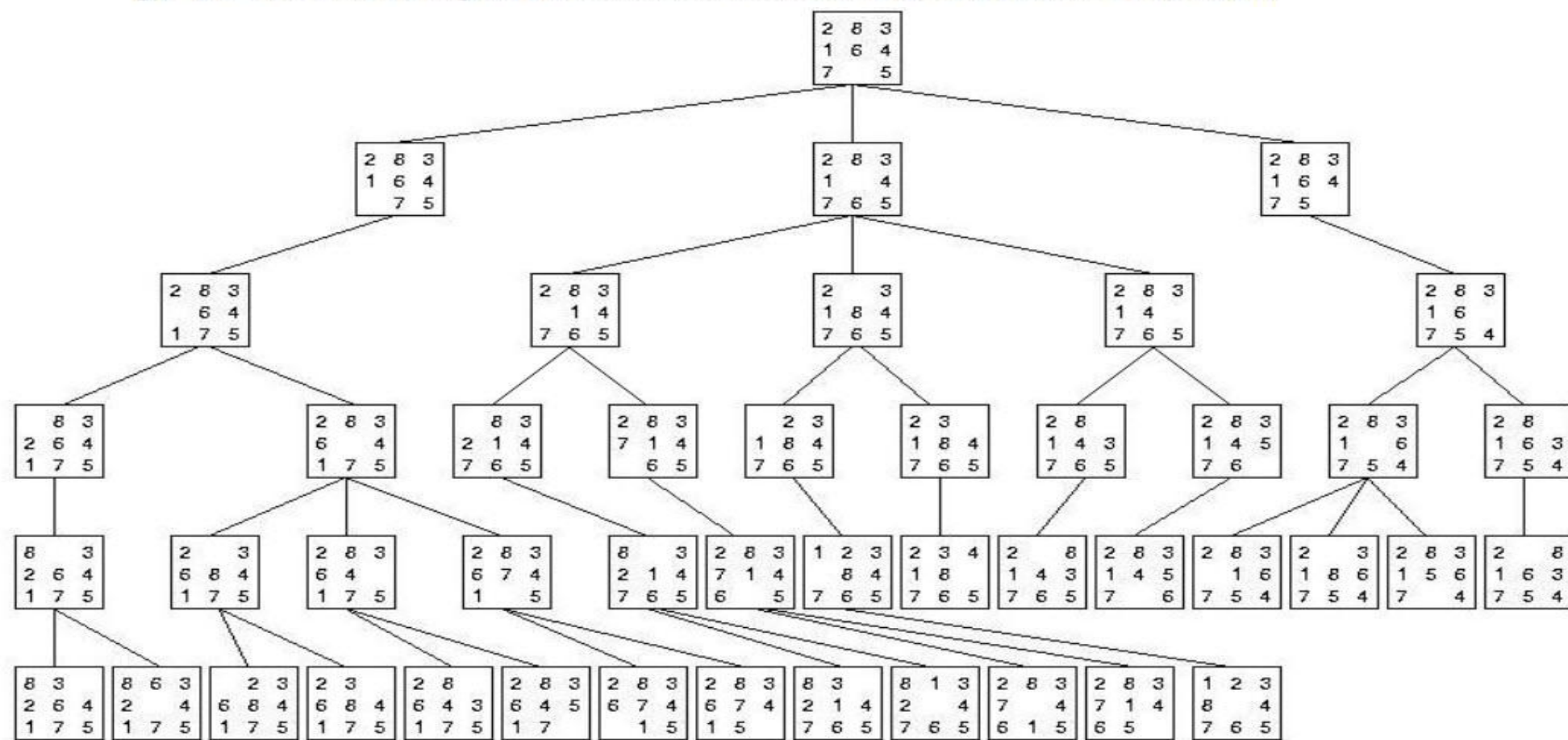




Sheet 2: Search Blind Search

Eng. Abdulrahman ElMadkhoun

1. The following figure gives an example showing the BFS tree for the 8 puzzle problem. List the order of the visited nodes by the following search algorithms:
 - a) Breadth first
 - b) Depth First
 - c) Limited Depth First with limit $d=3$.
 - d) Best first Search using the misplaced tiles as a heuristic function.
 - e) Best first Search using the Manhattan Distance as a heuristic function.
 - f) A* Search using the misplaced tiles as a heuristic function.
 - g) A* Search using the Manhattan Distance as a heuristic function.



2	8	3
1	6	4
7		5

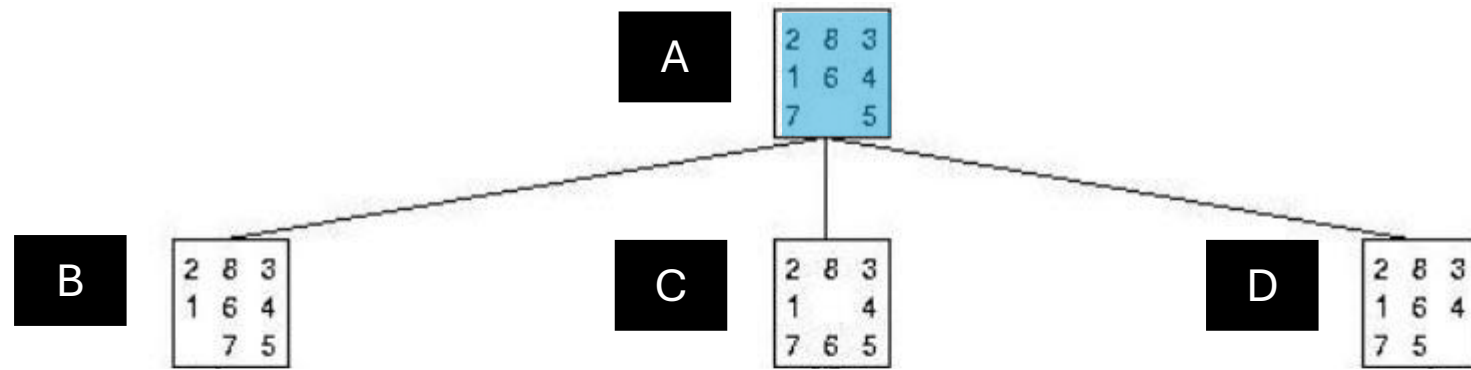
Initial State

1	2	3
8		4
7	6	5

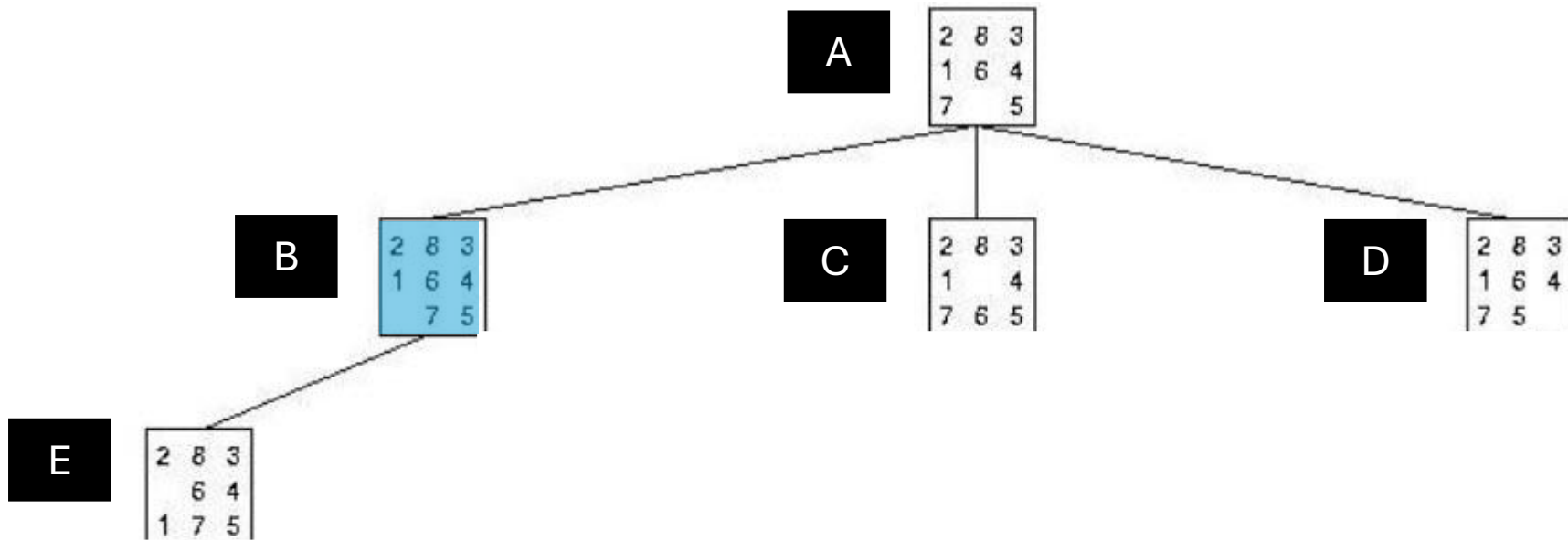
Goal State

1-Breadth-first search

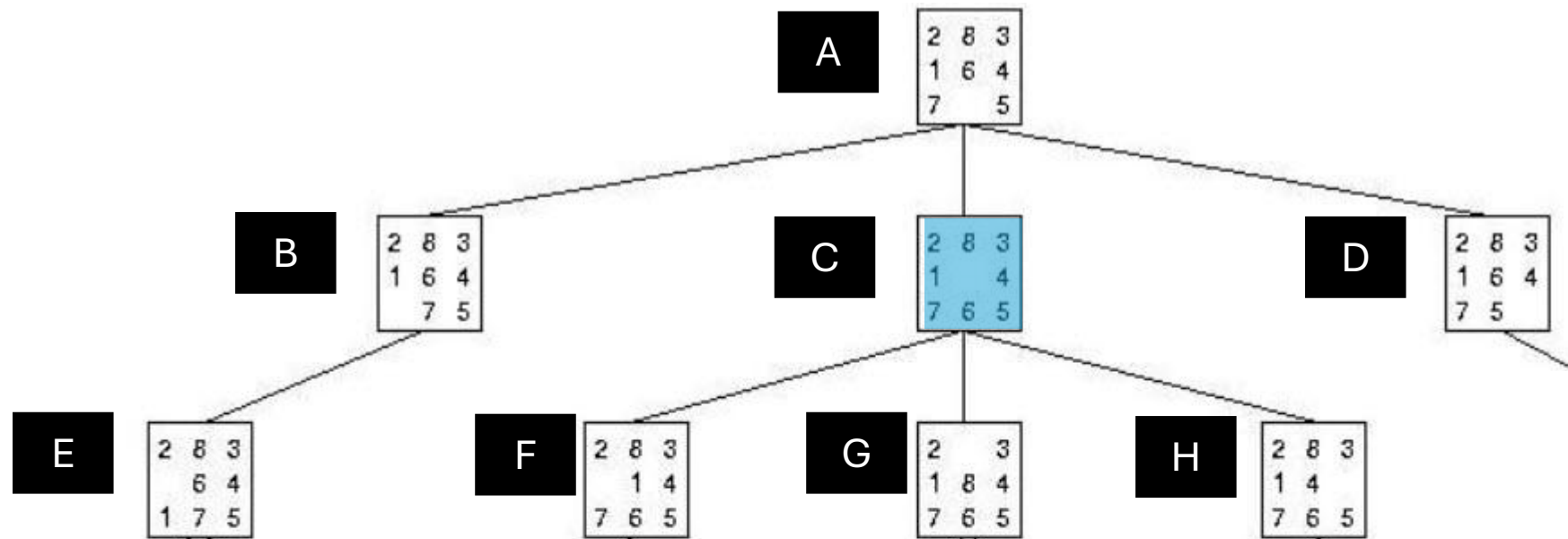
Breadth-first is blind search Moves from left to right, level by level until reaching the Goal



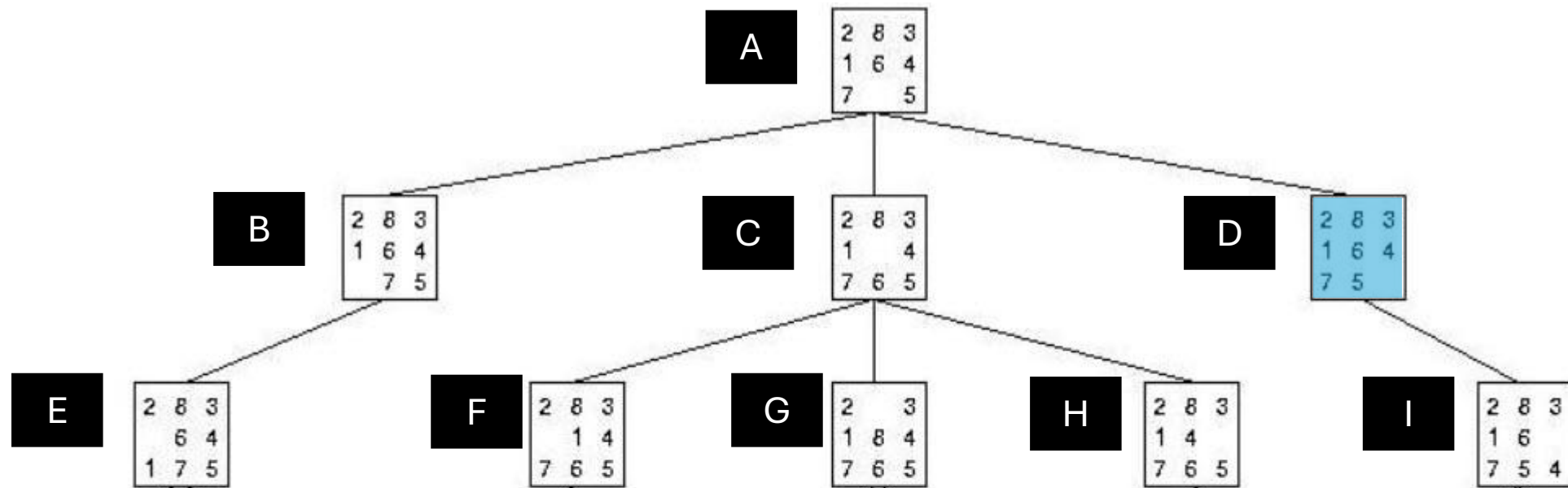
Order : A ,



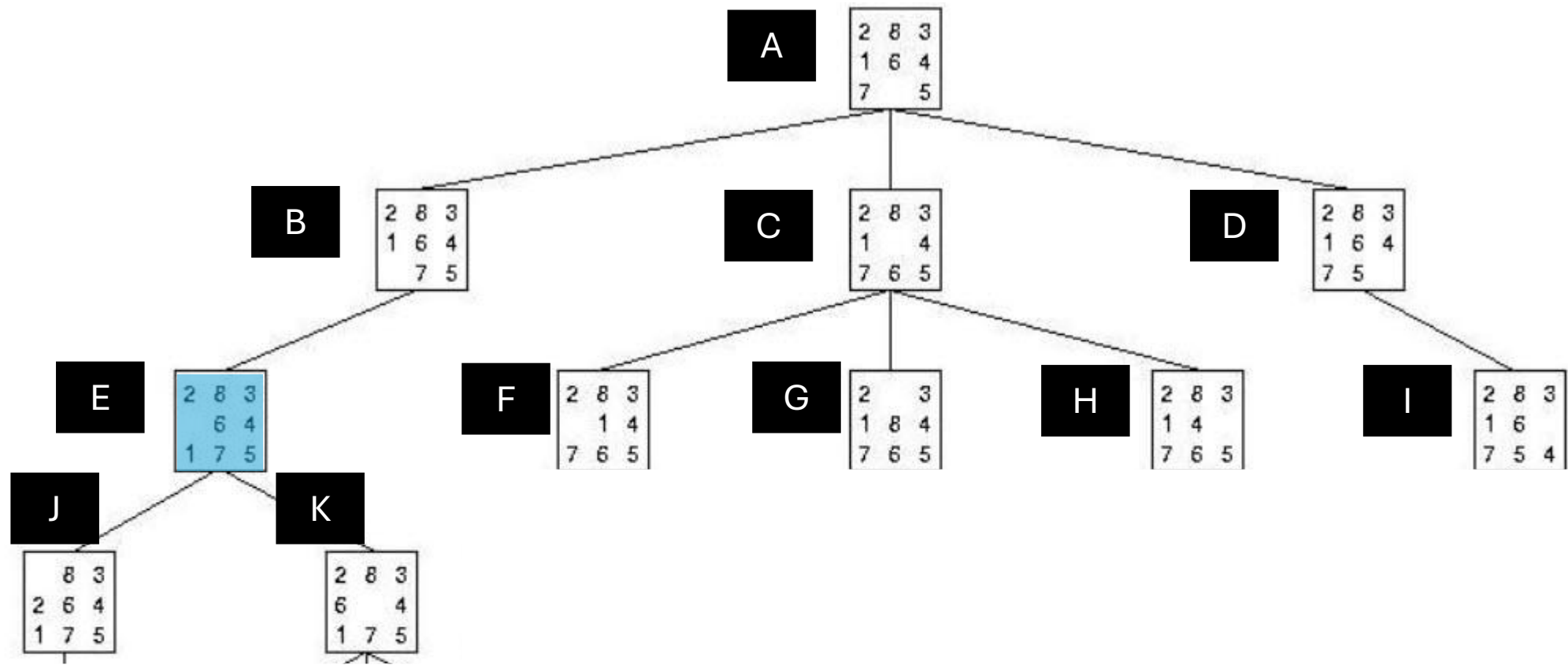
Order : A , B ,



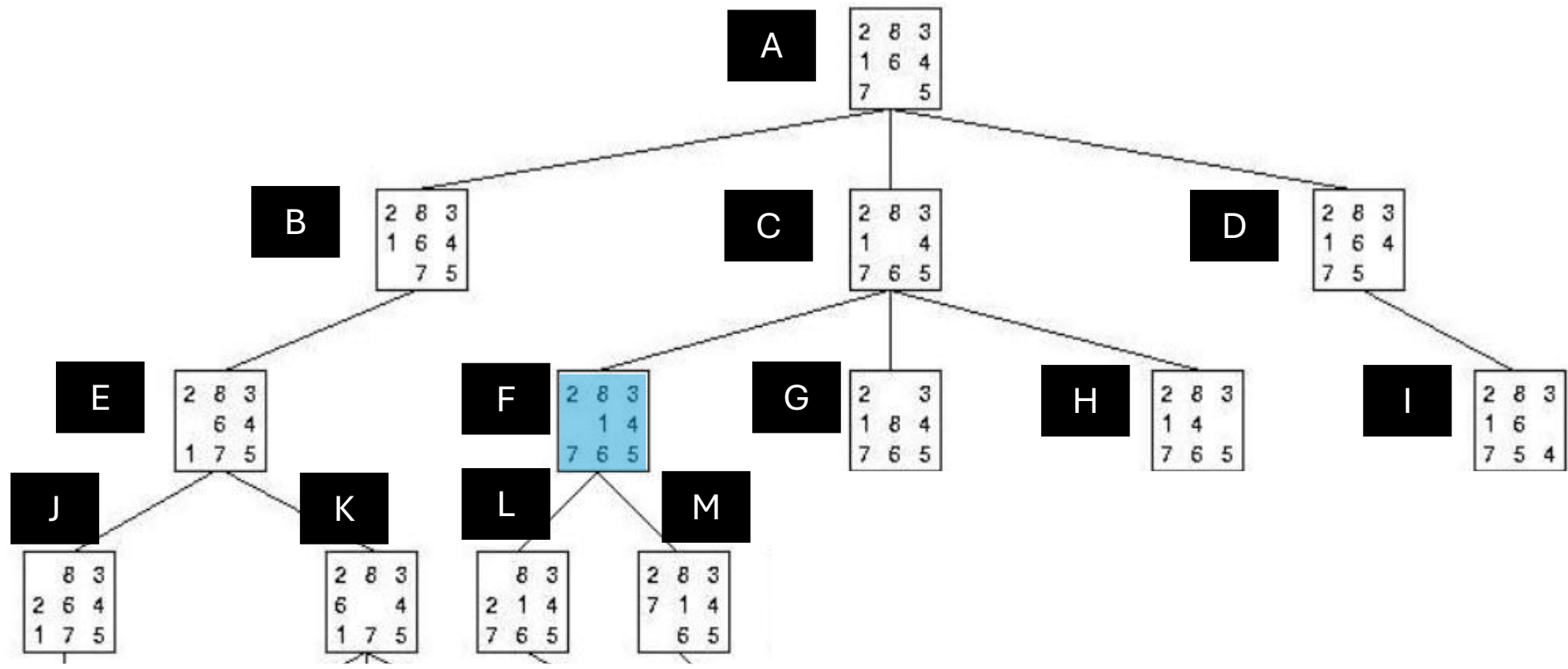
Order : A , B , C



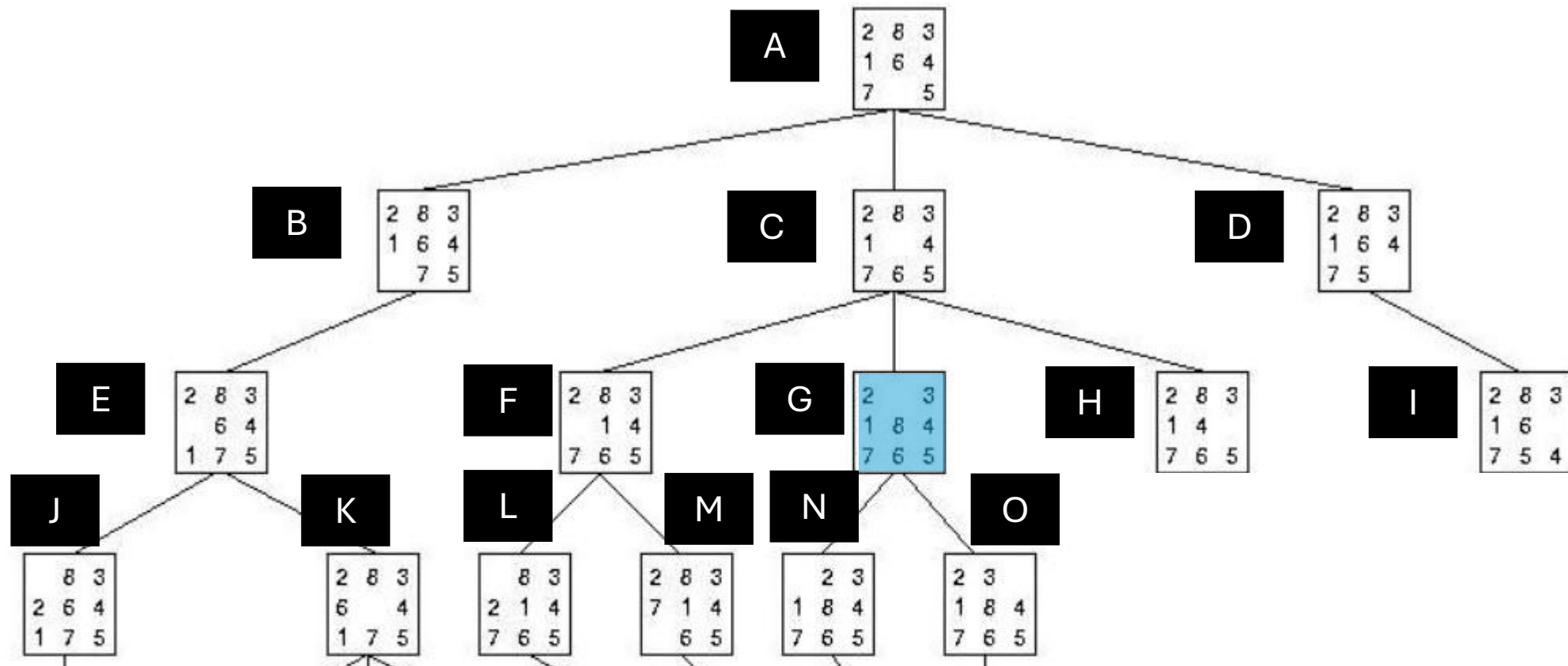
Order : A , B , C ,D



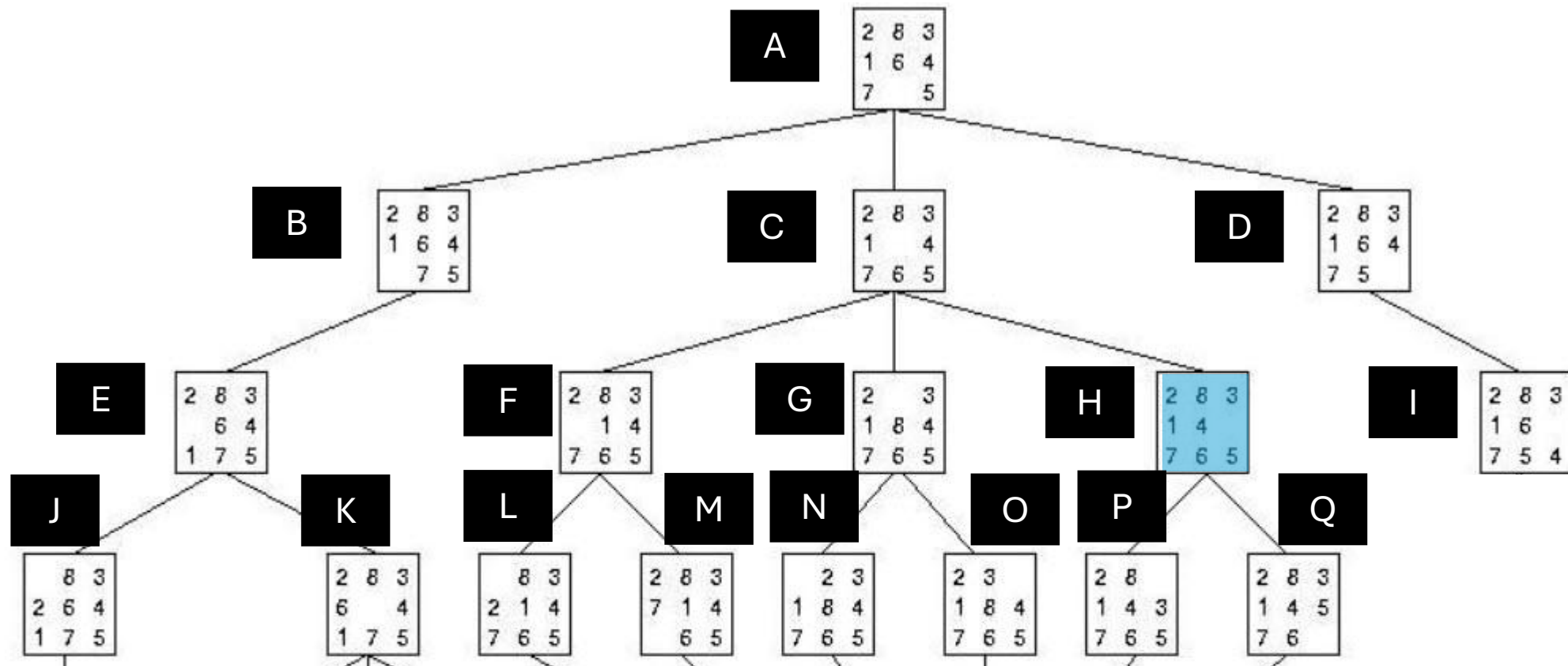
Order : A , B , C , D , E



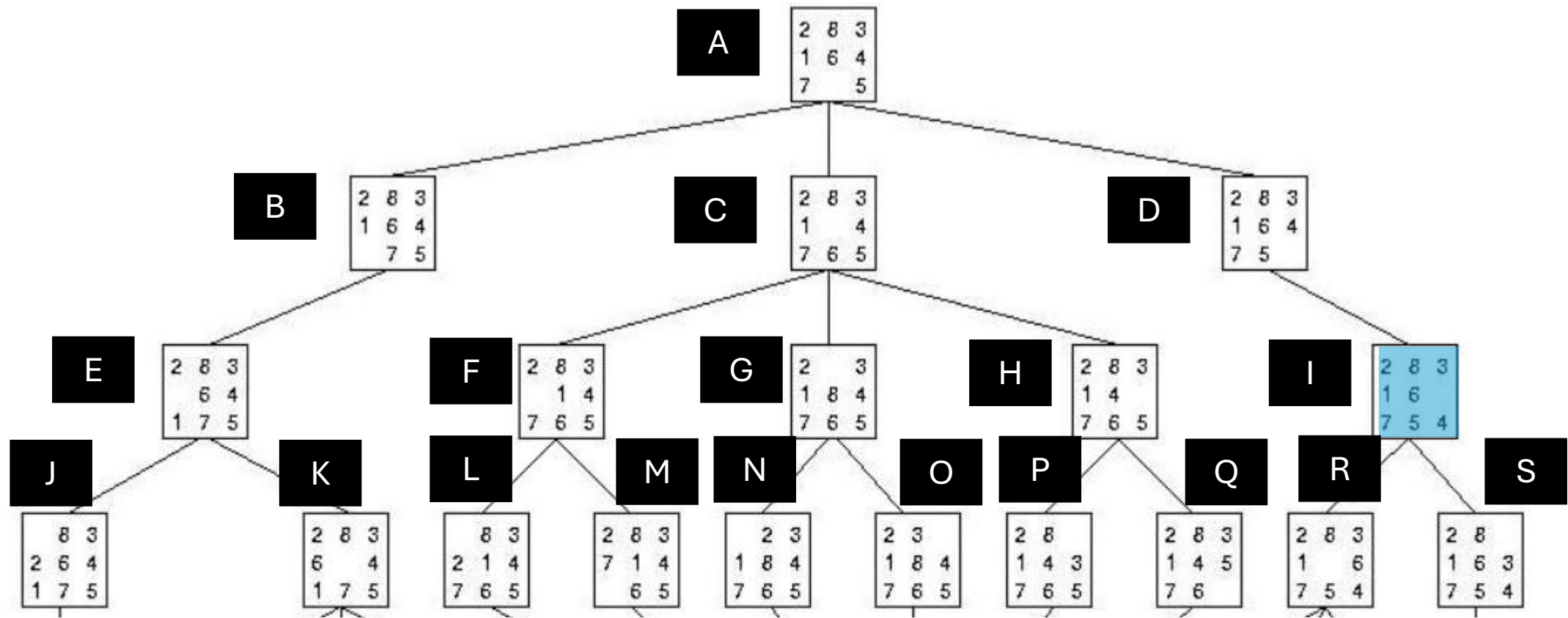
Order : A , B , C , D , E , F



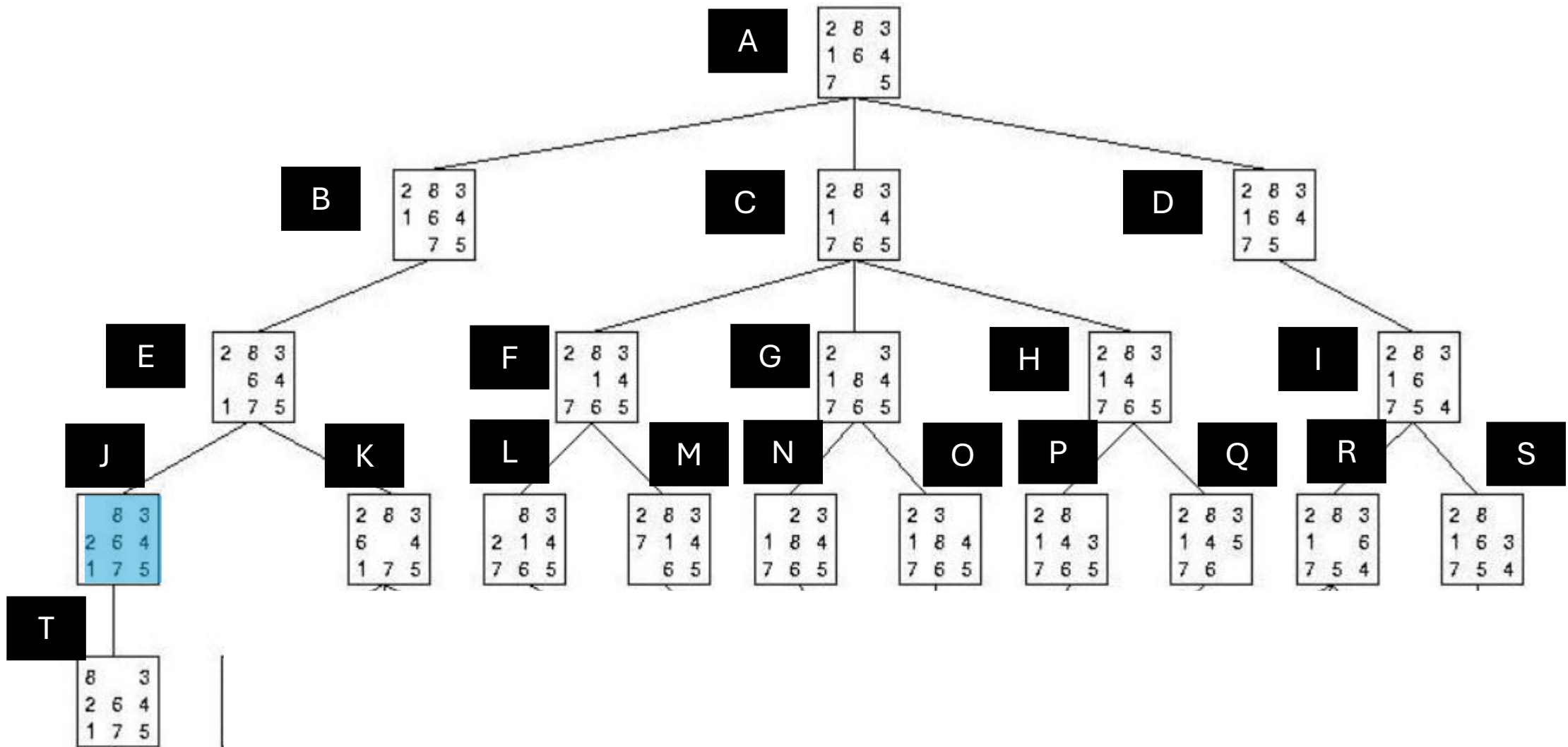
Order : A , B , C , D , E , F , G



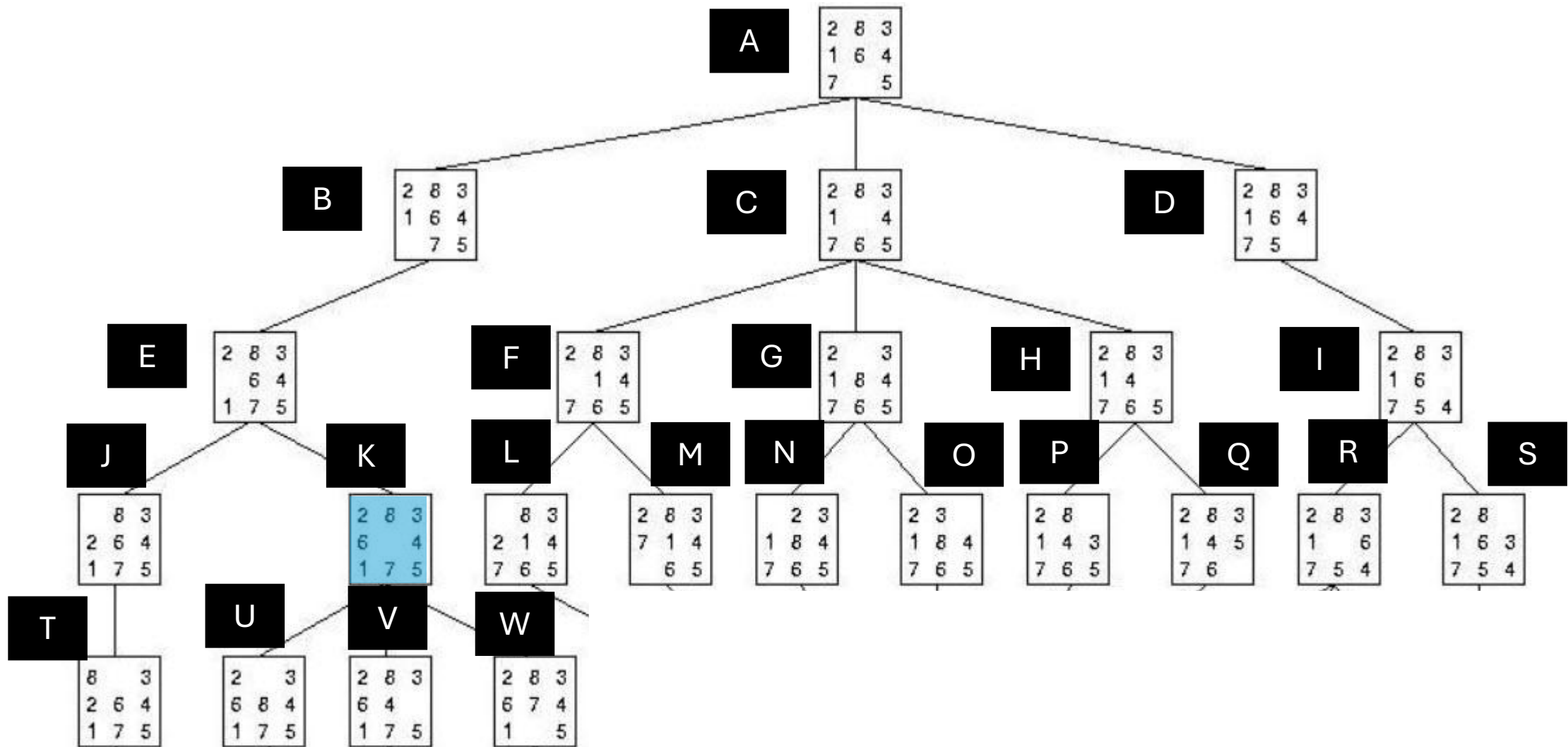
Order : A , B , C , D , E , F , G , H



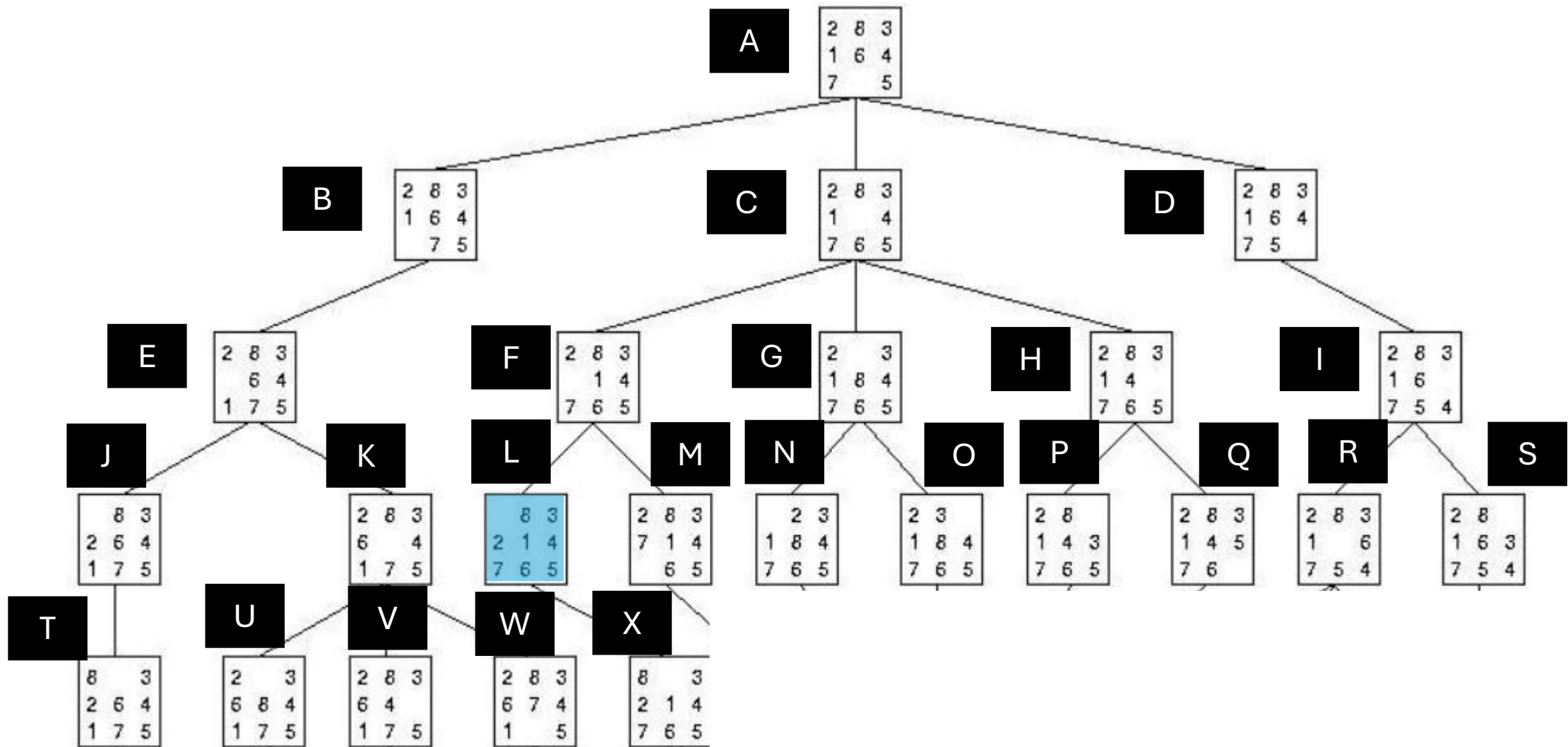
Order : A , B , C , D , E , F , G , H , I



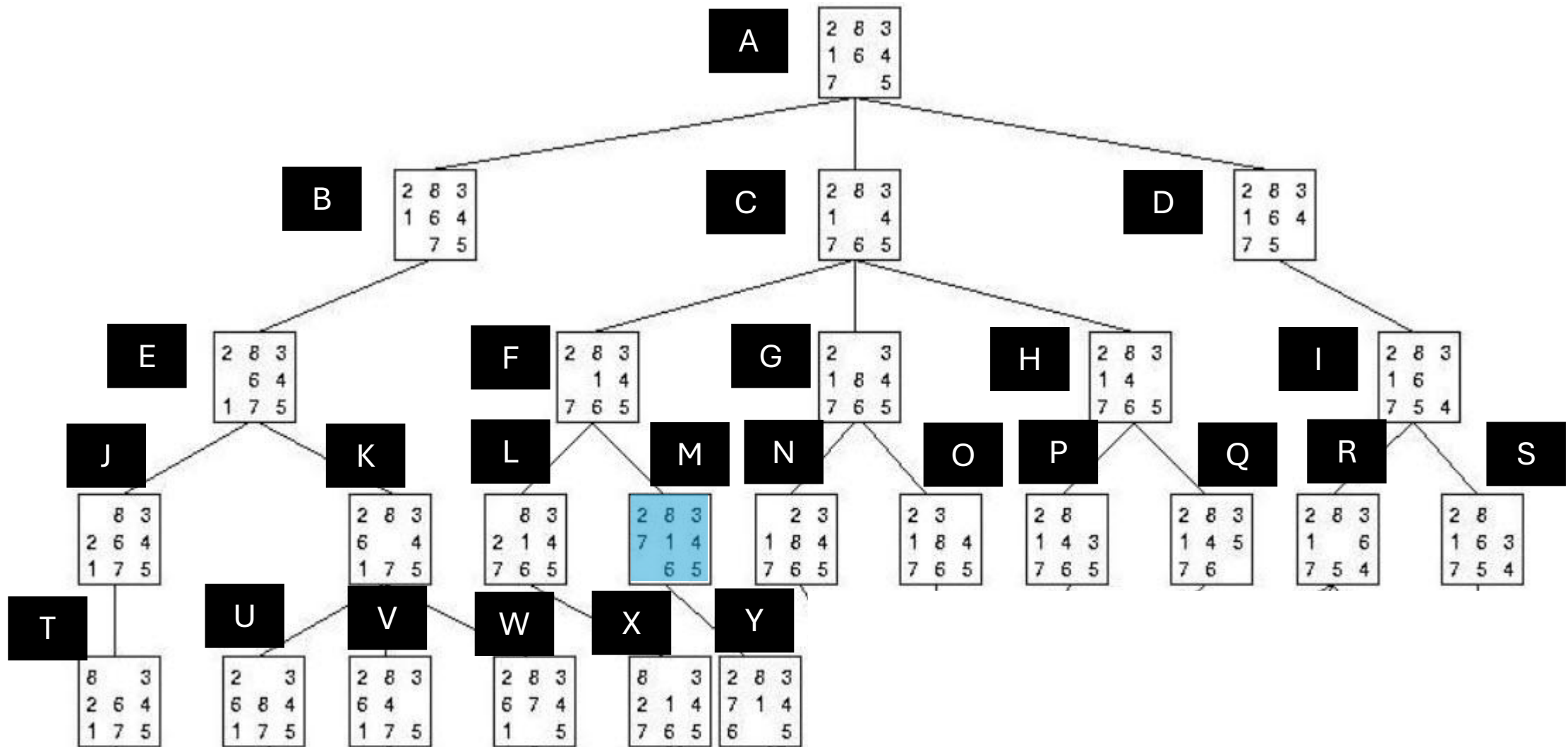
Order : A , B , C , D , E , F , G , H , I , J



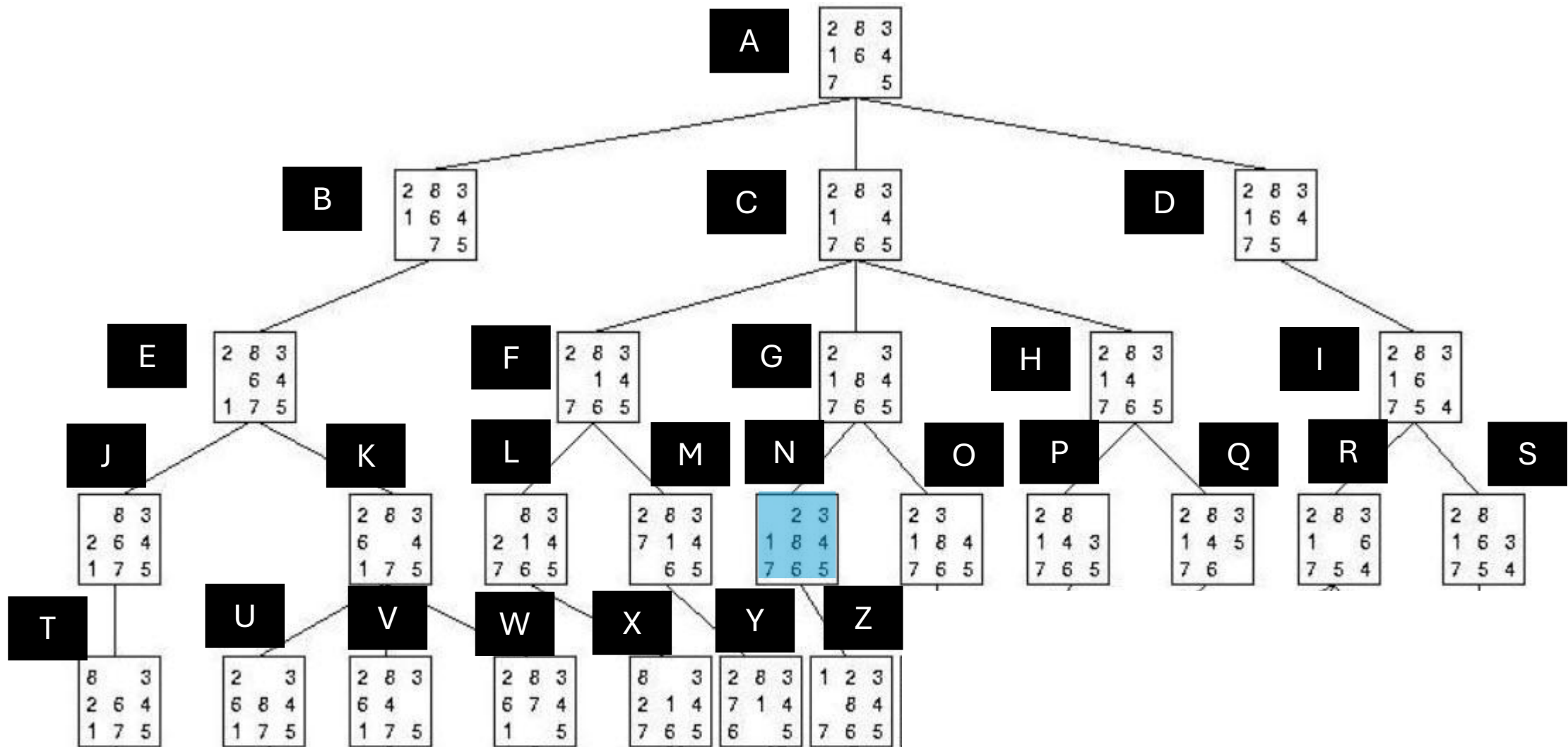
Order : A , B , C , D , E , F , G , H , I , J , K



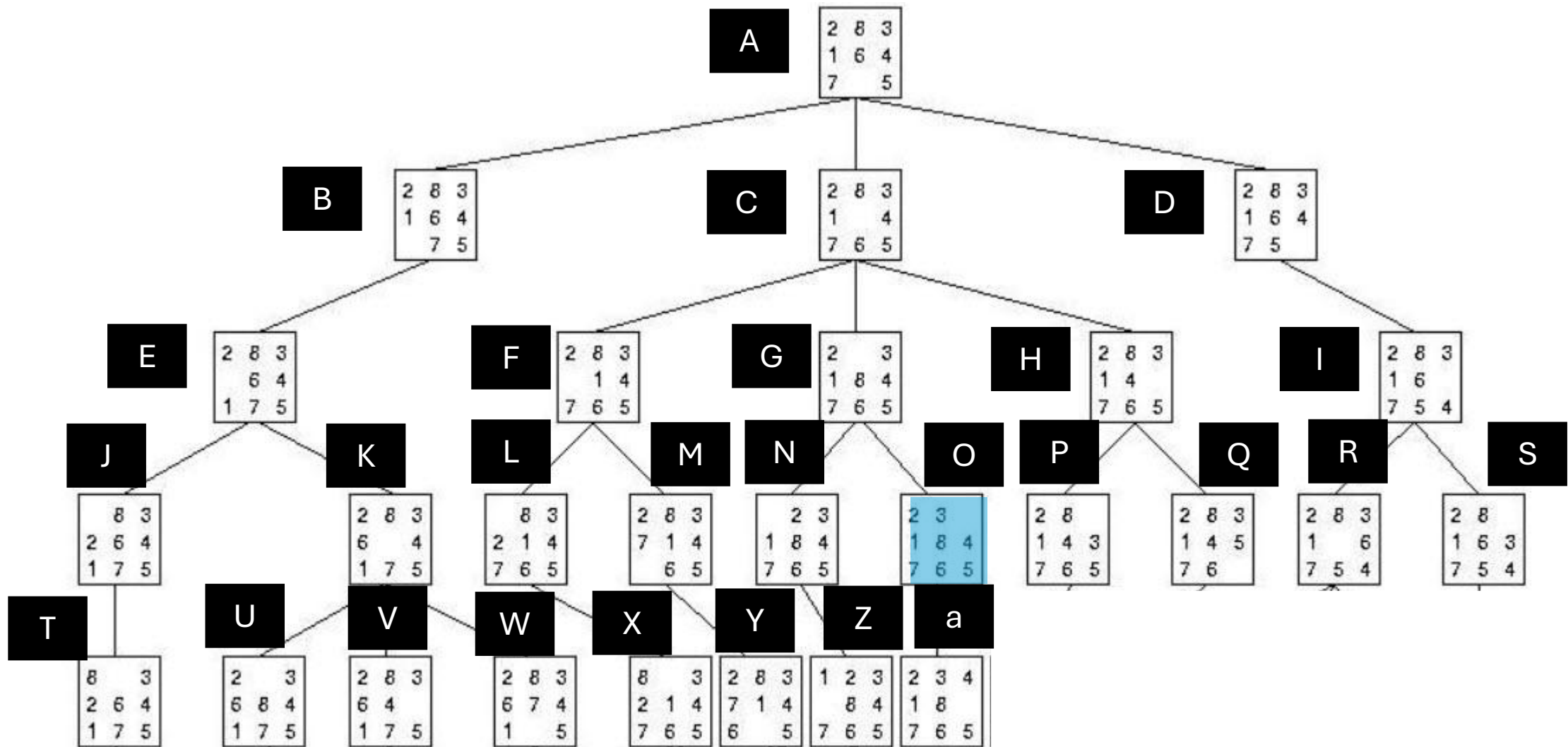
Order : A , B , C , D , E , F , G , H , I , J , K , L



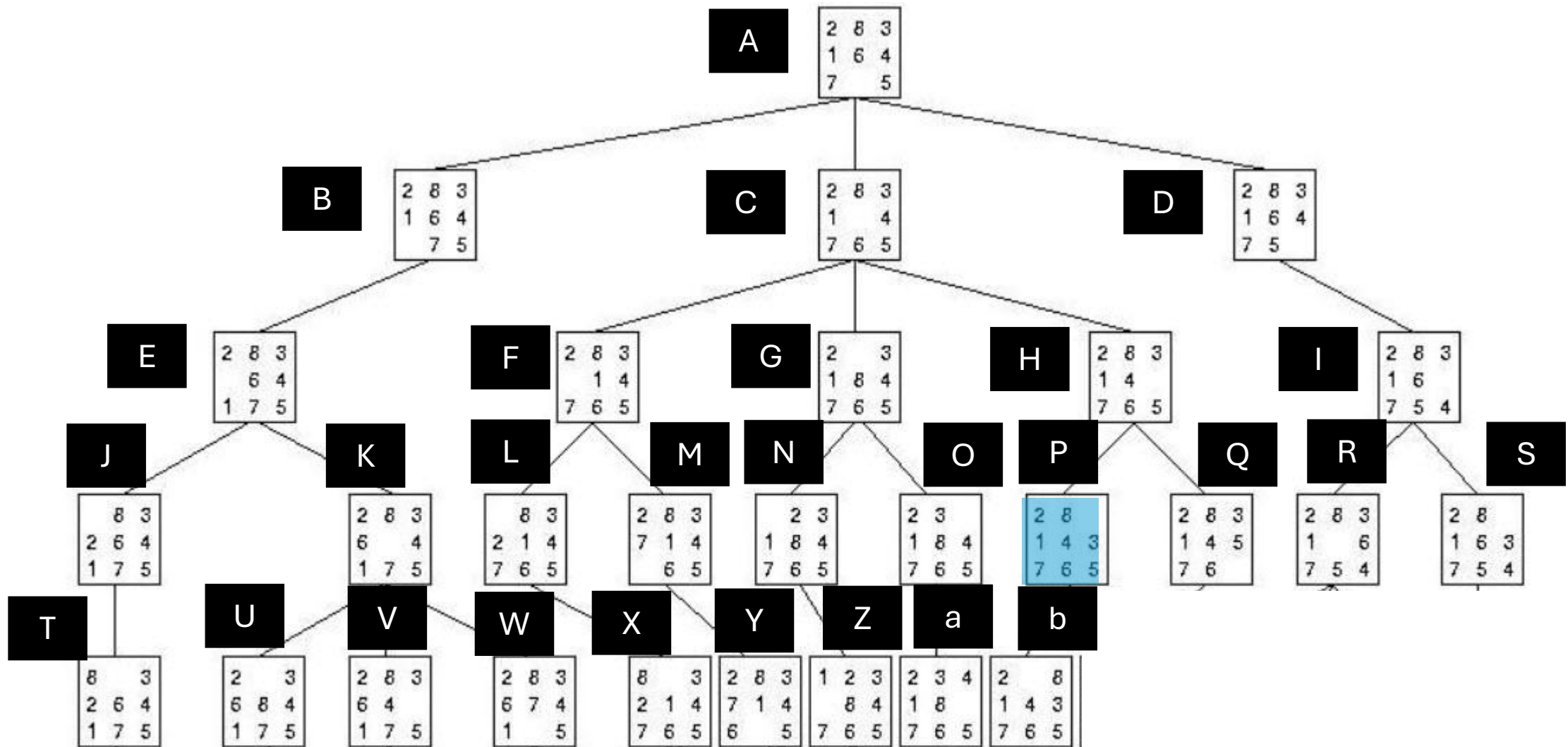
Order : A , B , C , D , E , F , G , H , I , J , K , L , M



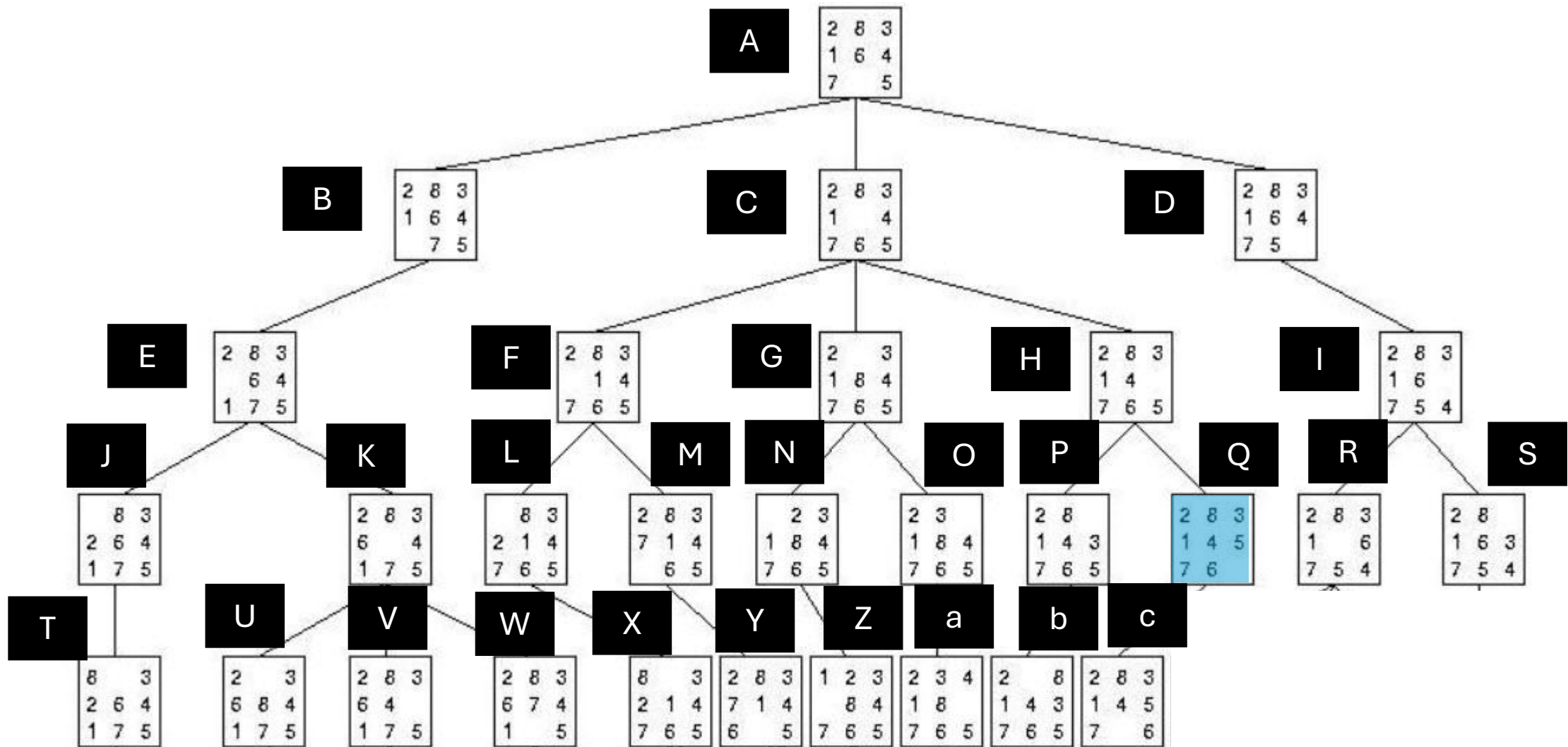
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N



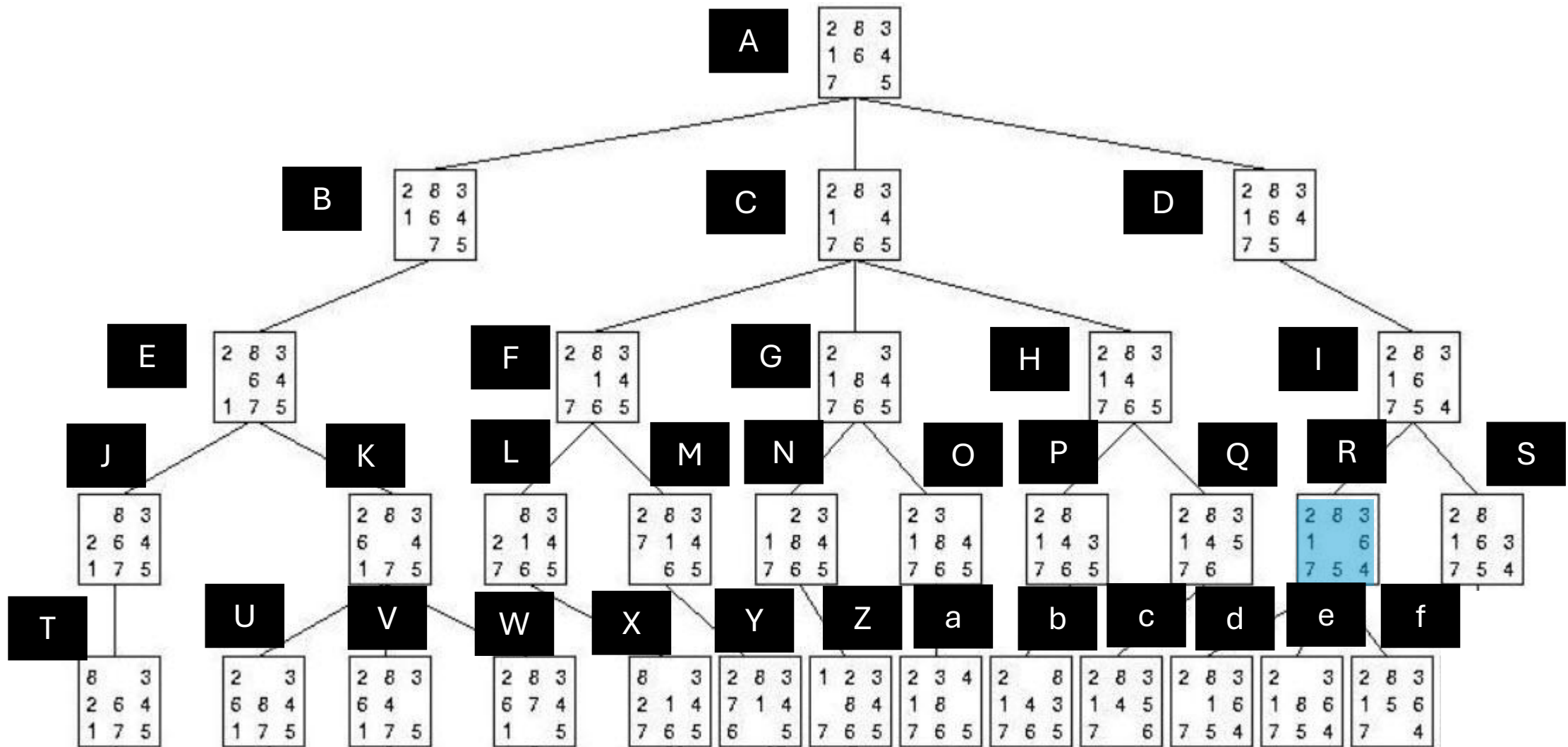
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O



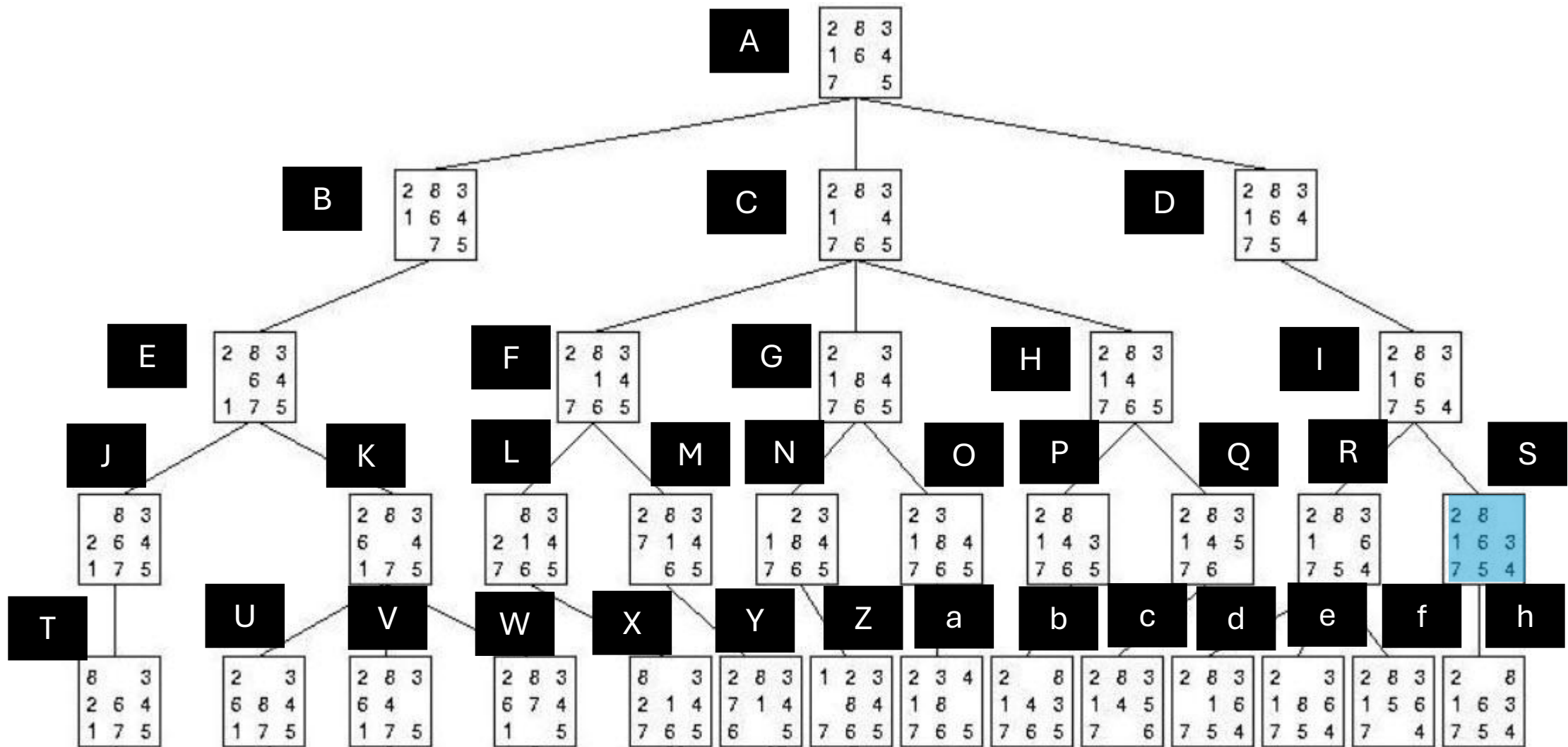
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P



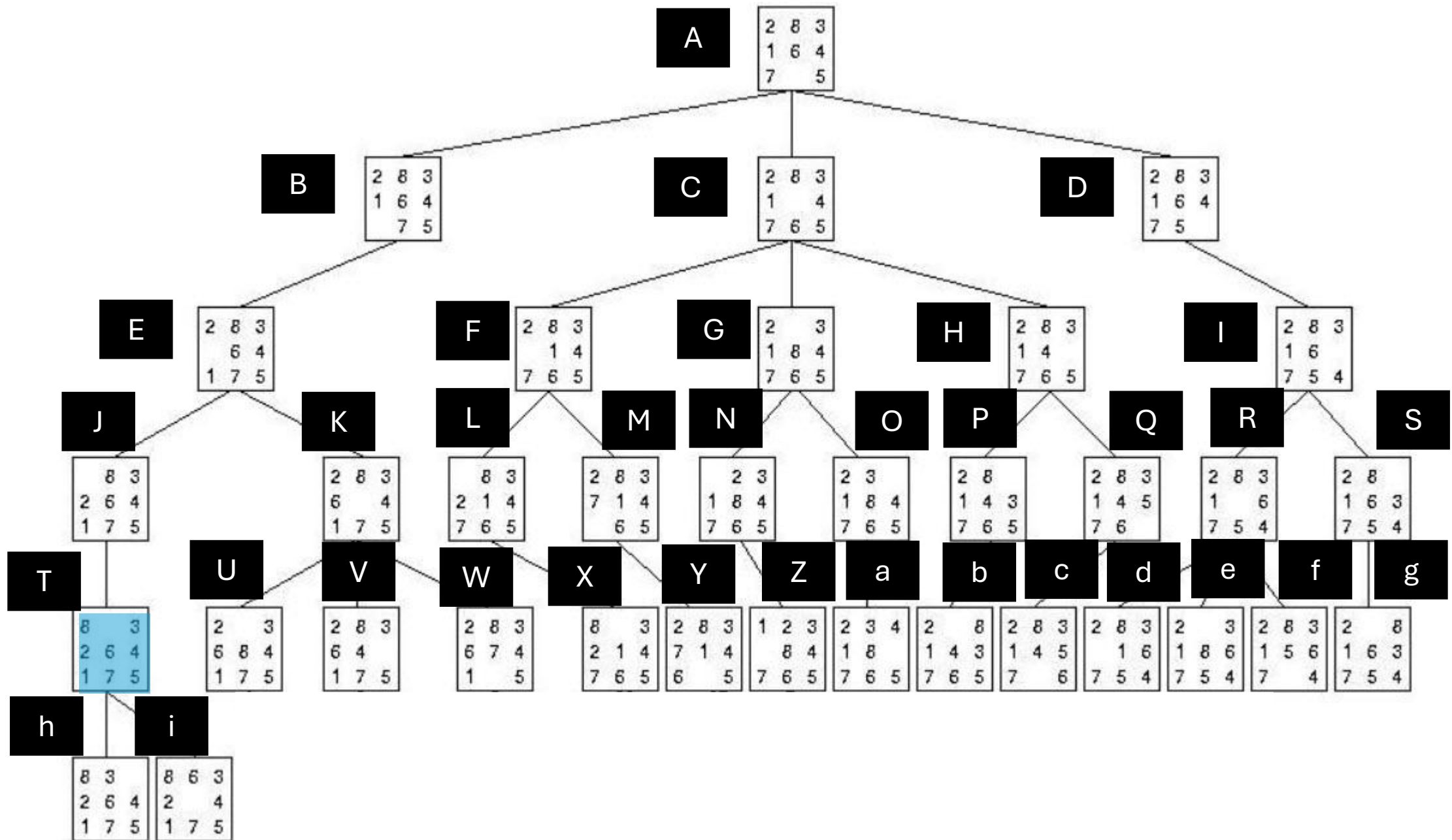
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q



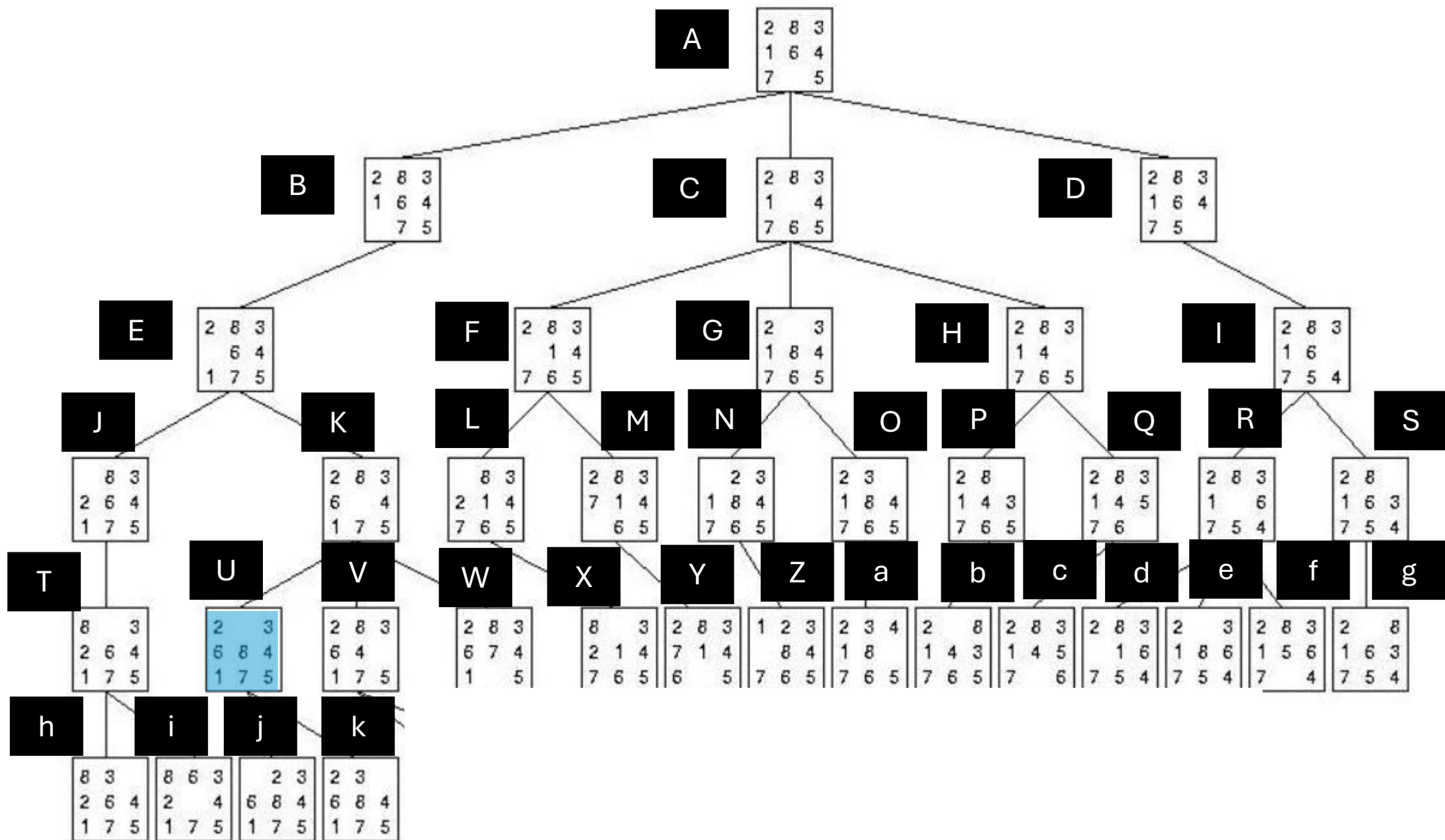
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R



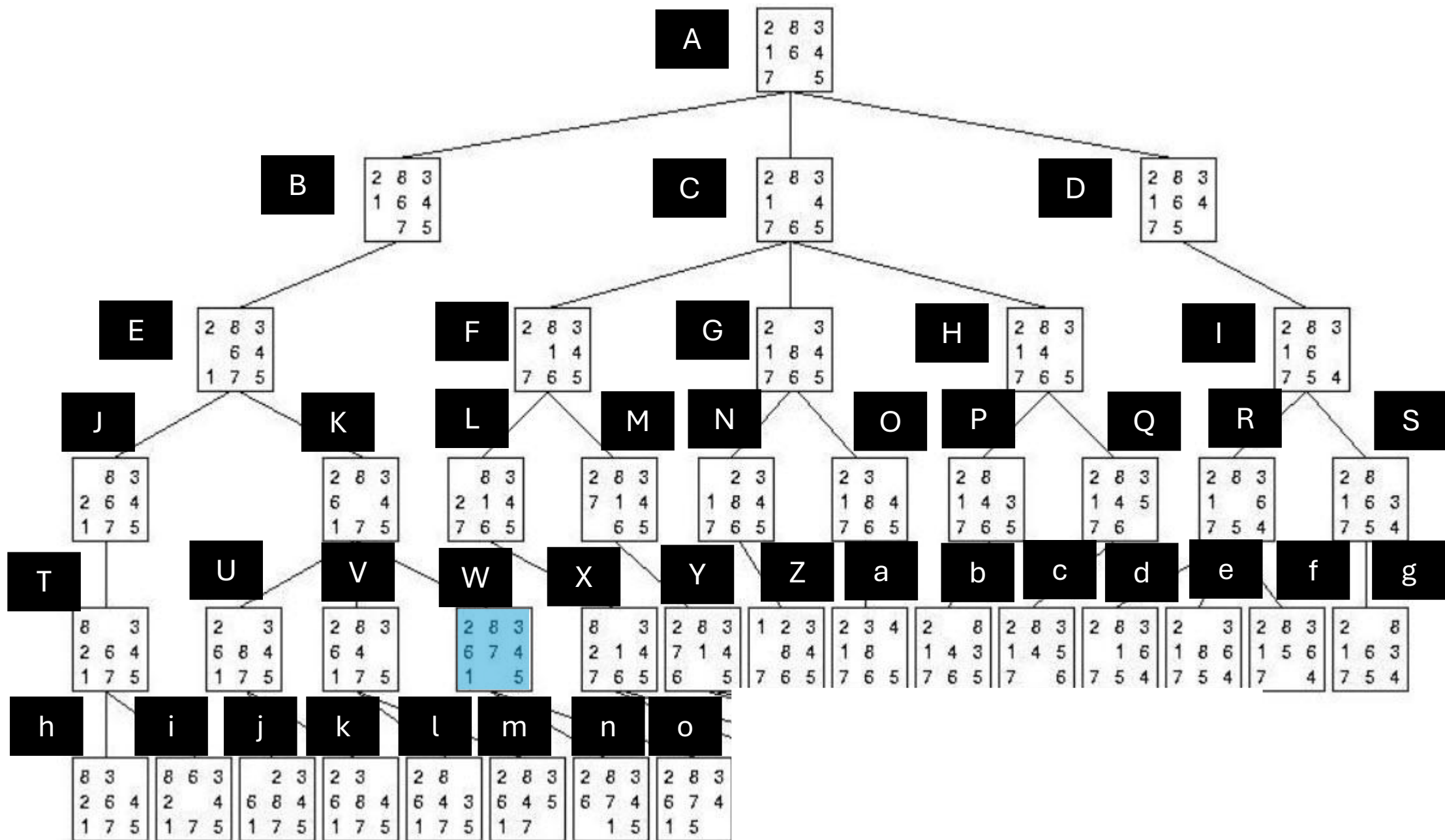
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S



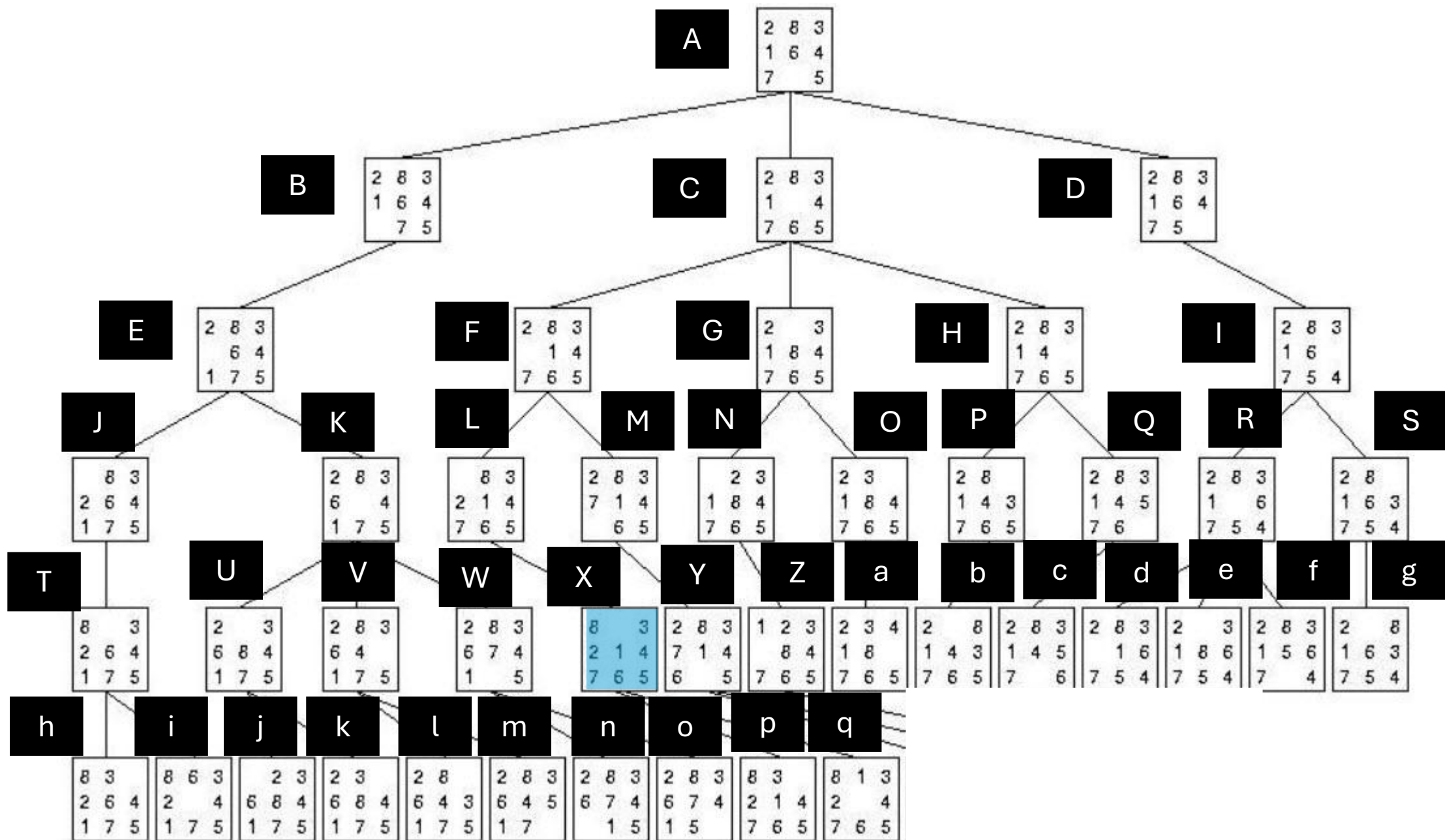
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , T



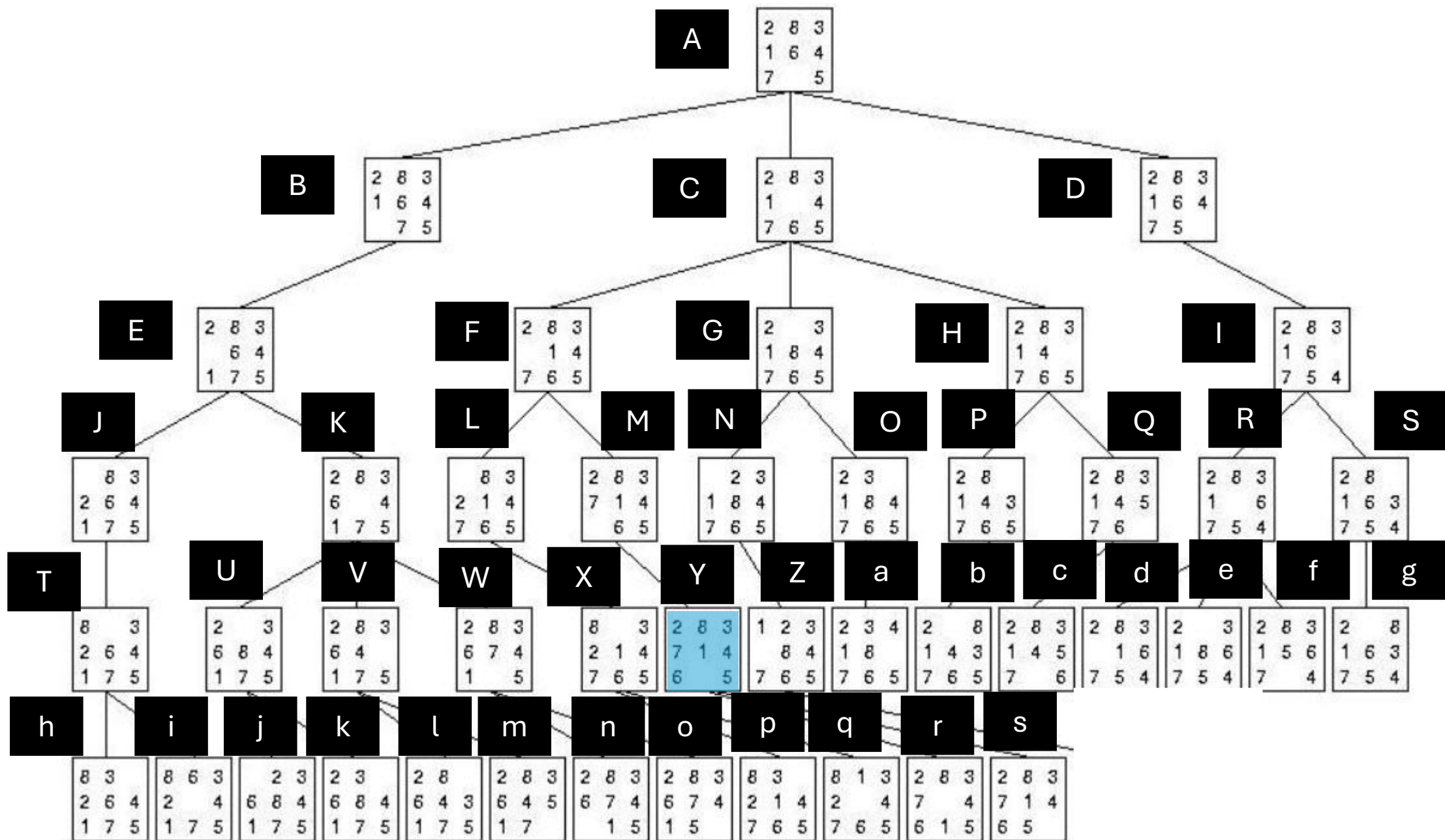
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U



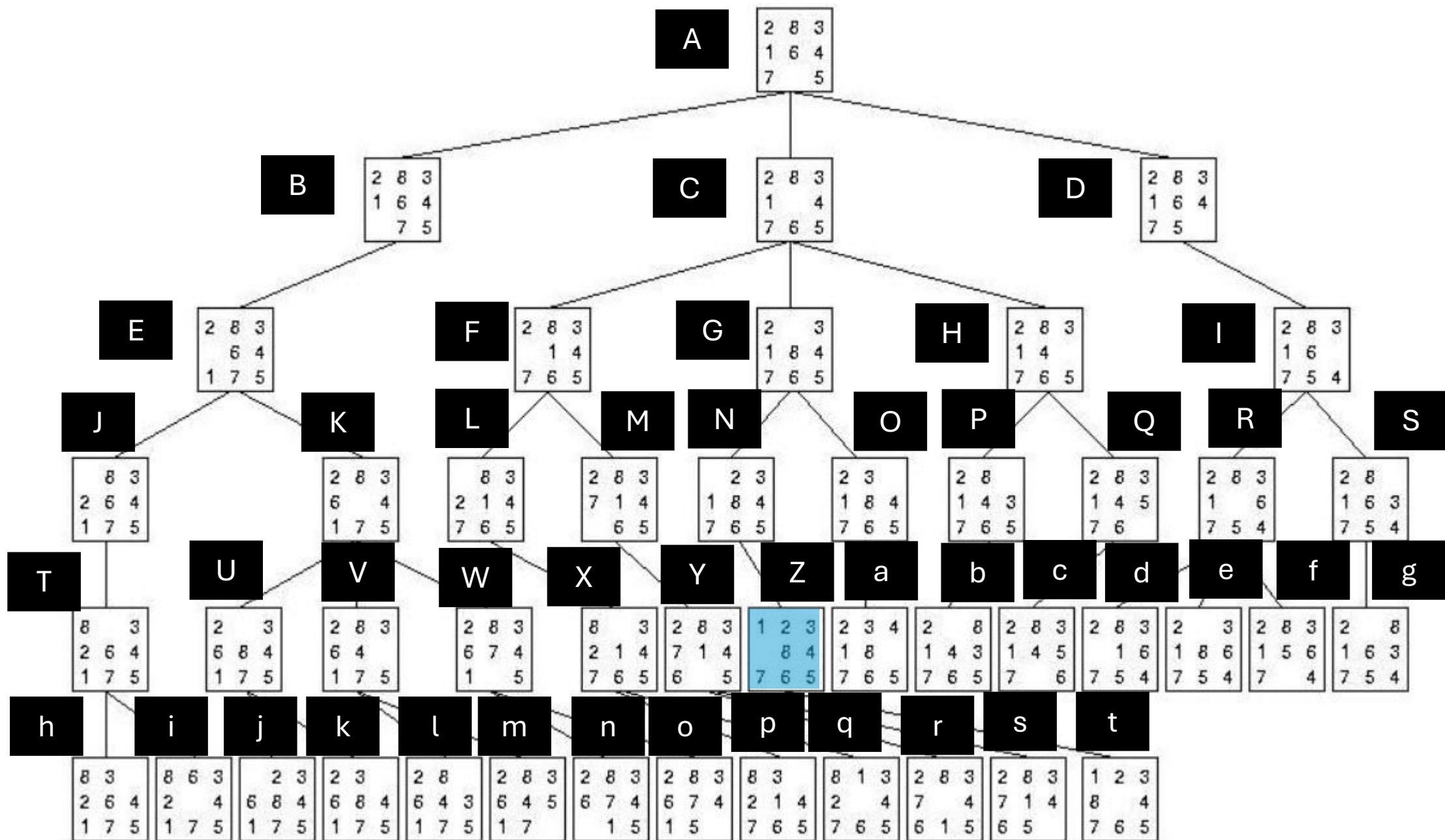
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W



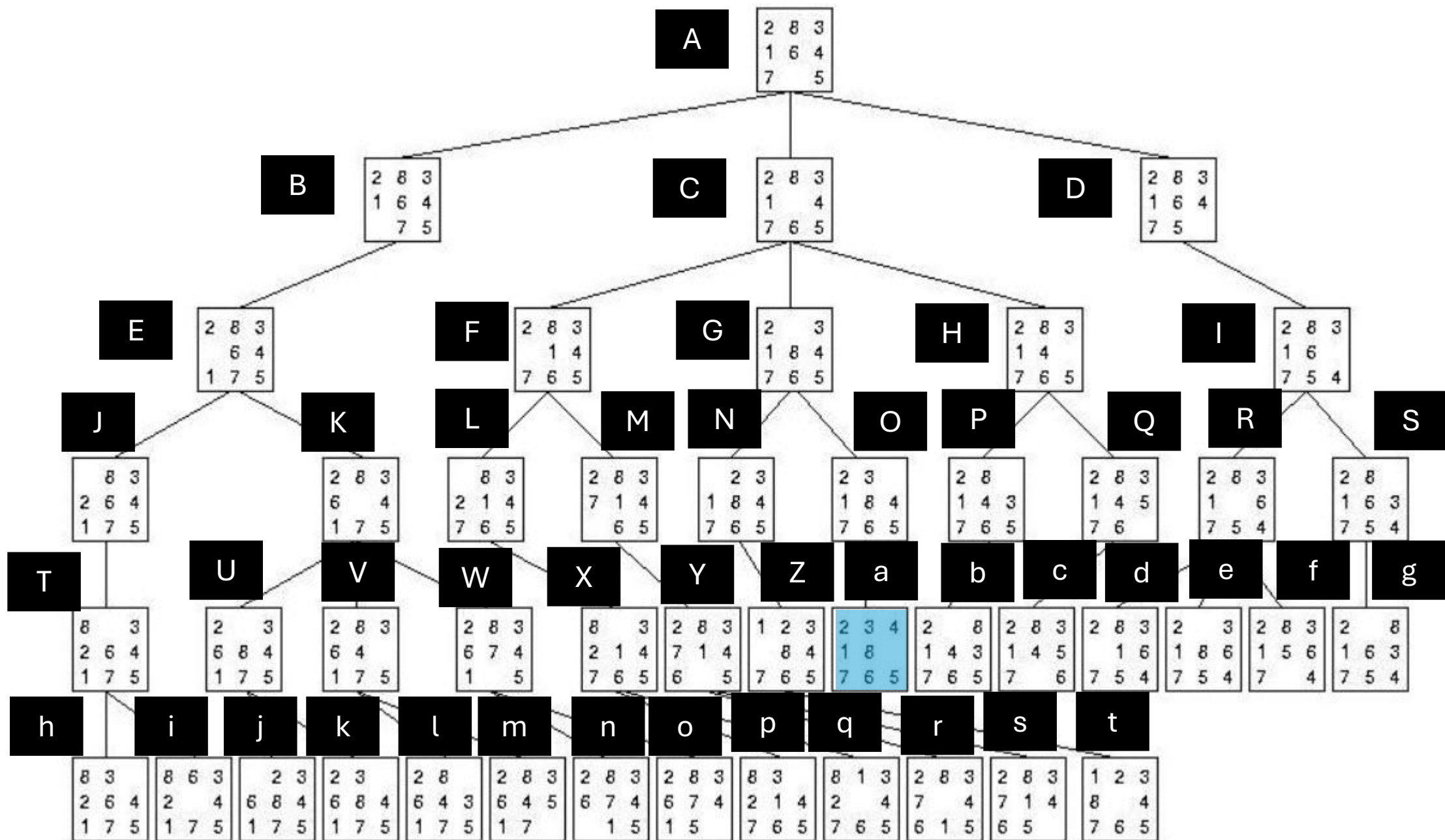
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X



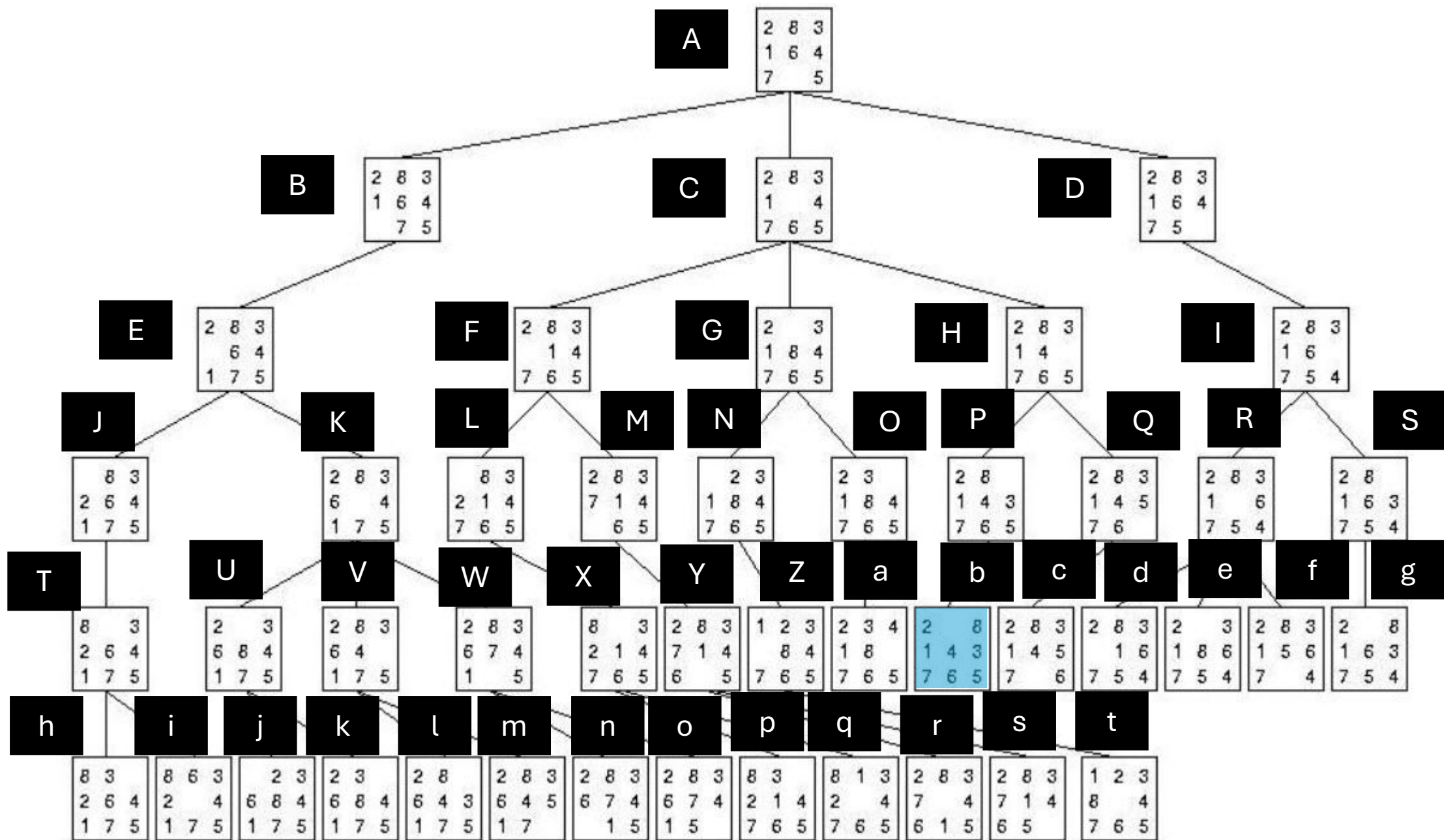
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y



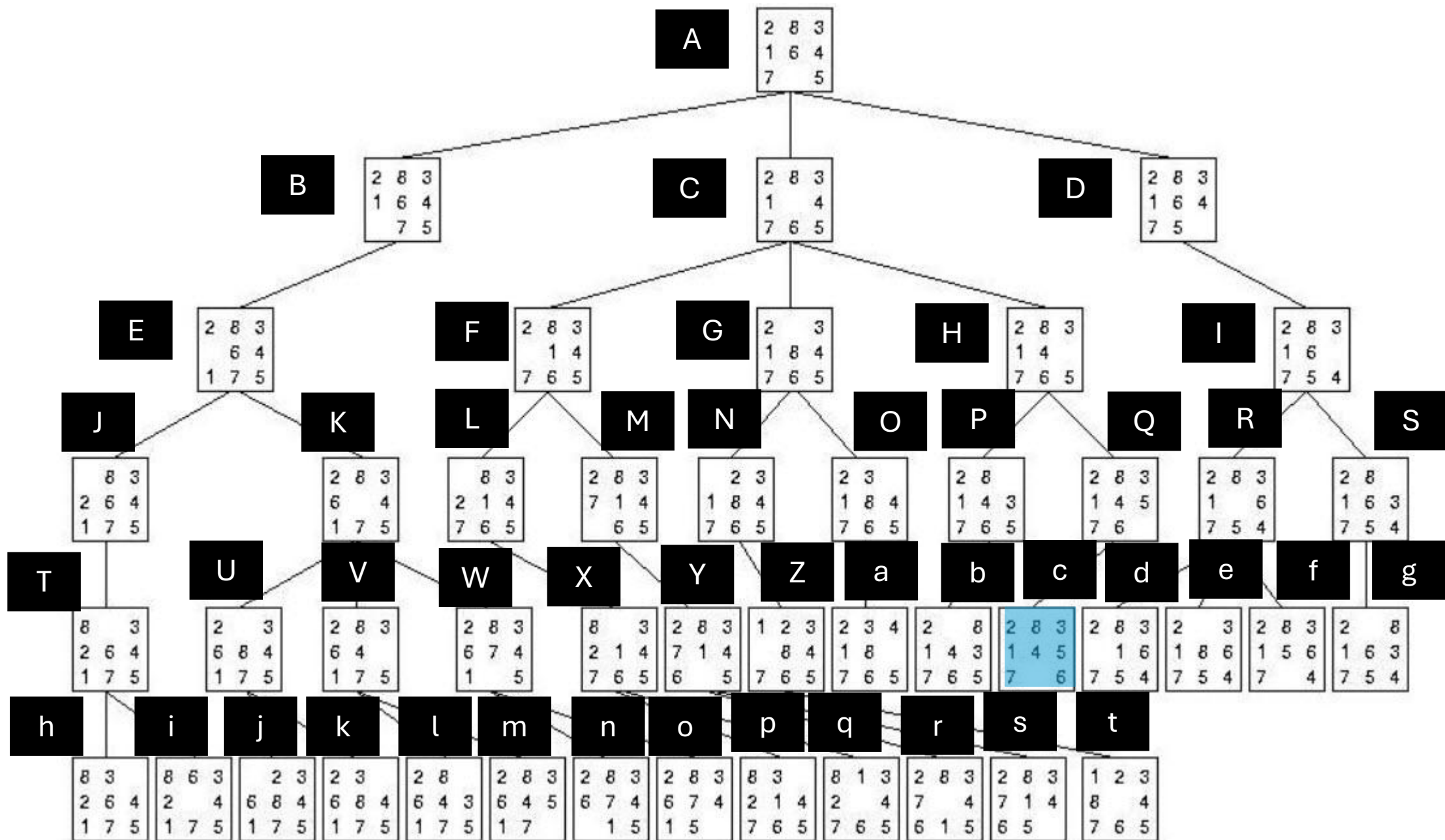
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z



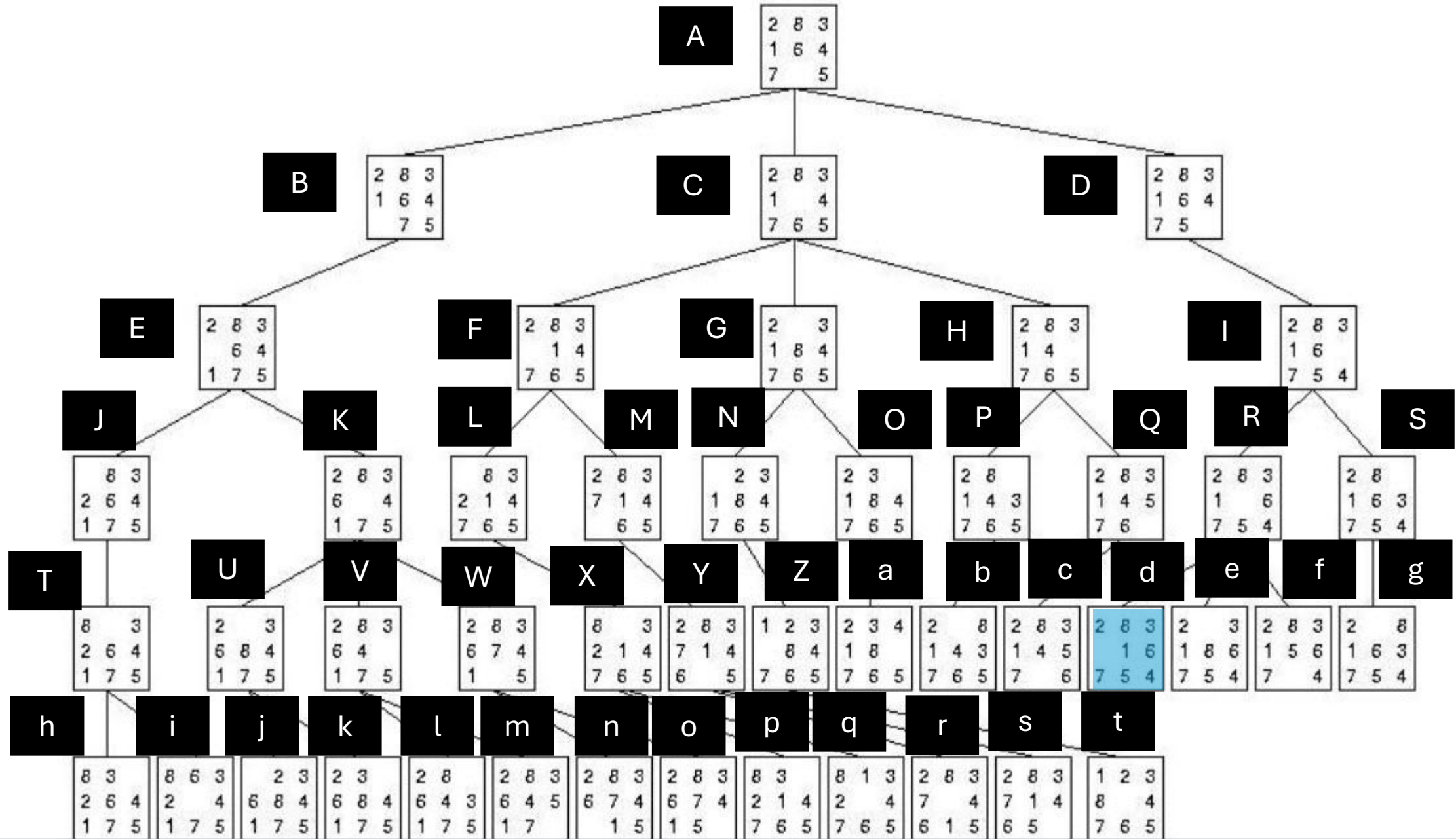
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a



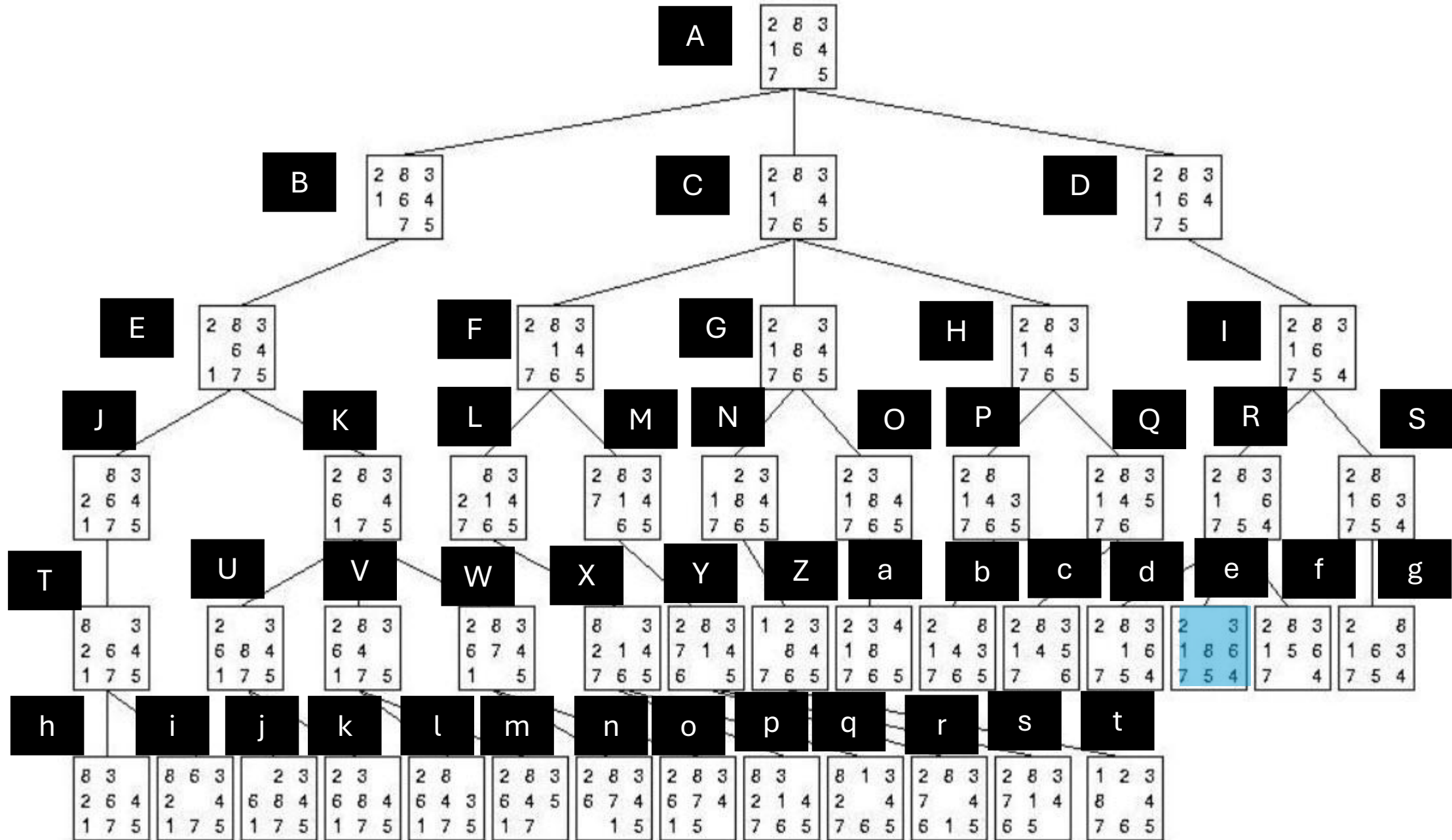
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b



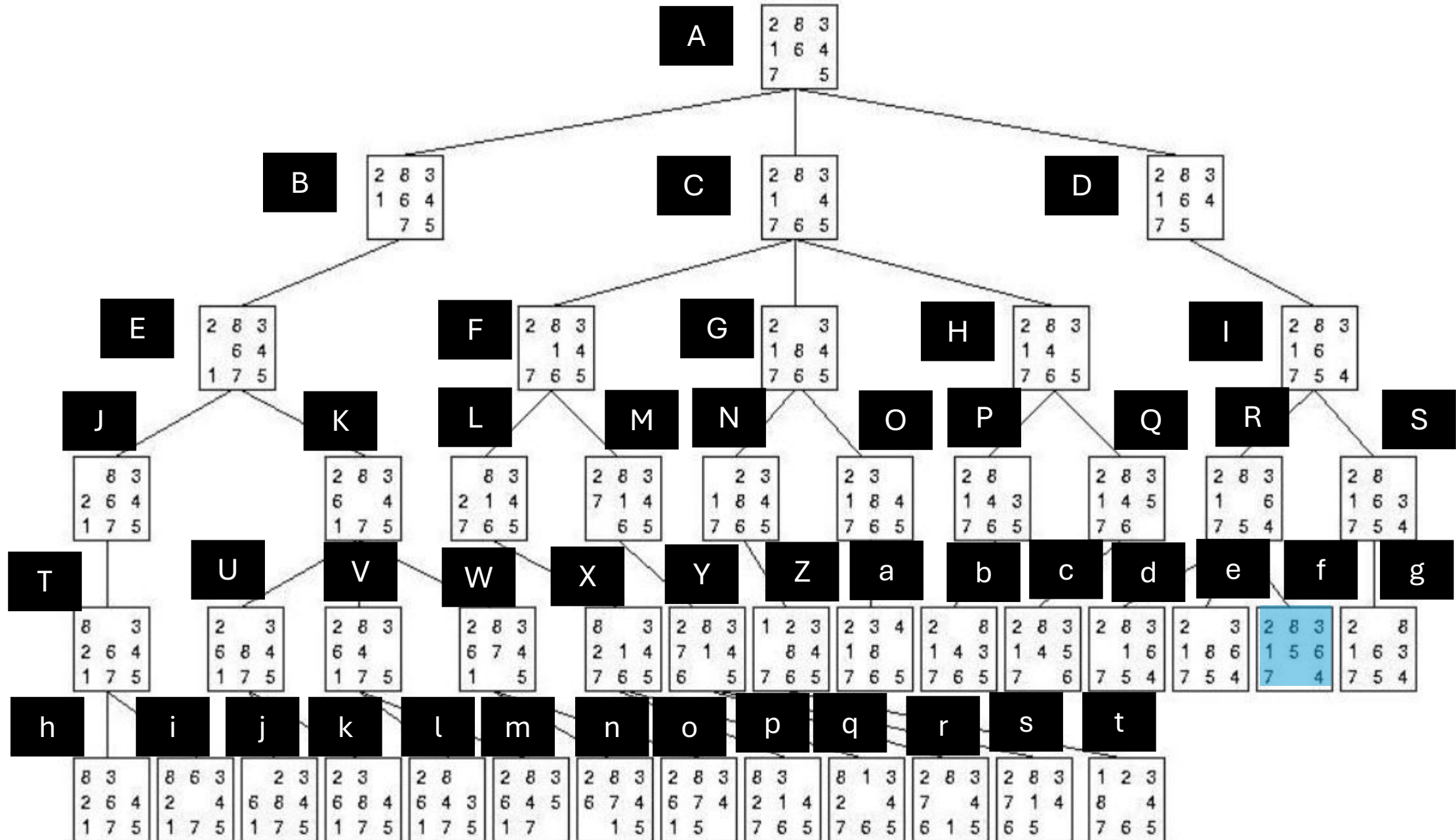
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c



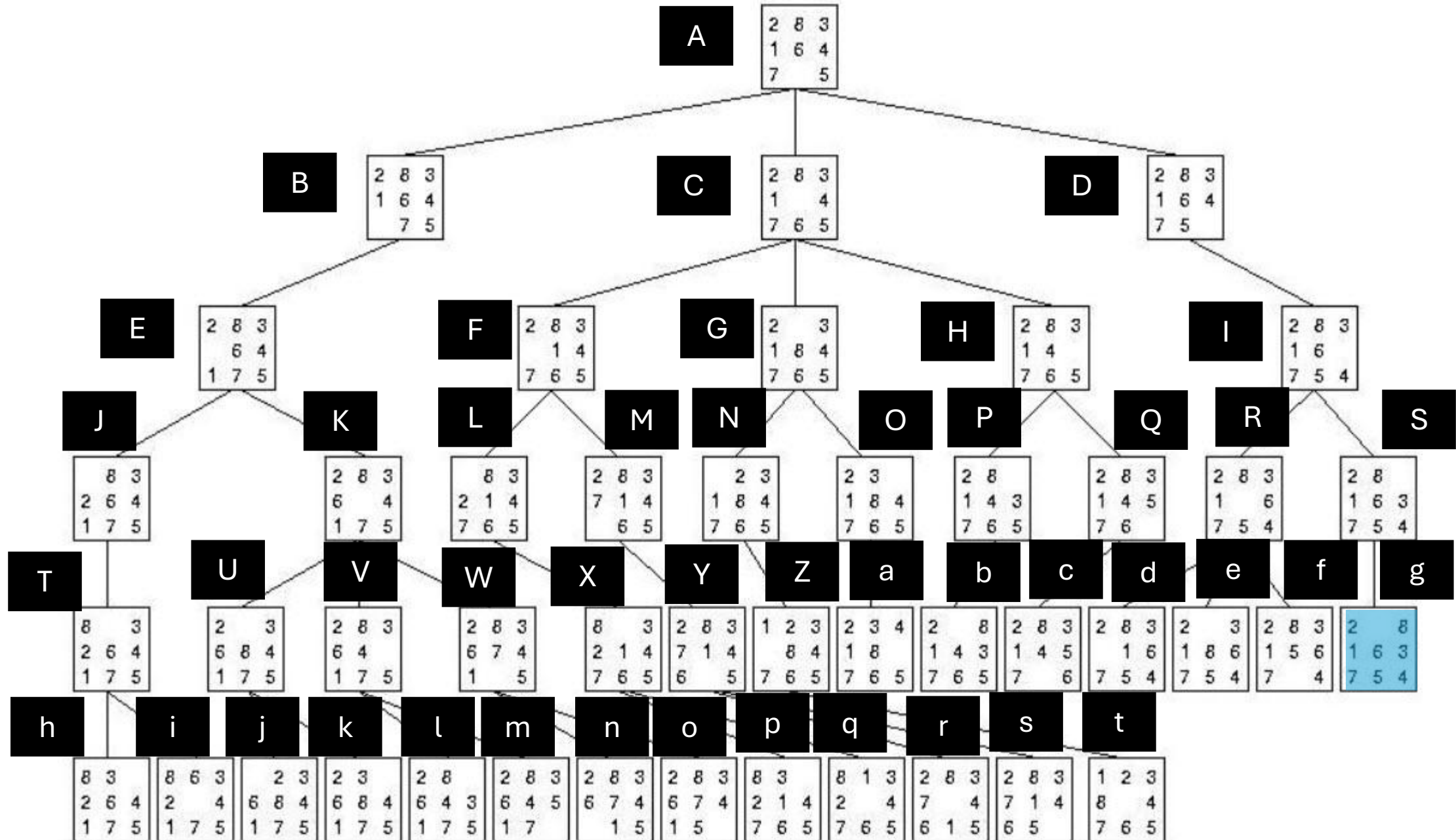
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d



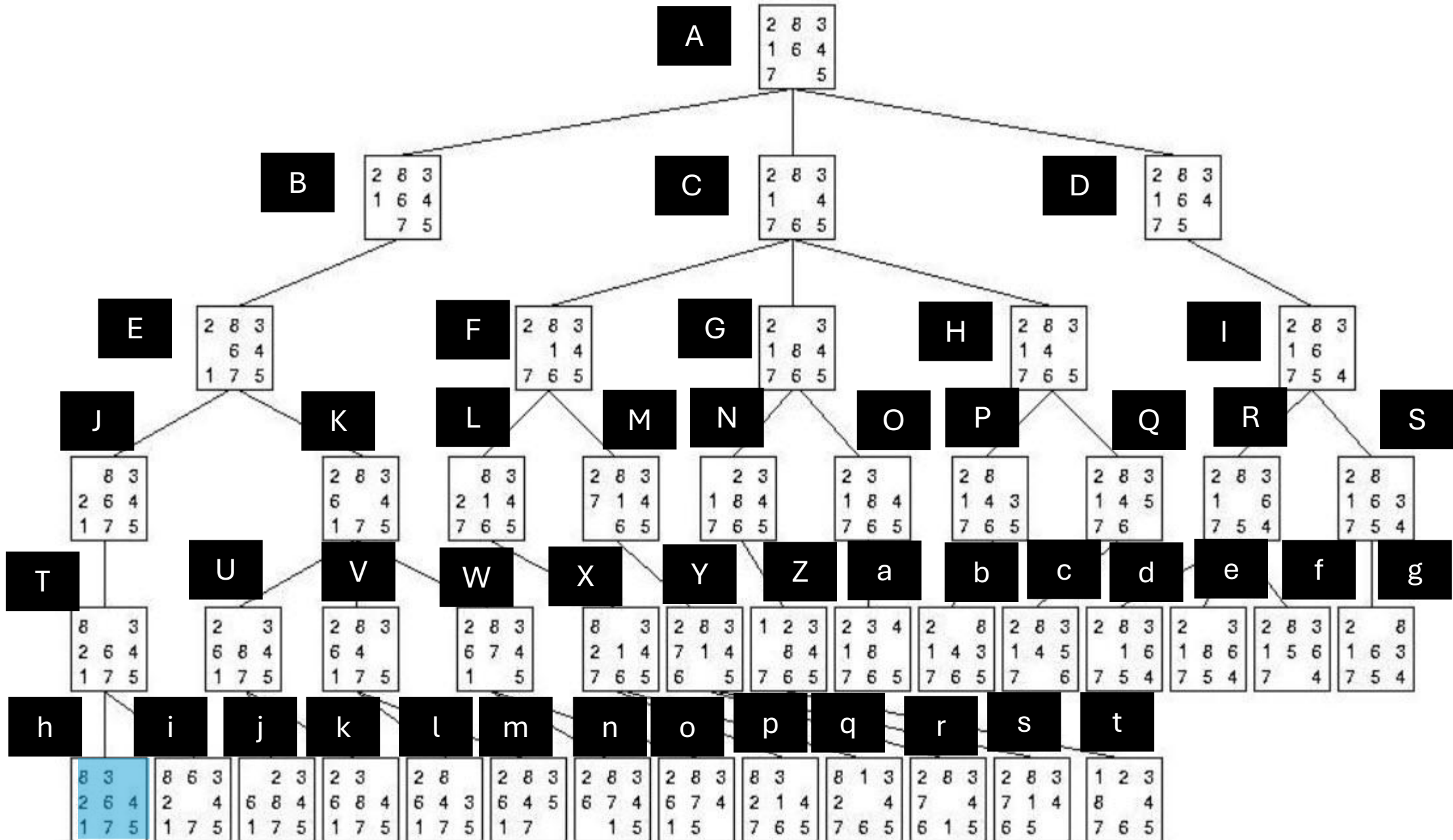
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e

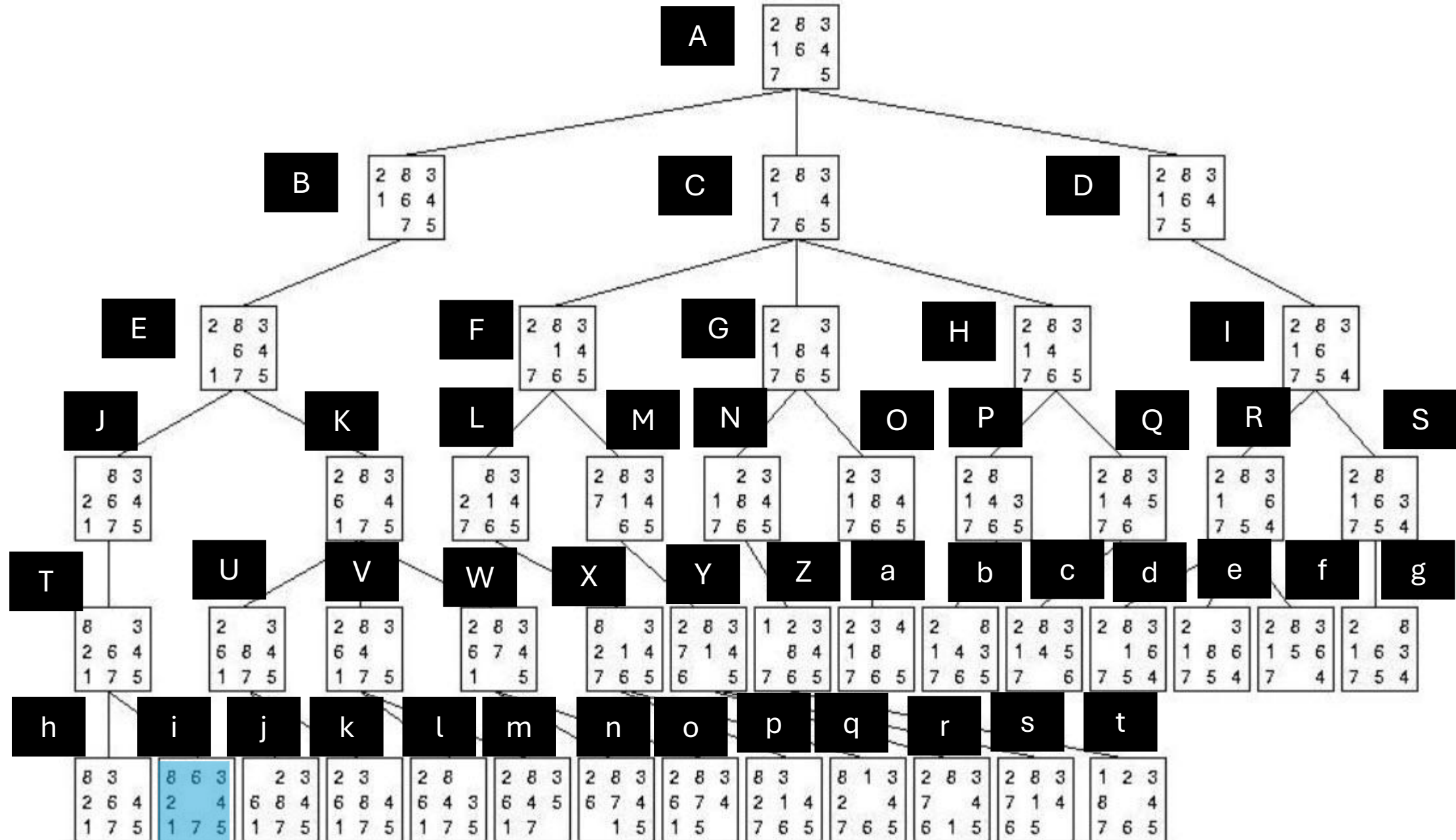


Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e , f

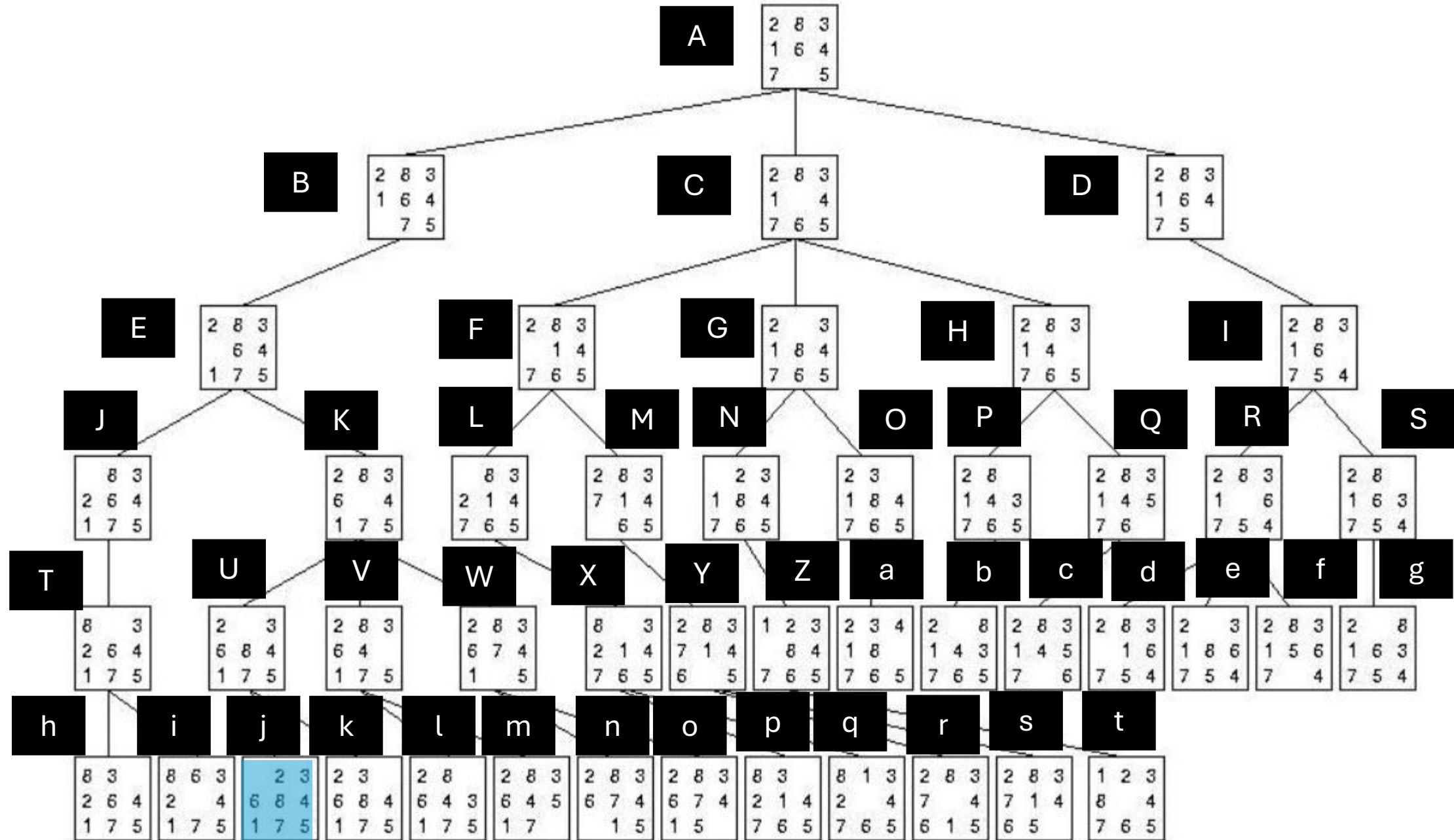


Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e , f , g

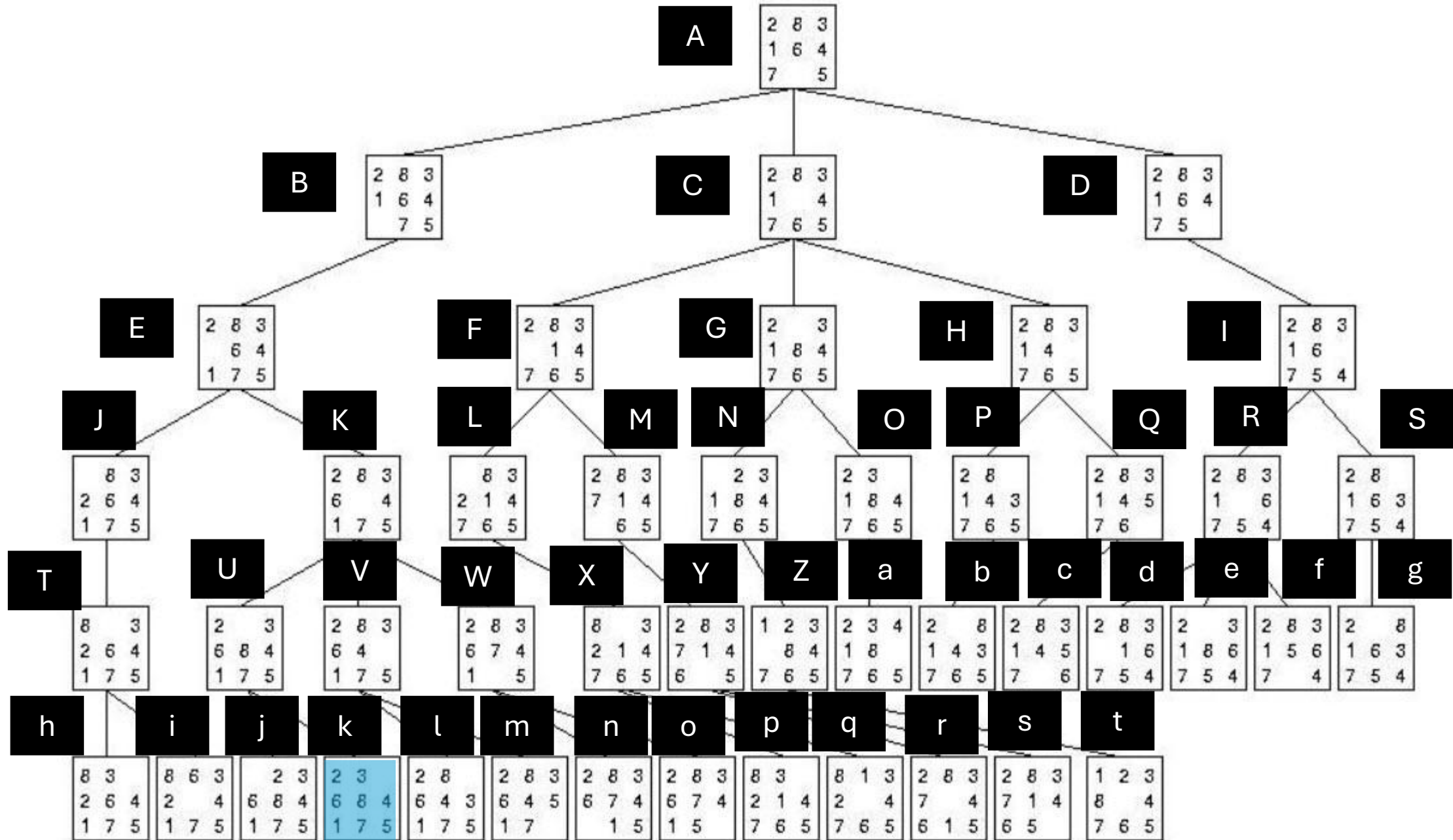




Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e , f , g , h , i

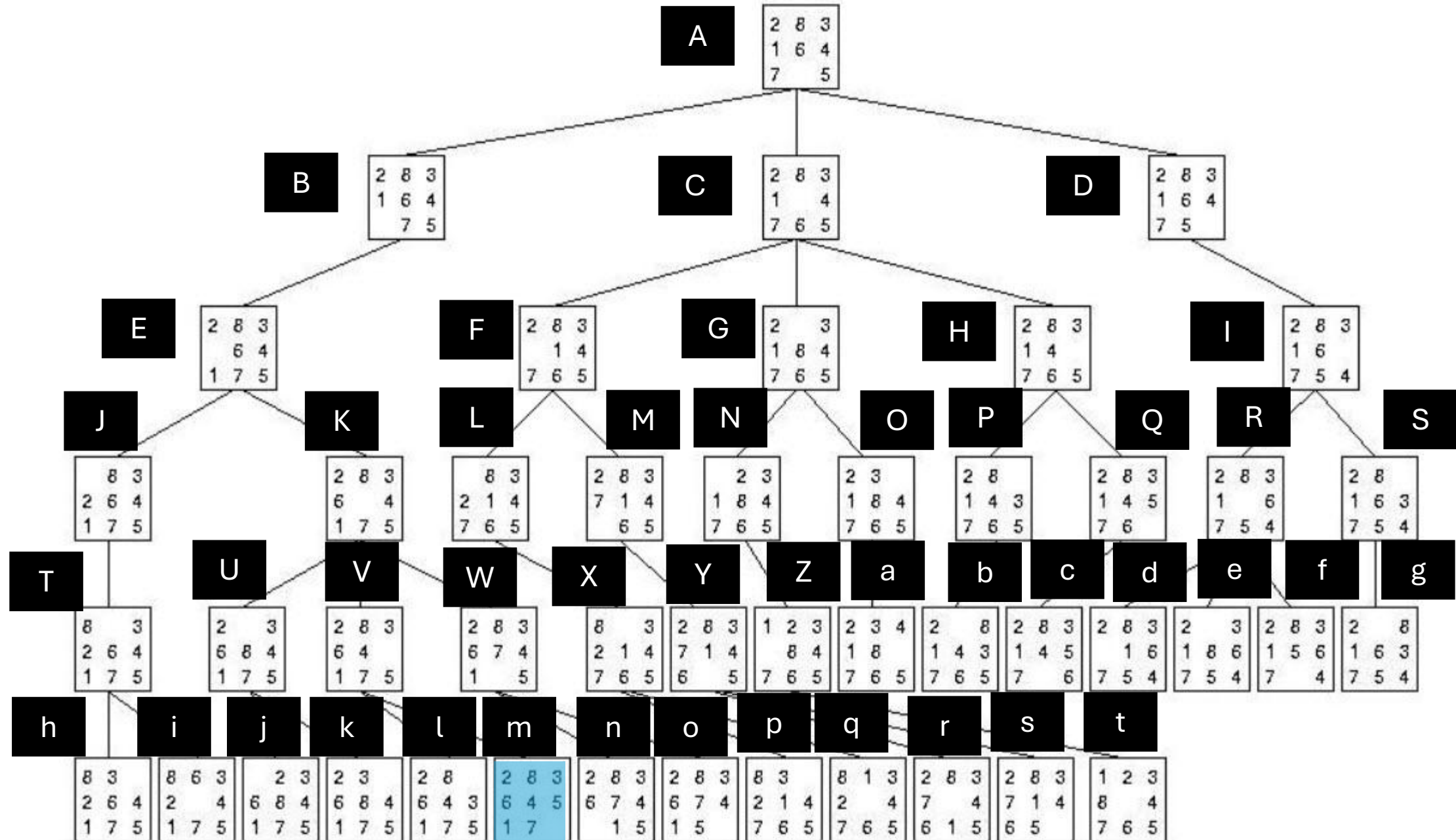


Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e , f , g , h , i , j

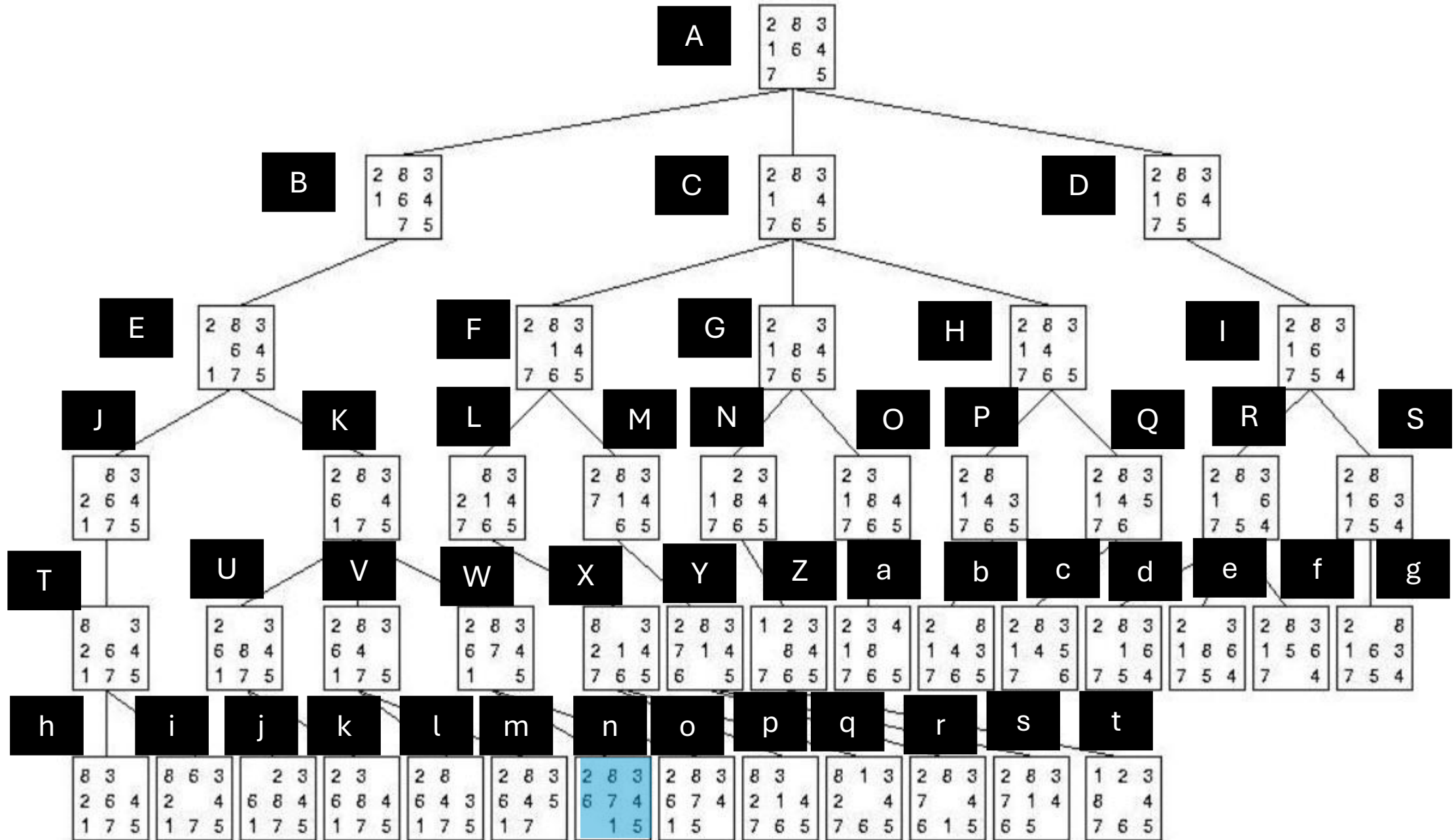




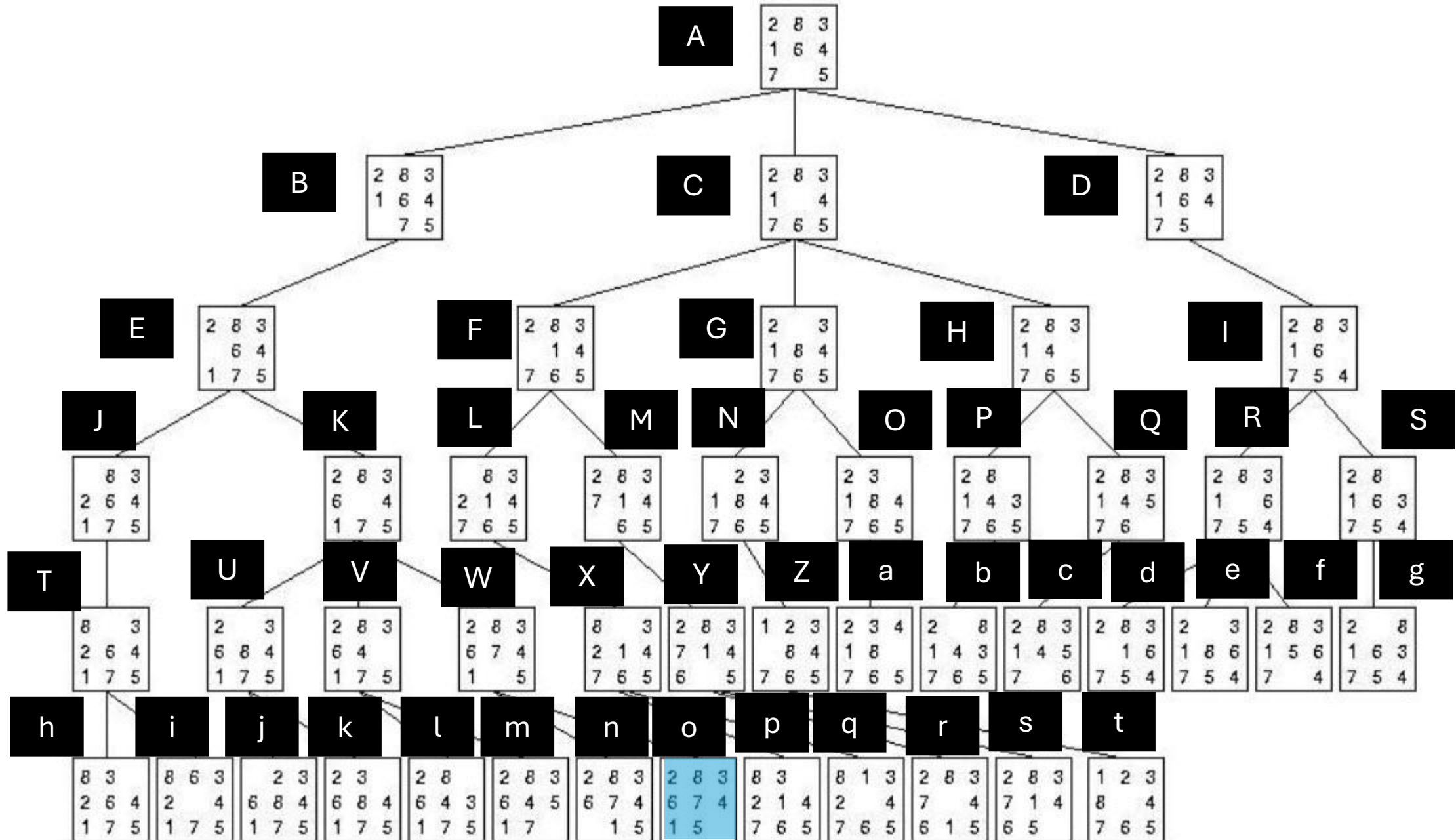
Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e , f , g , h , i , j , k , l

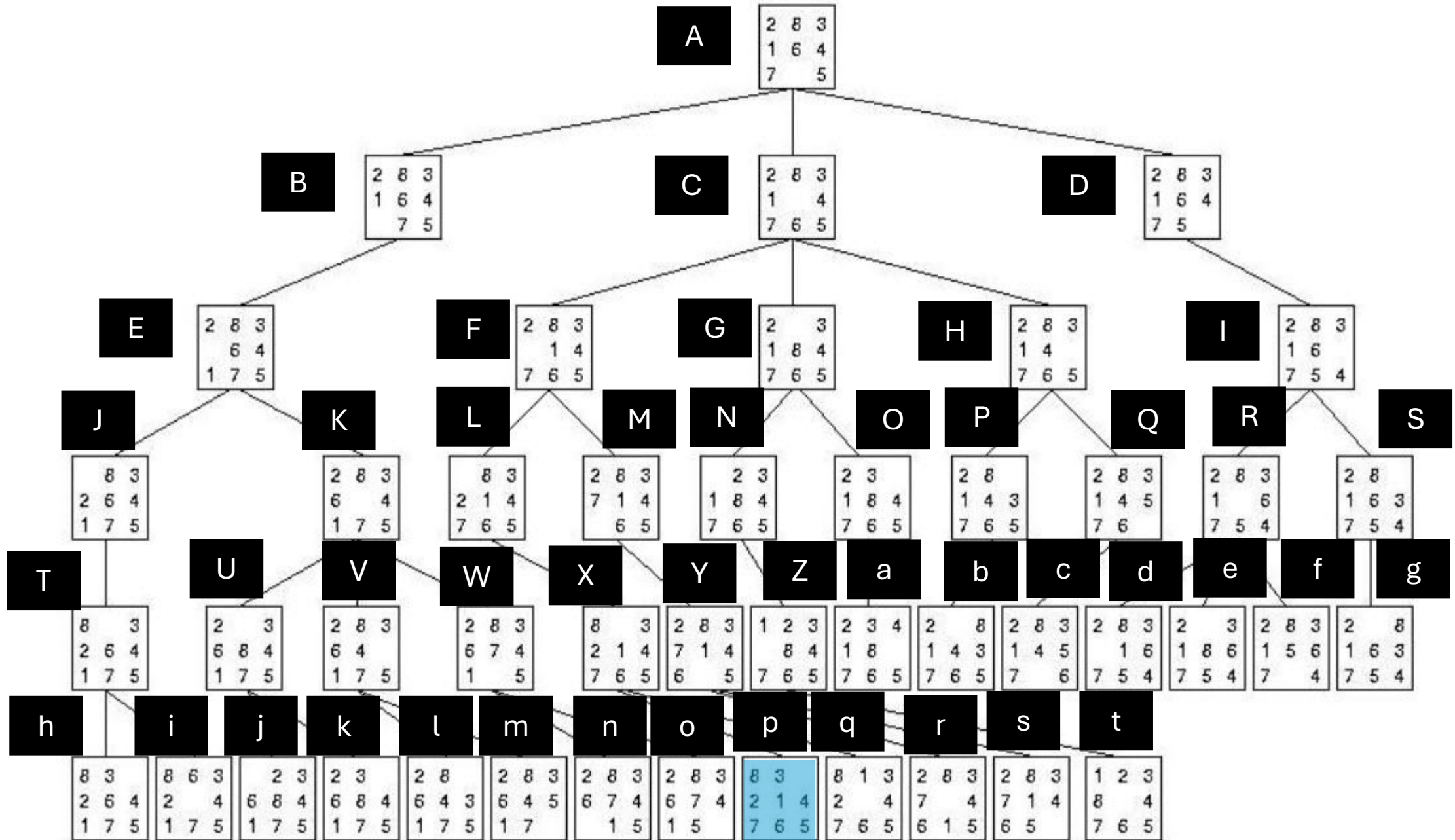


Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e , f , g , h , i , j , k , l , m

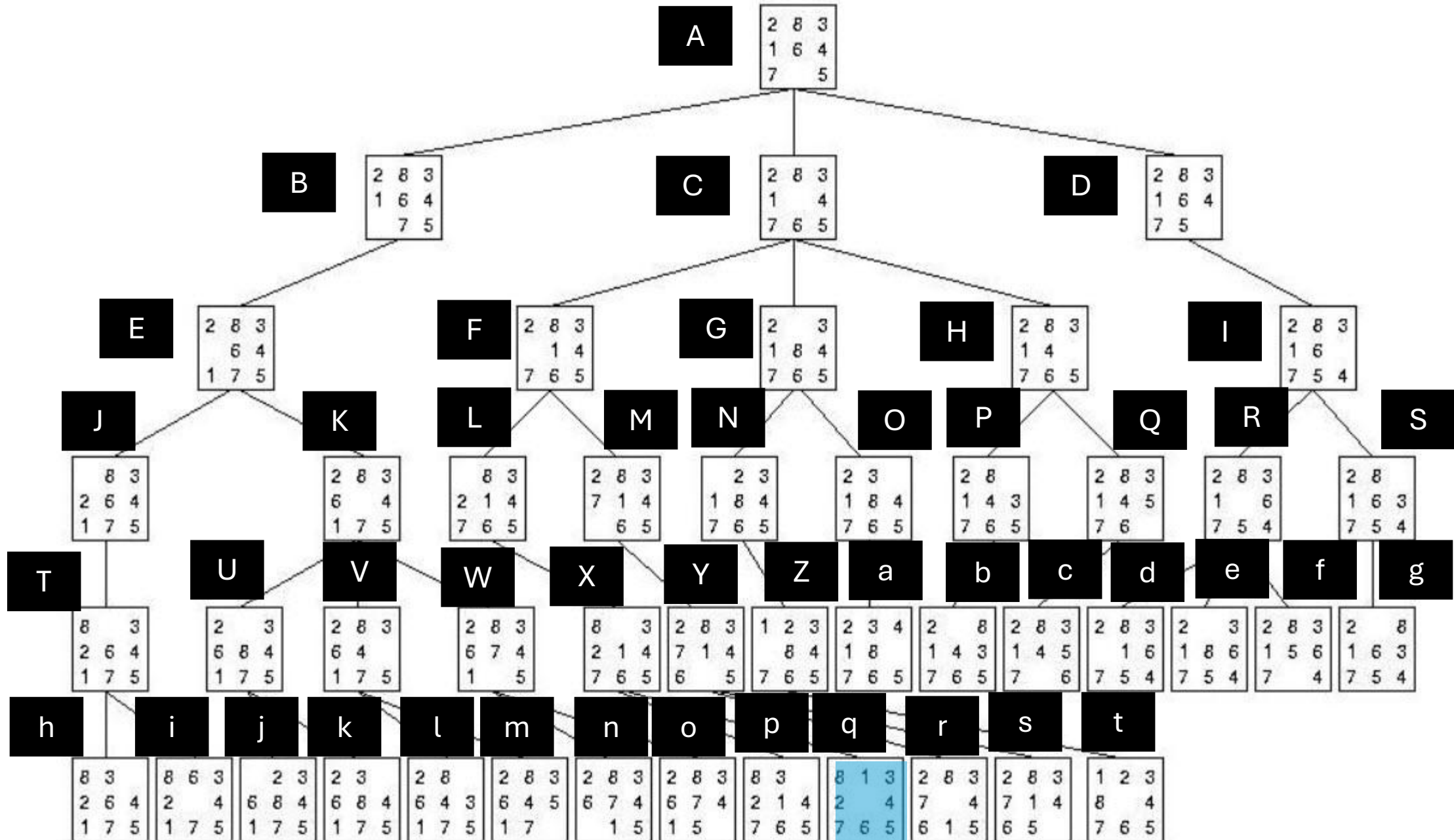


Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S ,
U , V , W , X , Y , Z , a , b , c , d , e , f , g , h , i , j , k , l , m , n

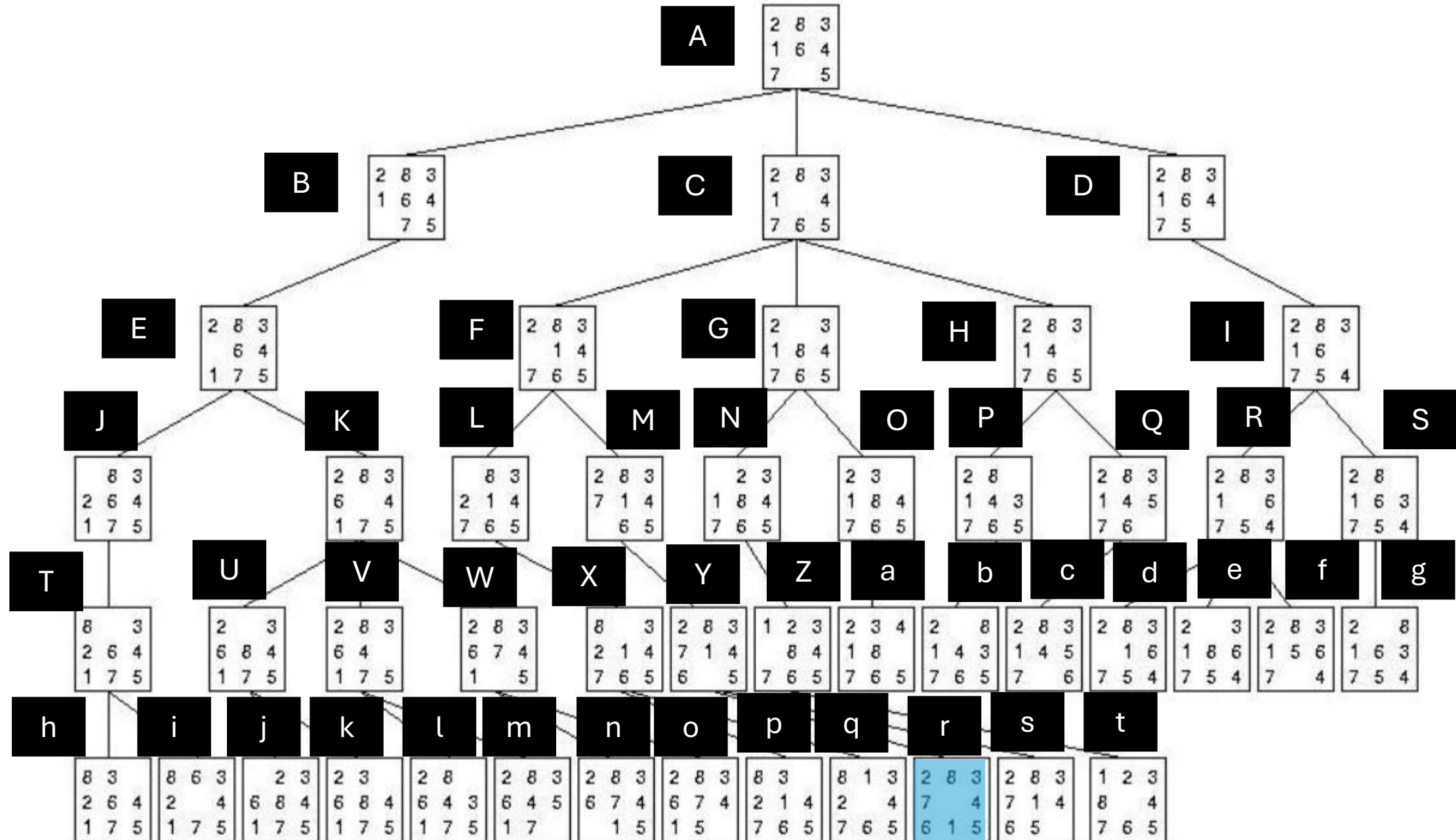


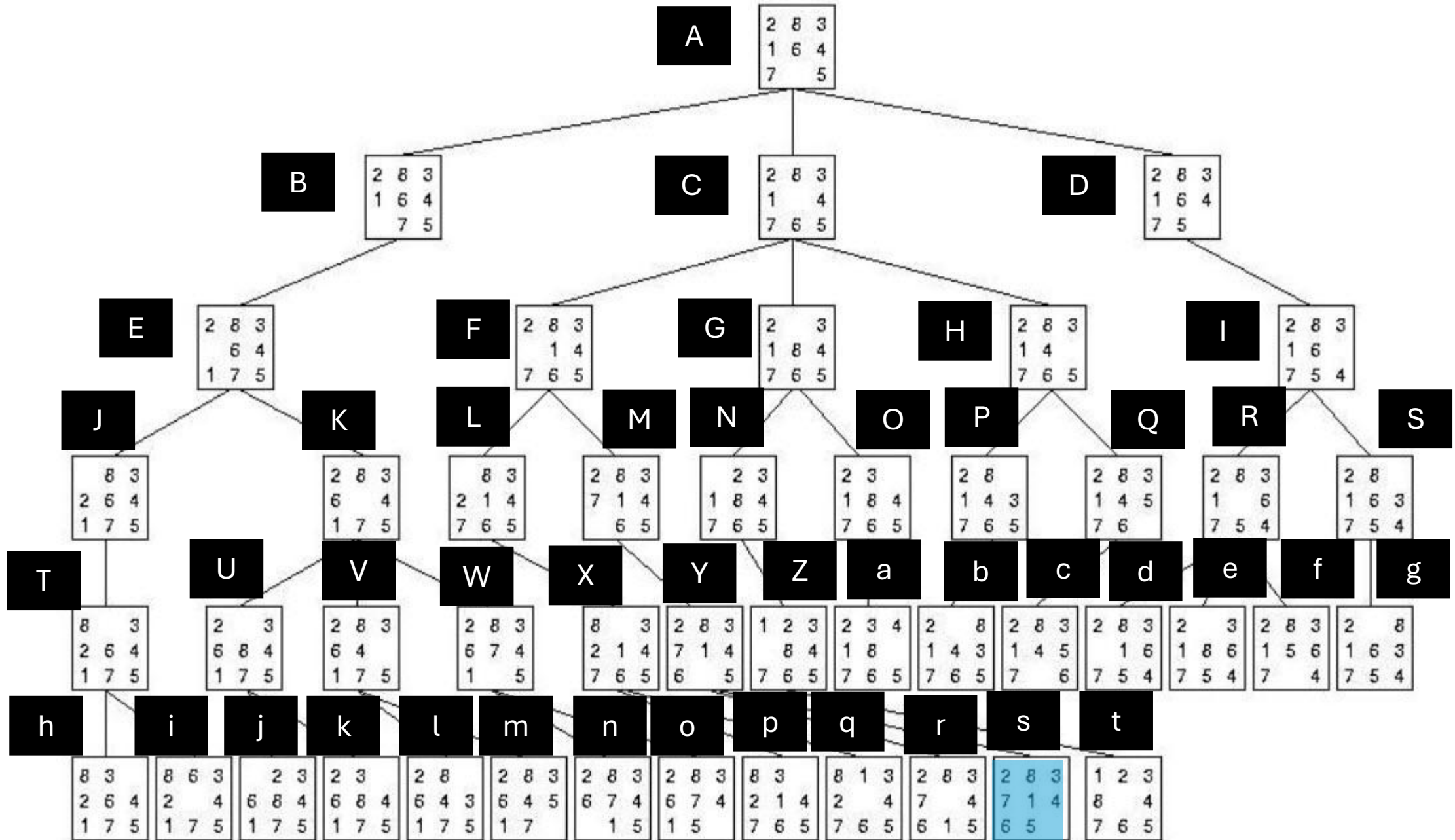


Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e , f , g , h , i , j , k , l , m , n , o , p

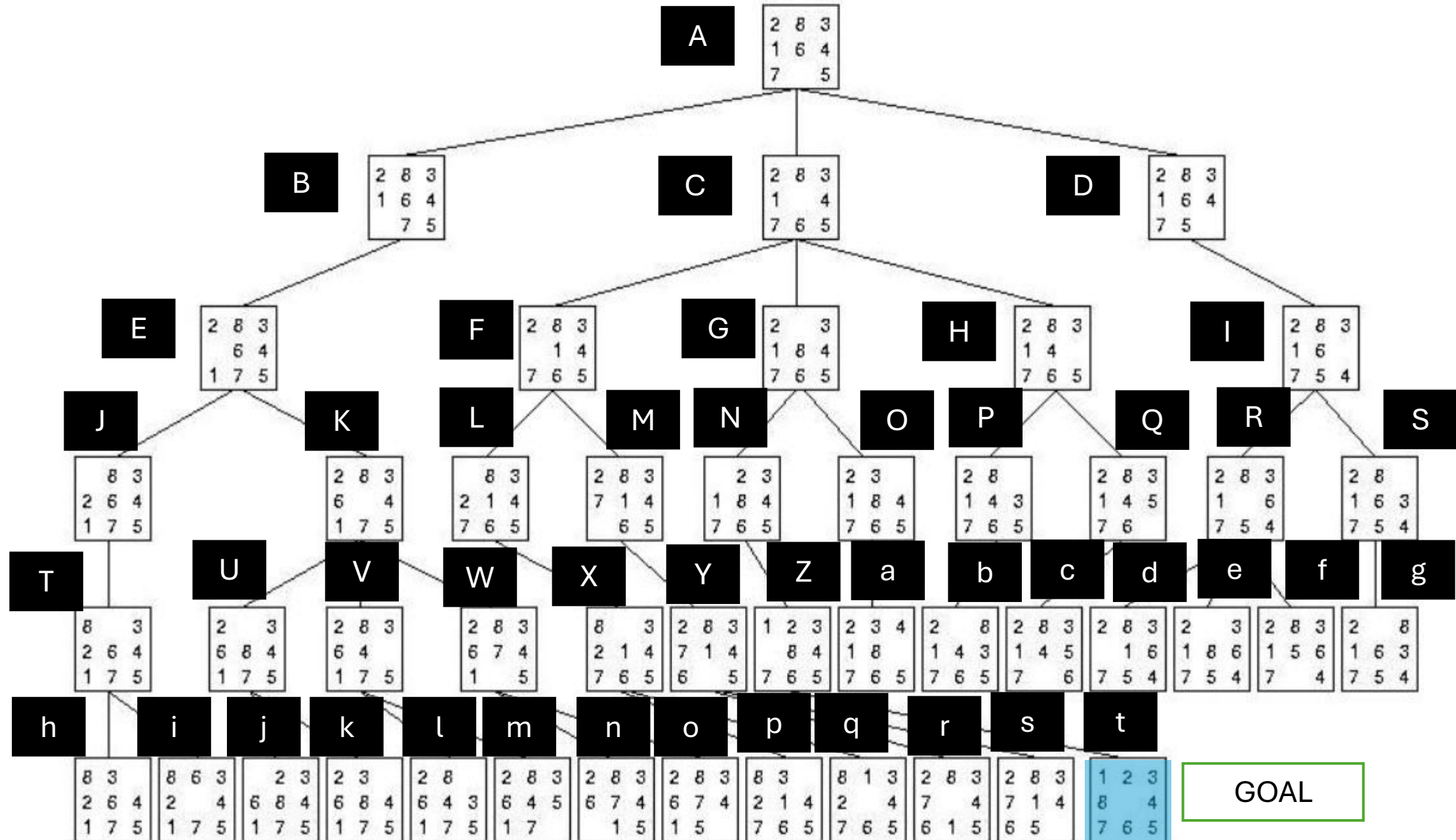


Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e , f , g , h , i , j , k , l , m , n , o , p , q





Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S , U , V , W , X , Y , Z , a , b , c , d , e , f , g , h , i , j , k , l , m , n , o , p , q , r , s



GOAL

Order : A , B , C , D , E , F , G , H , I , J , K , L , M , N , O , P , Q , R , S ,
U , V , W , X , Y , Z , a , b , c , d , e , f , g , h , i , j , k , l , m , n , o , p , q , r , s , t

1-Depth-first search

The Depth First Search is also Blind , Moving from top to bottom , branch by branch until reaching the Goal

A

2	8	3
1	6	4
7		5

B

2	8	3
1	6	4
7		5

Order : A

A

2	8	3
1	6	4
7		5

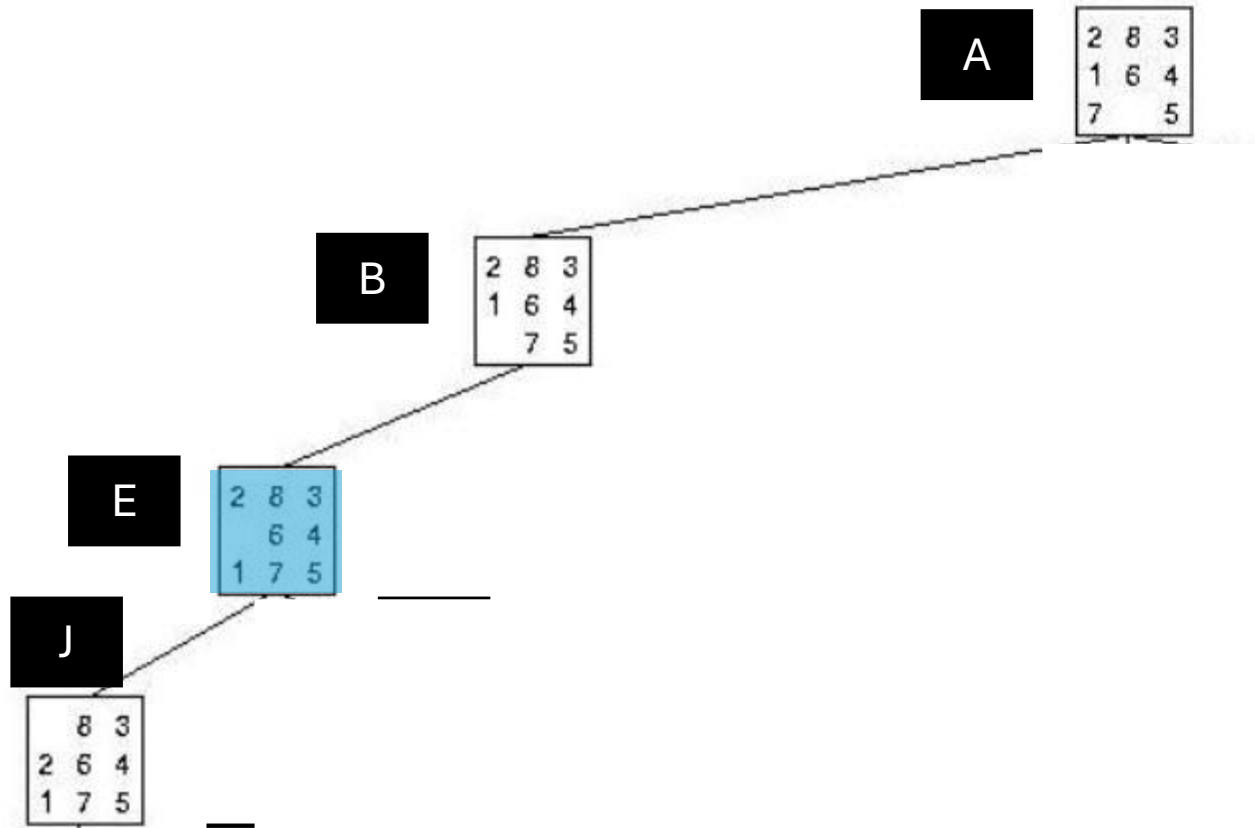
B

2	8	3
1	6	4
7	5	

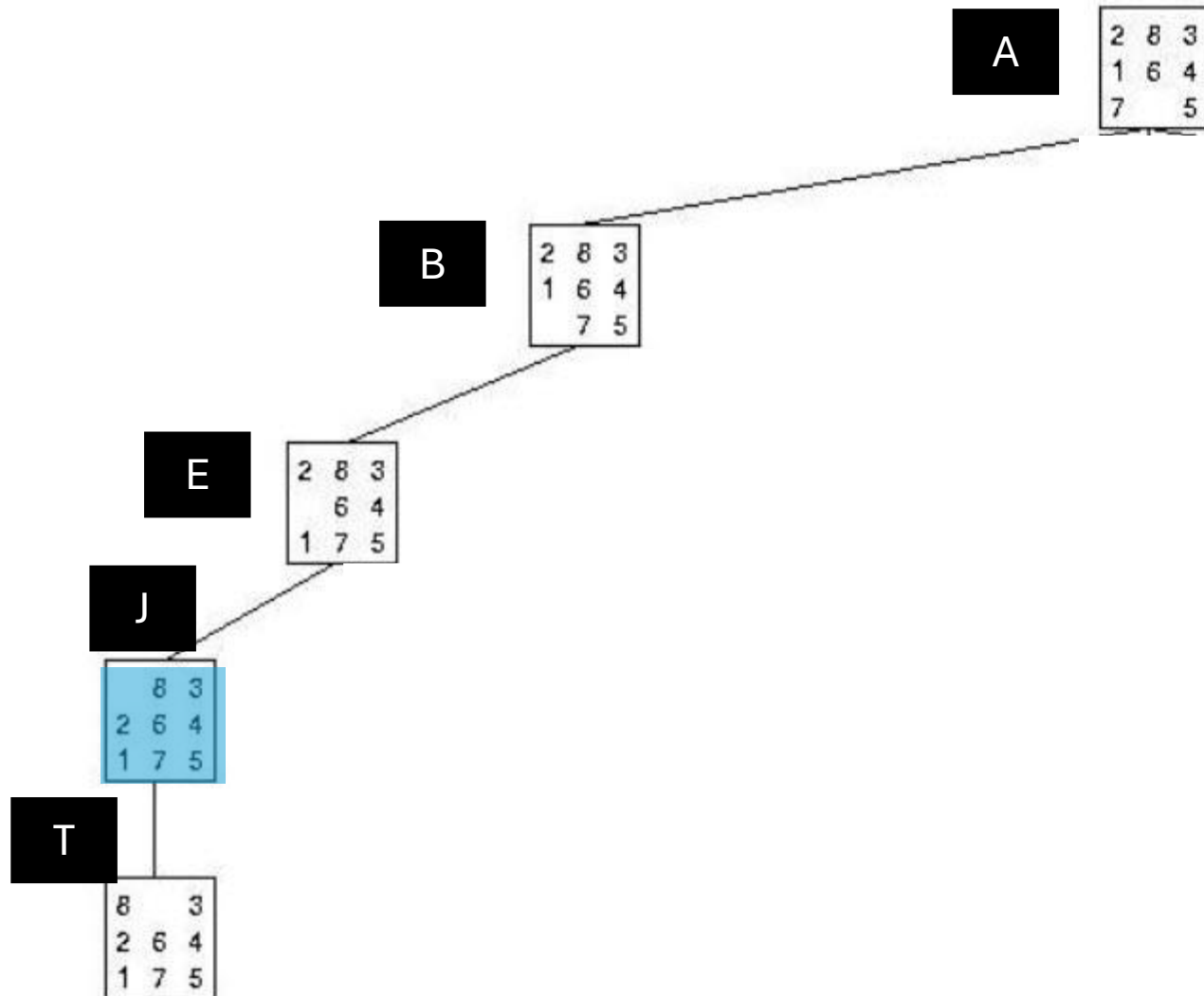
E

2	8	3
	6	4
1	7	5

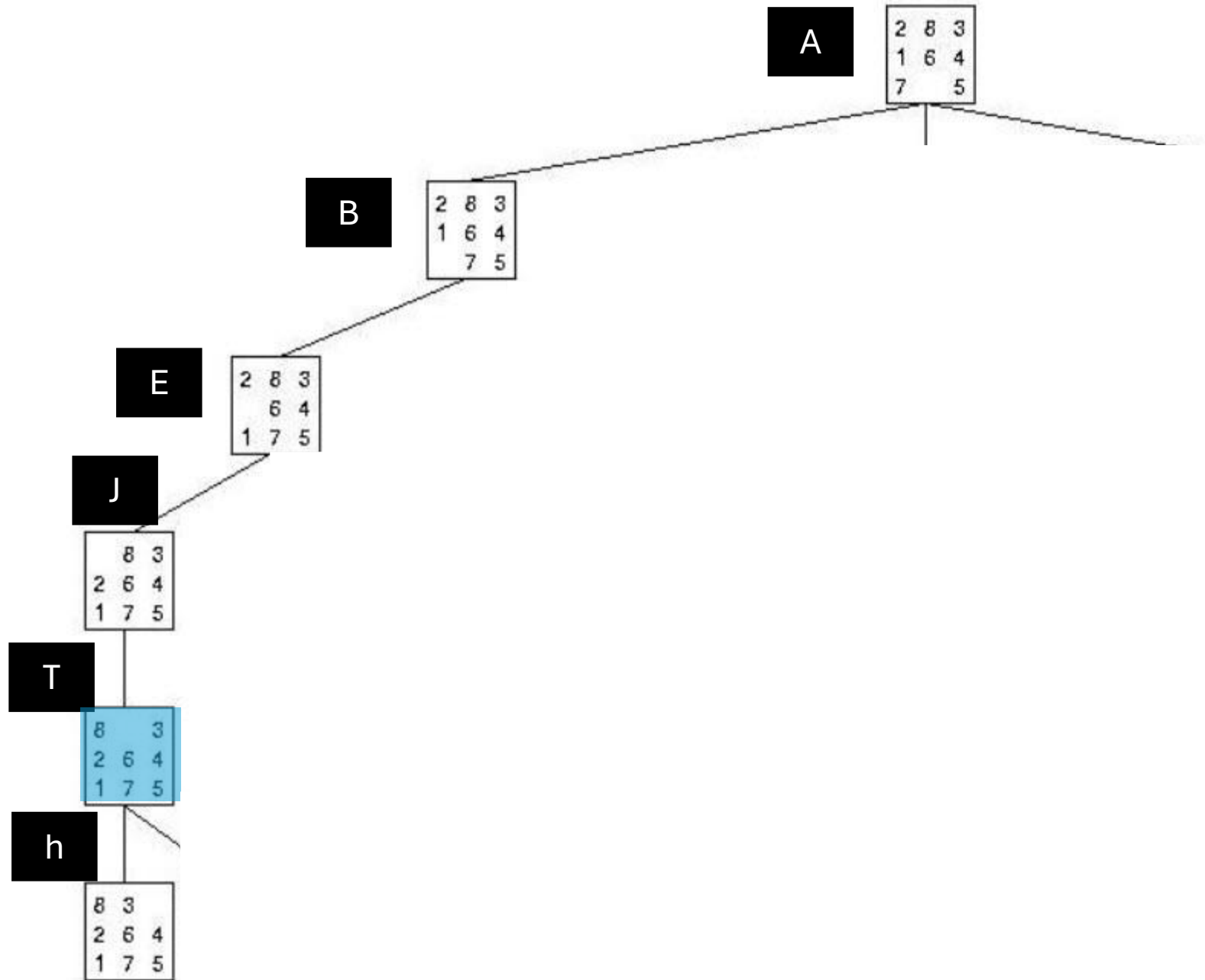
Order : A , B



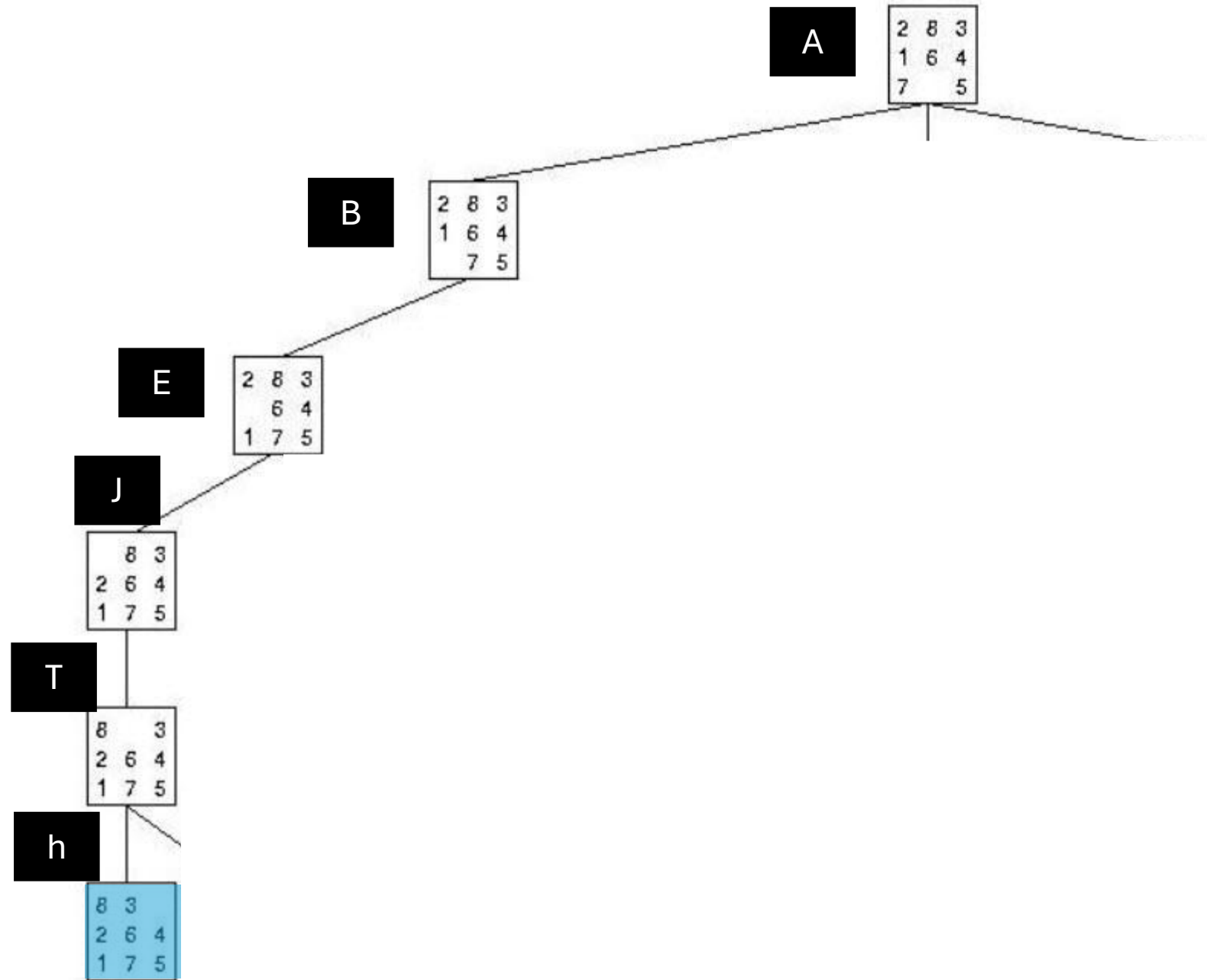
Order : A , B , E



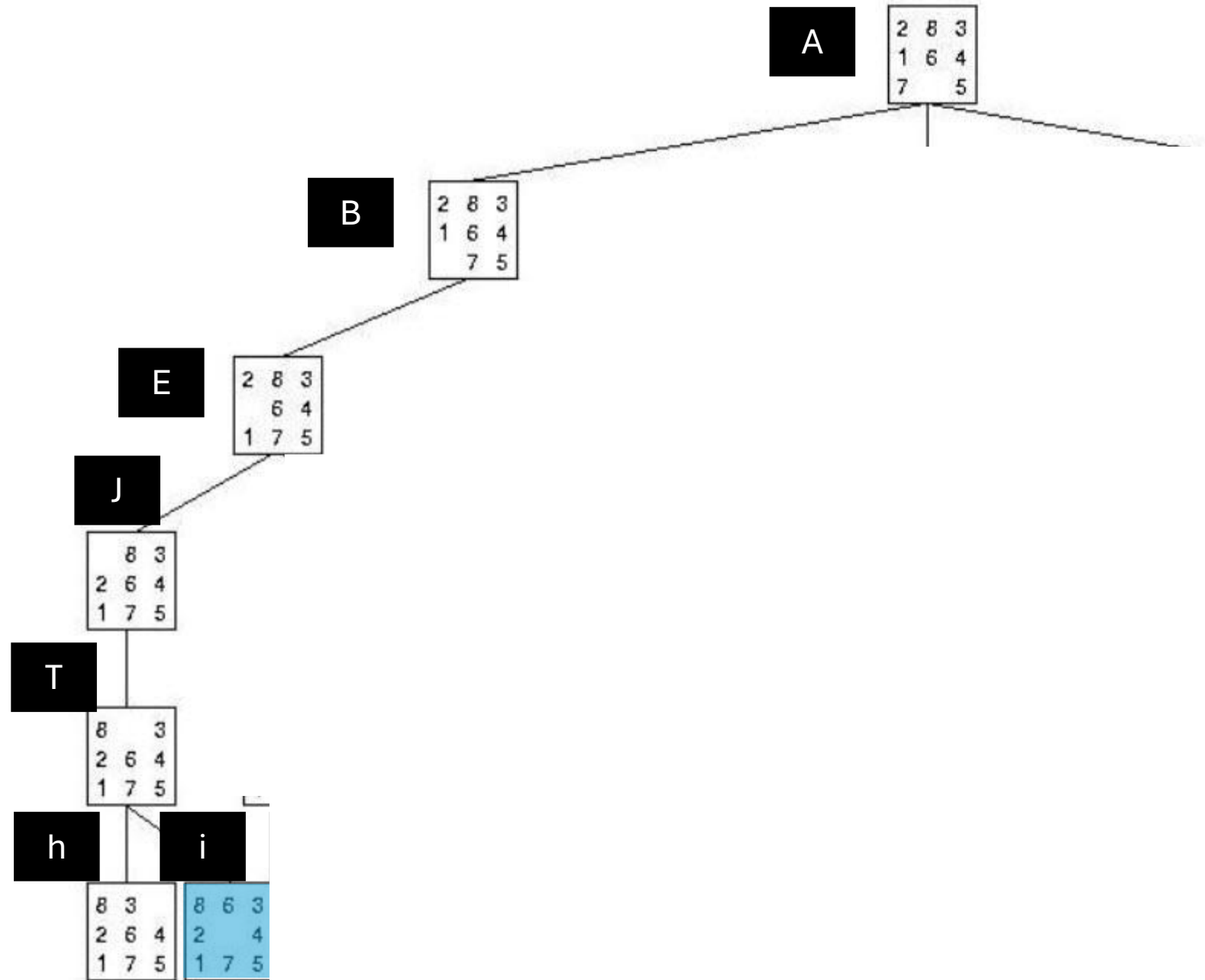
Order : A , B , E , J



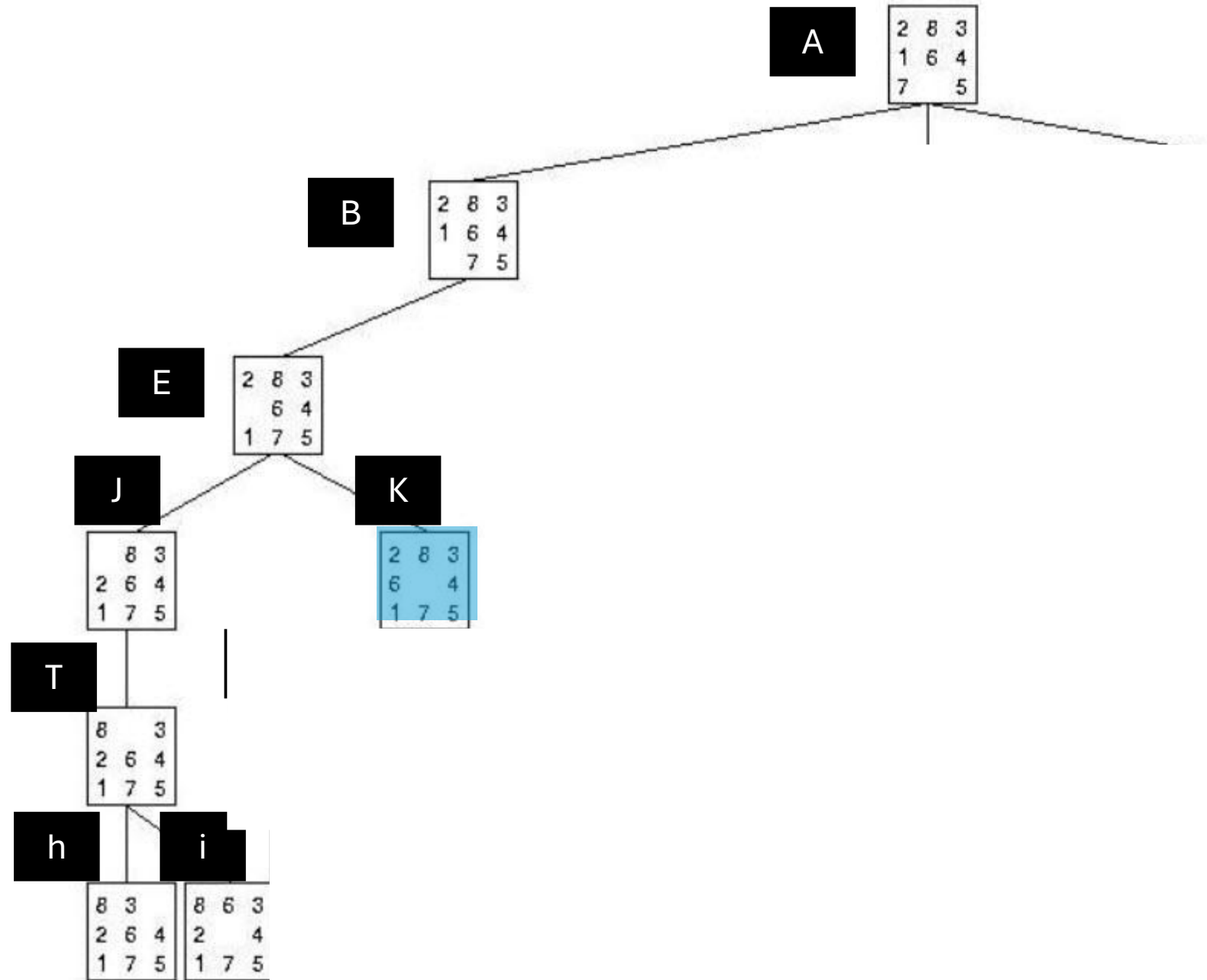
Order : A , B , E , J , T ,



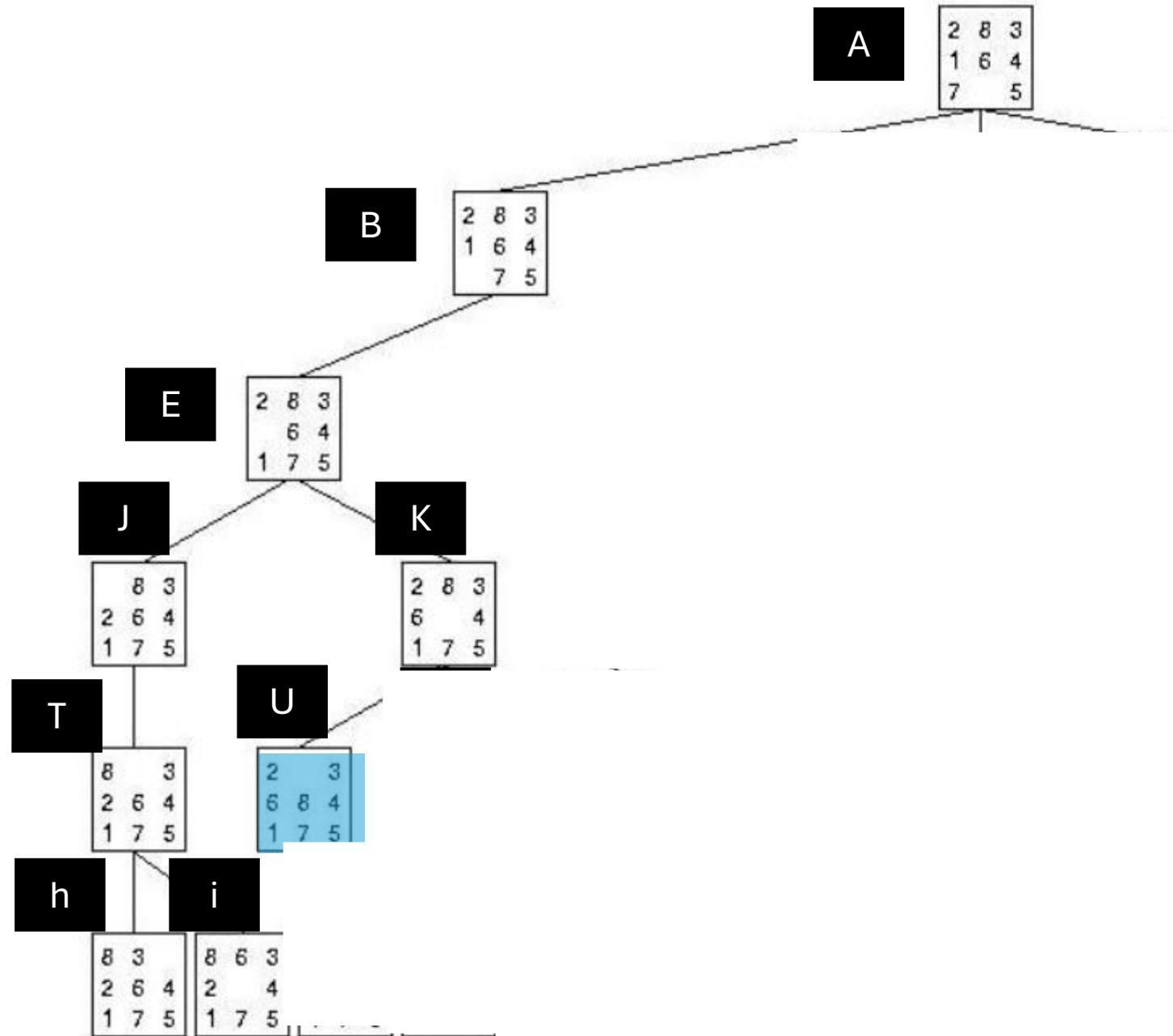
Order : A , B , E , J , T , H ,



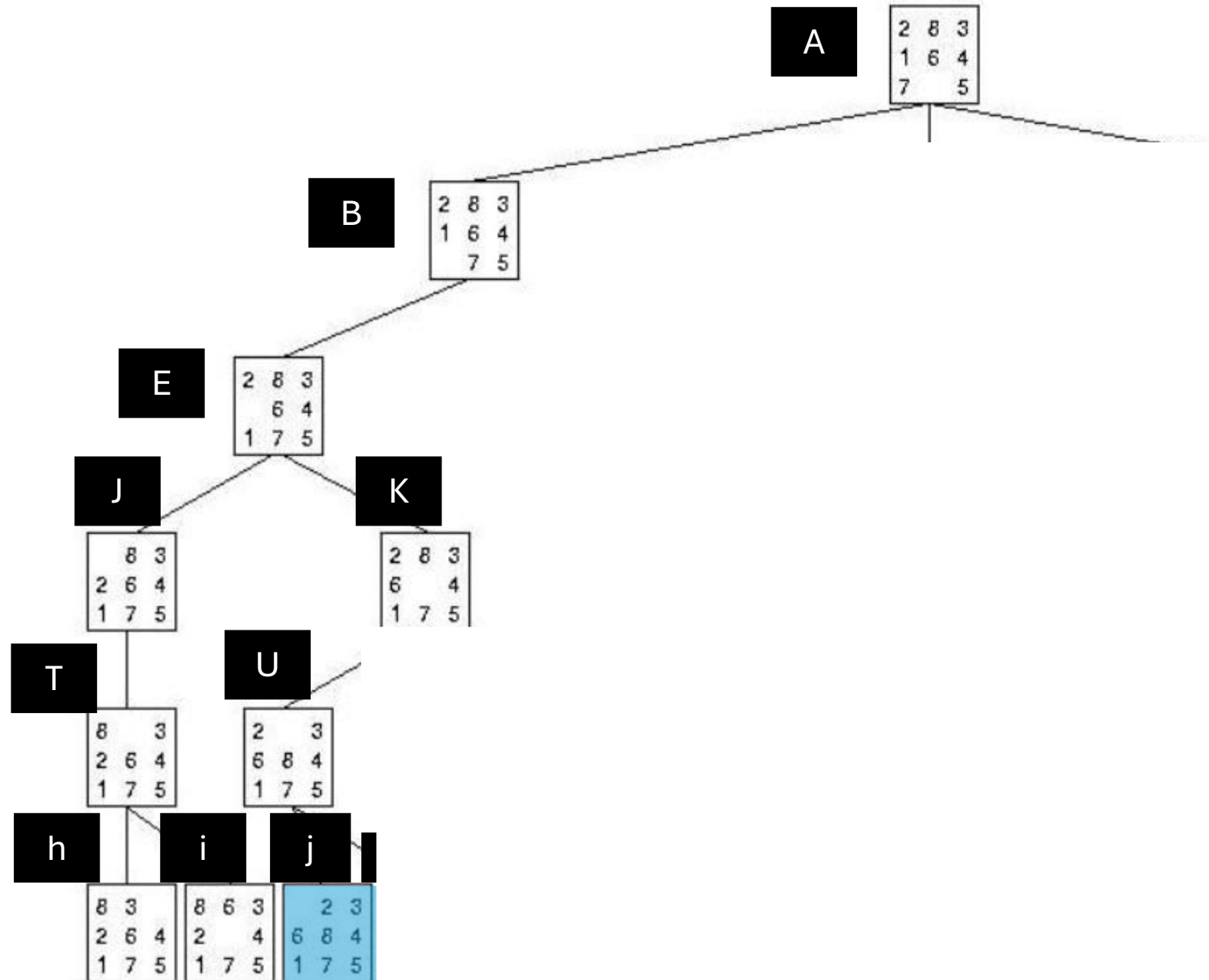
Order : A , B , E , J , T , h , i ,



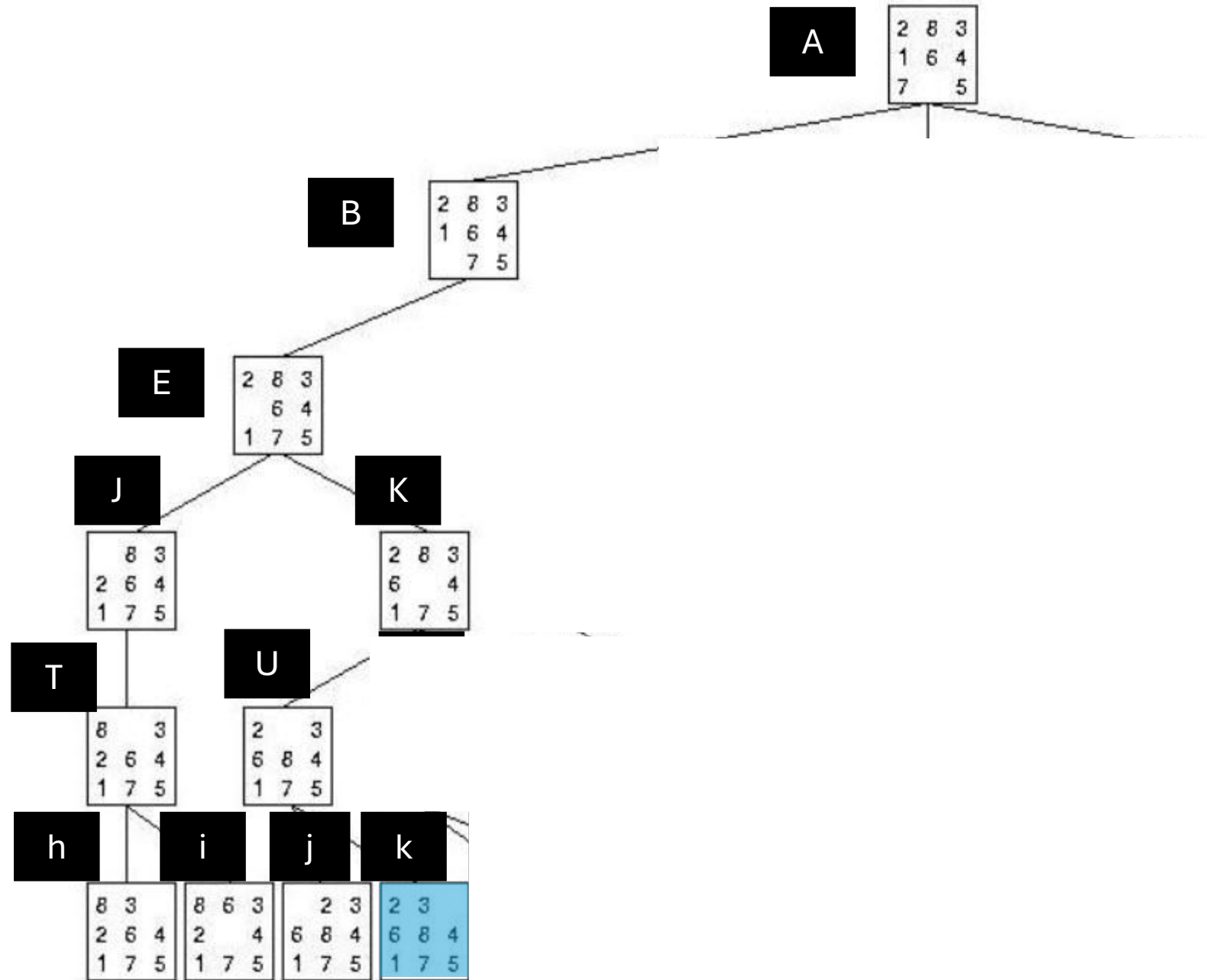
Order : A , B , E , J , T , h , i , K



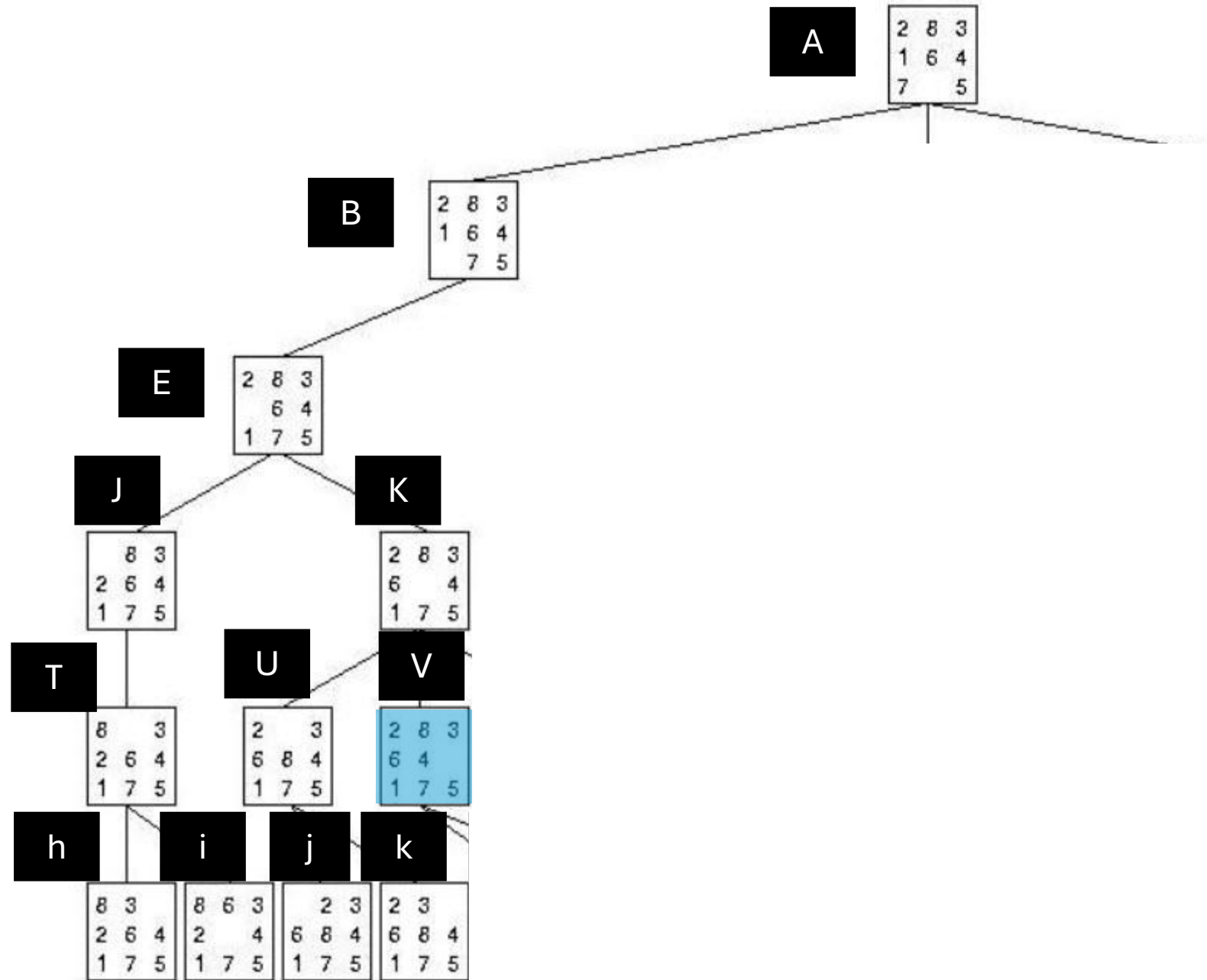
Order : A , B , E , J , T , h , i , K , U ,



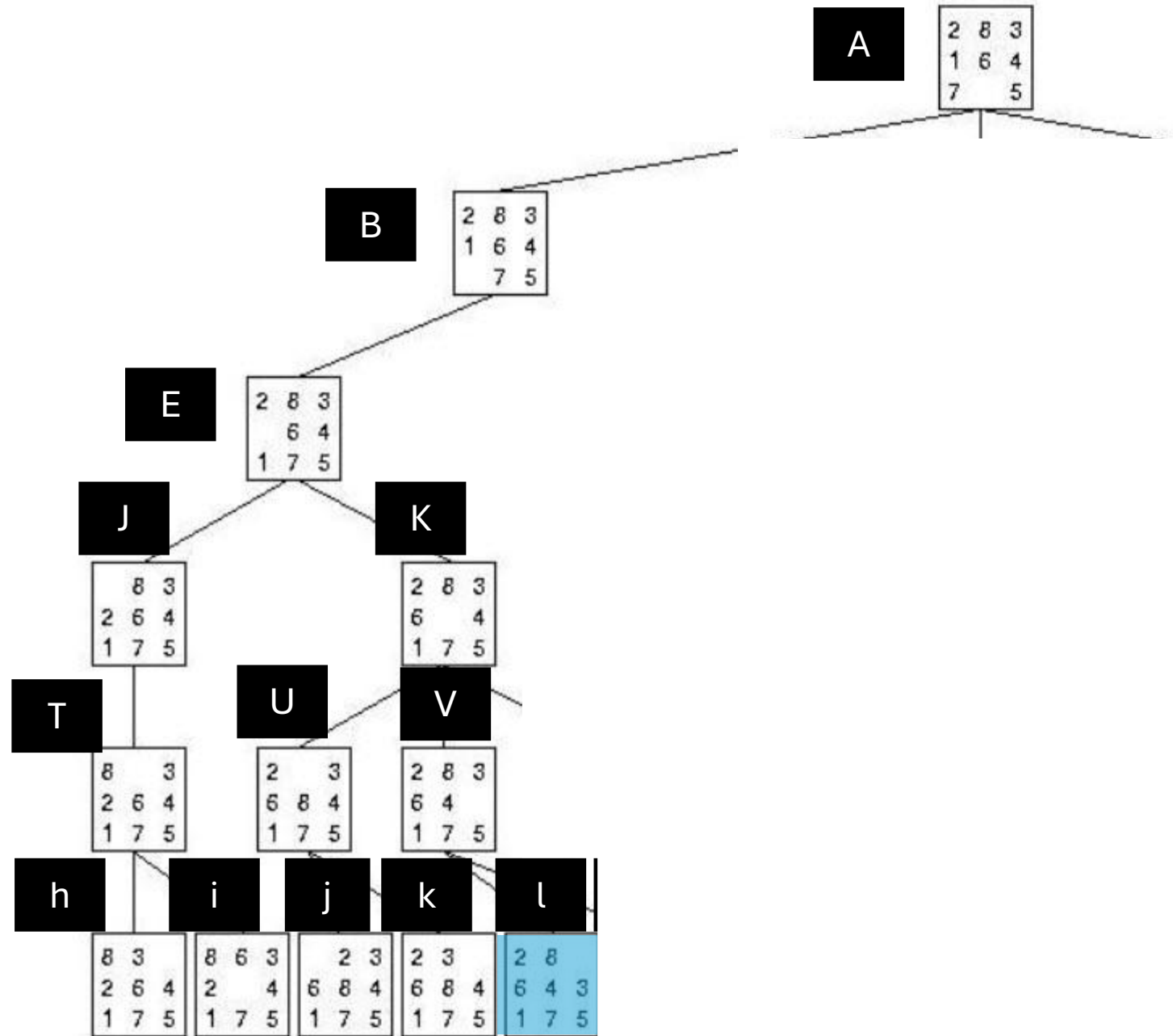
Order : A , B , E , J , T , h , i , K , U , j ,



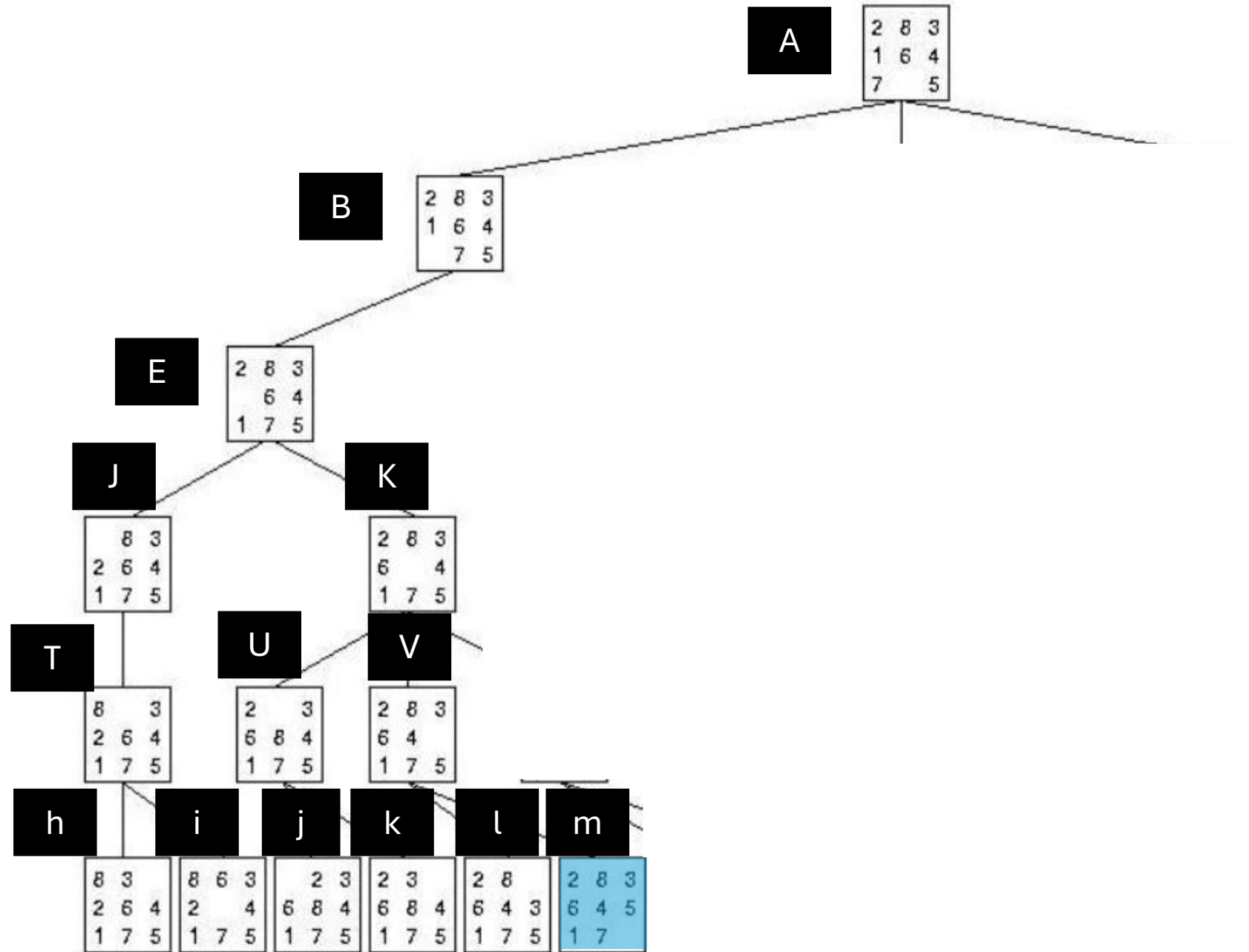
Order : A , B , E , J , T , h , i , K , U , j , k ,



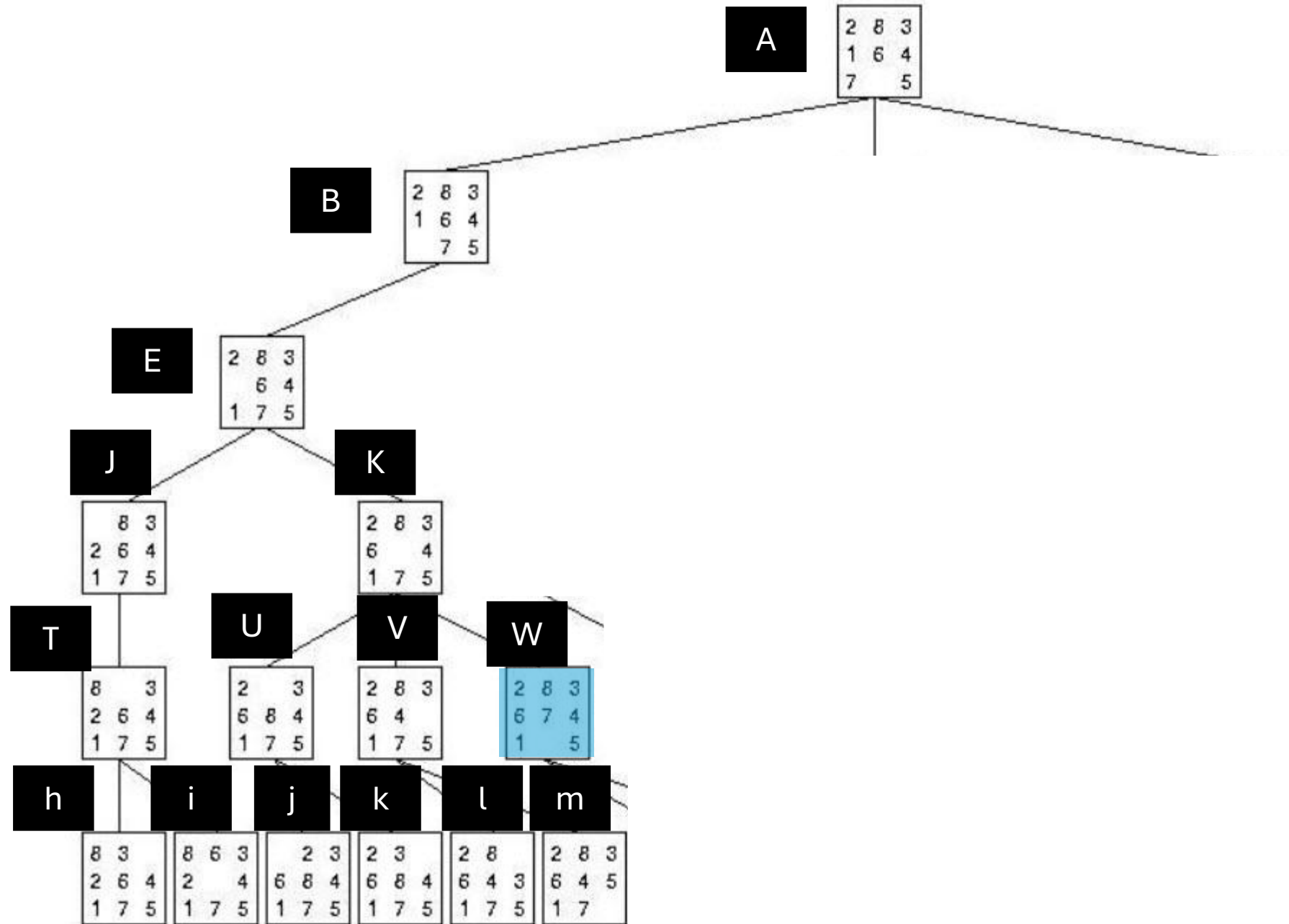
Order : A , B , E , J , T , h , i , K , U , j , k , V ,



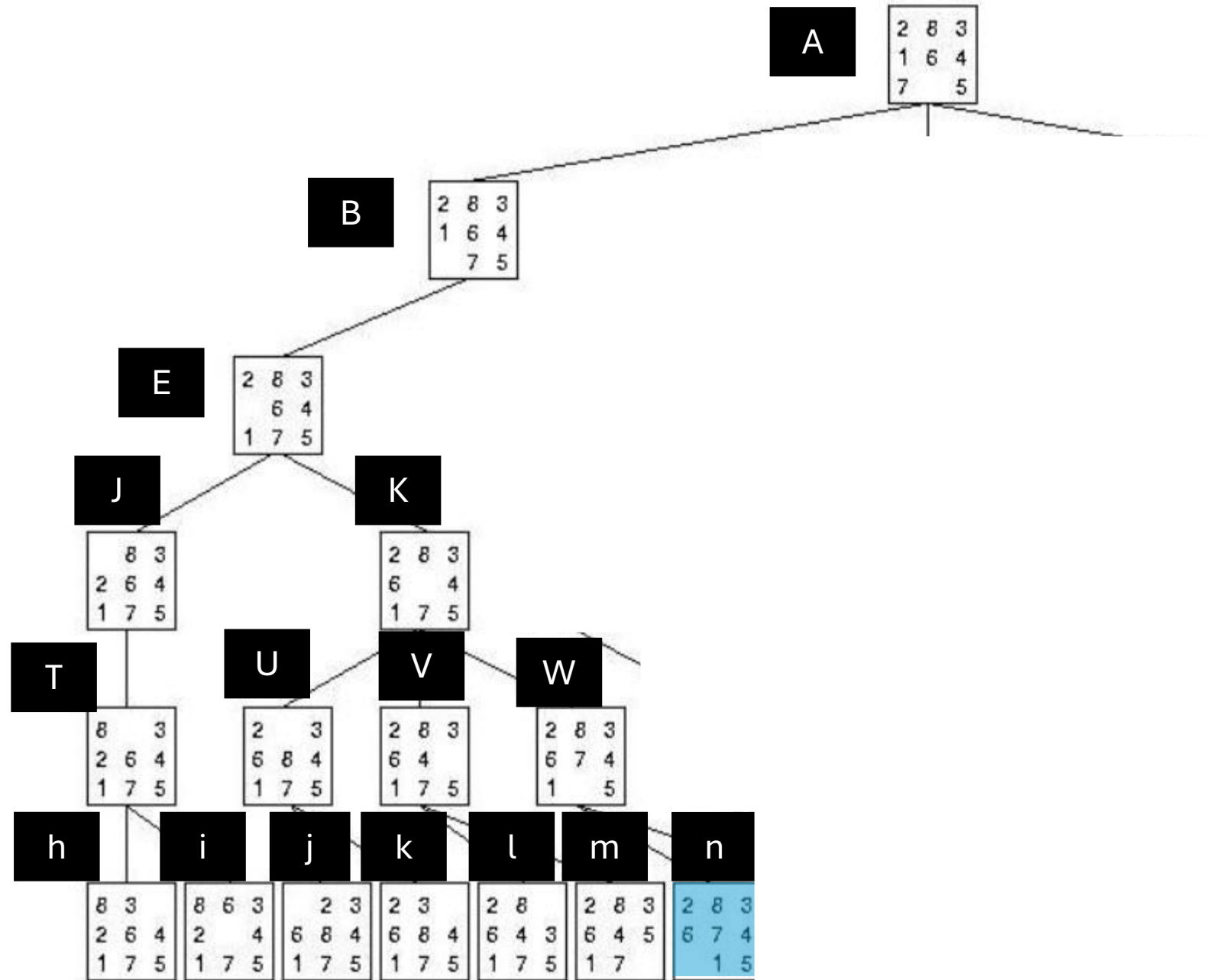
Order : A , B , E , J , T , h , i , K , U , j , k , V , l



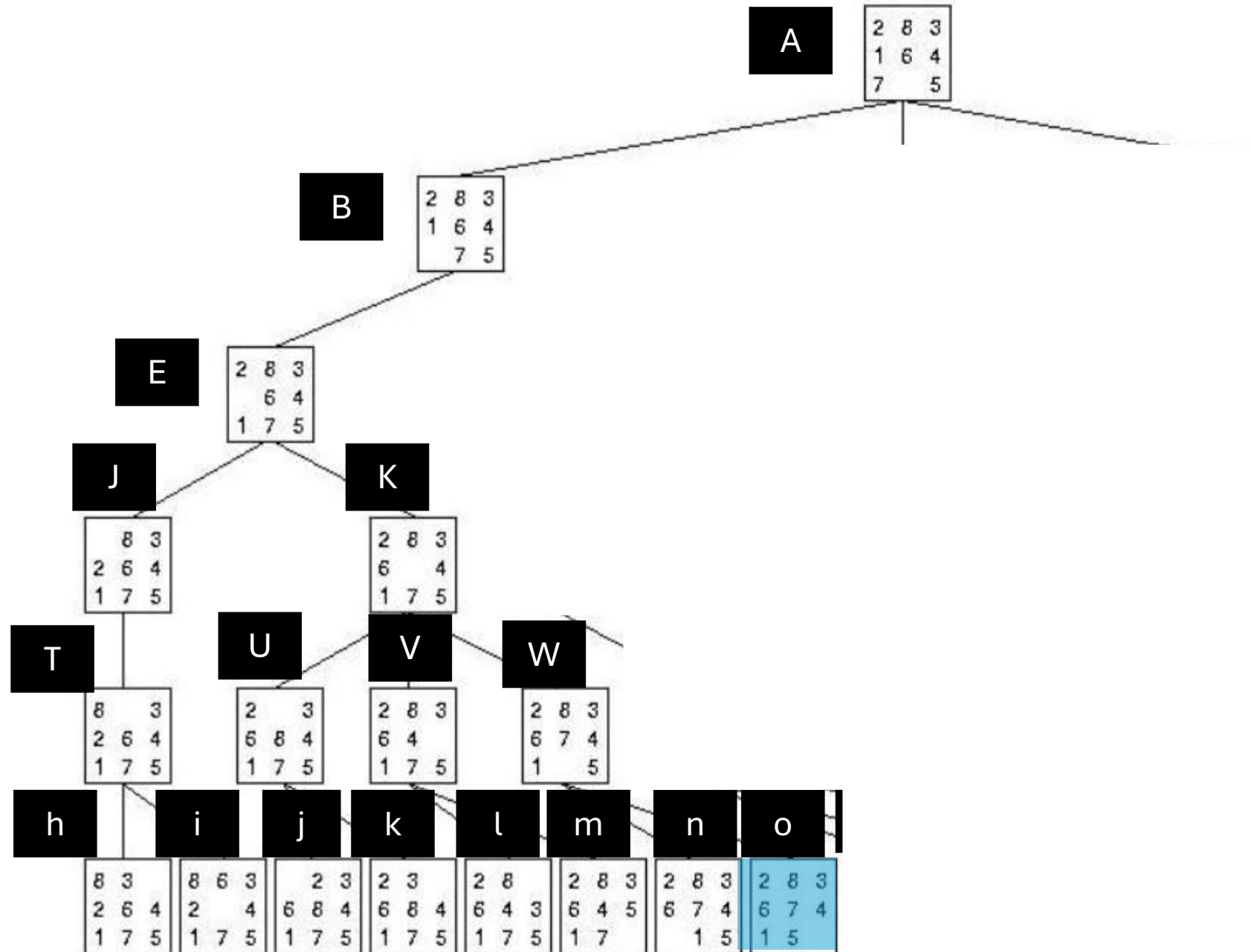
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m



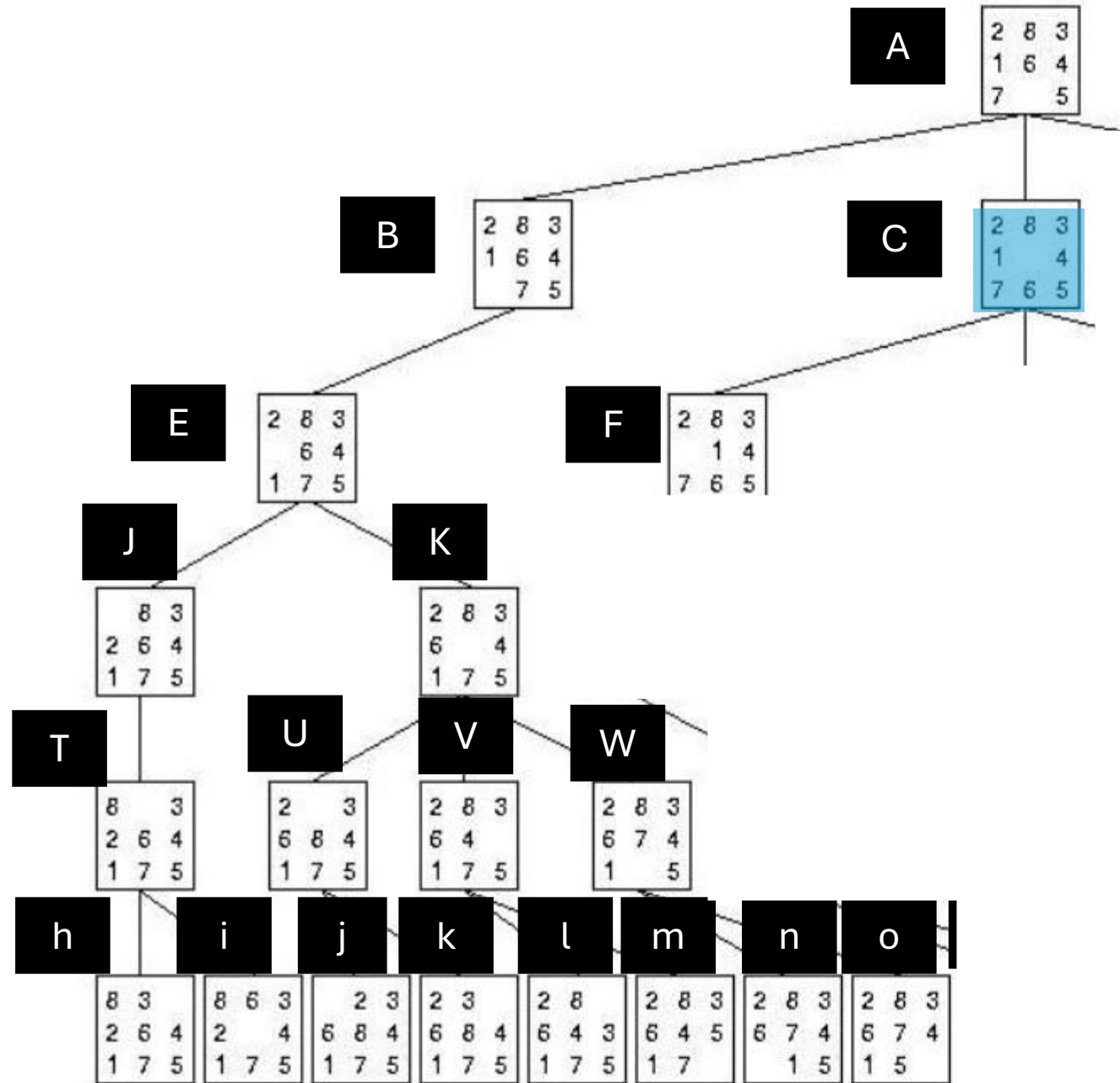
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W ,



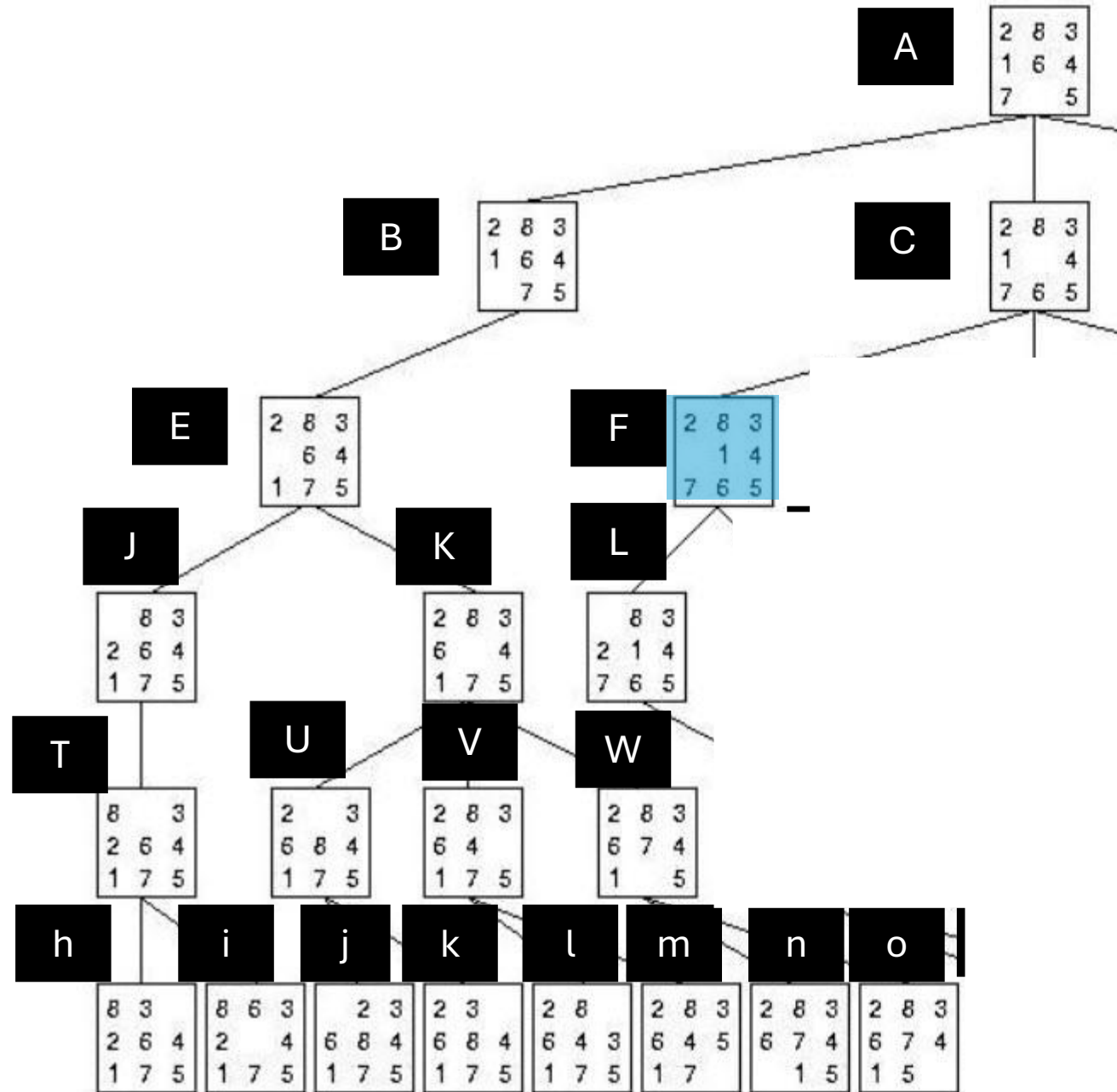
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n



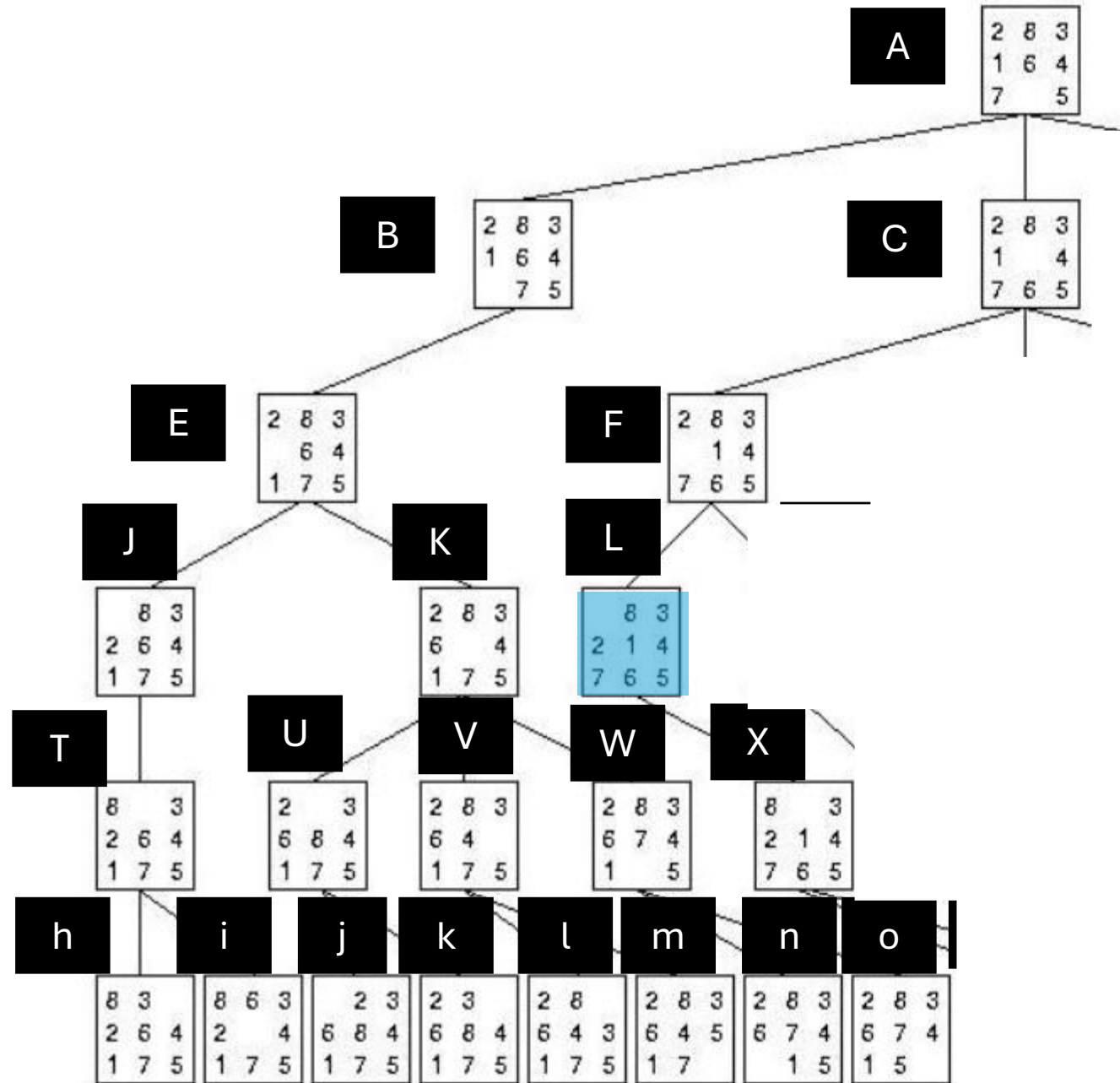
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o



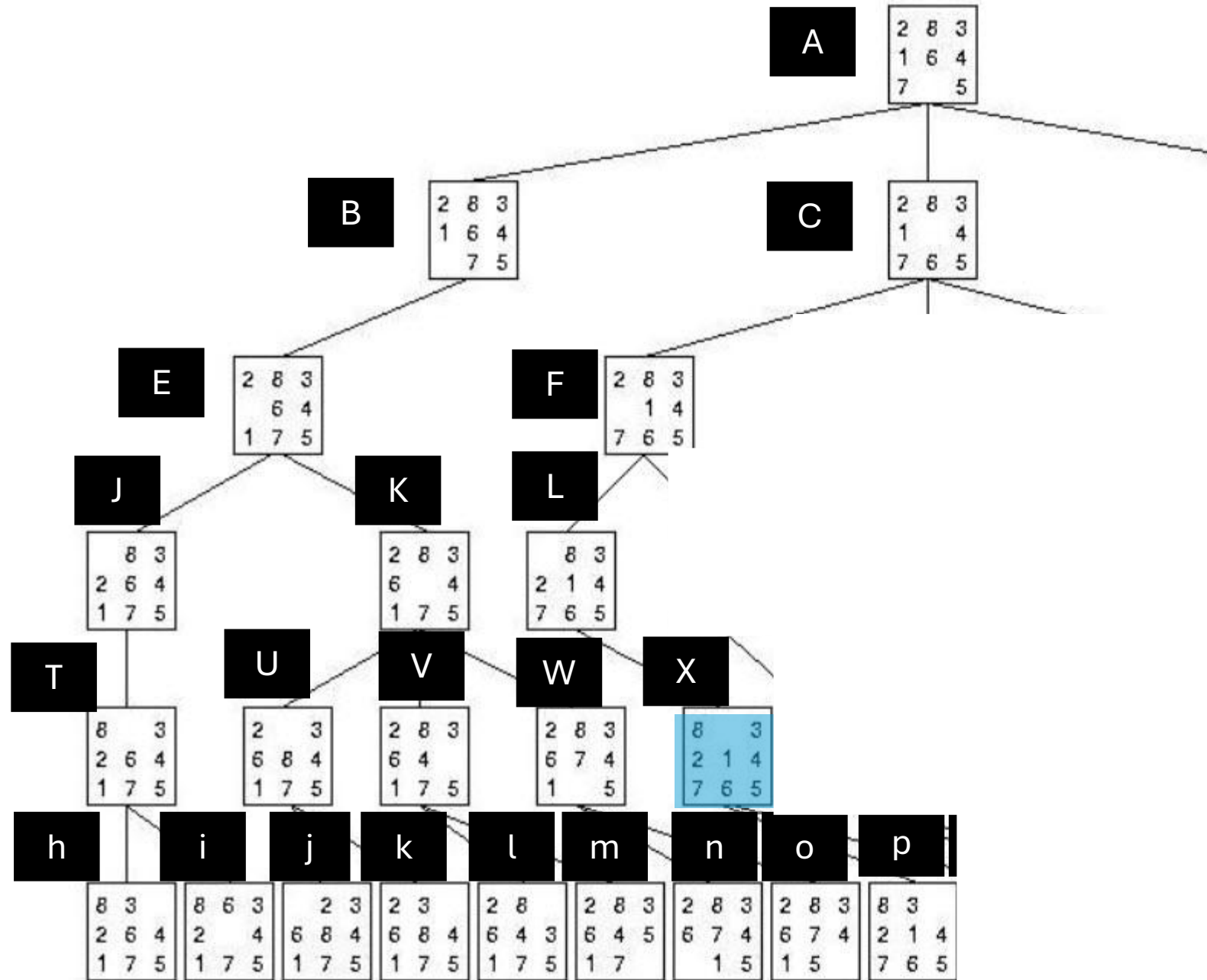
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C



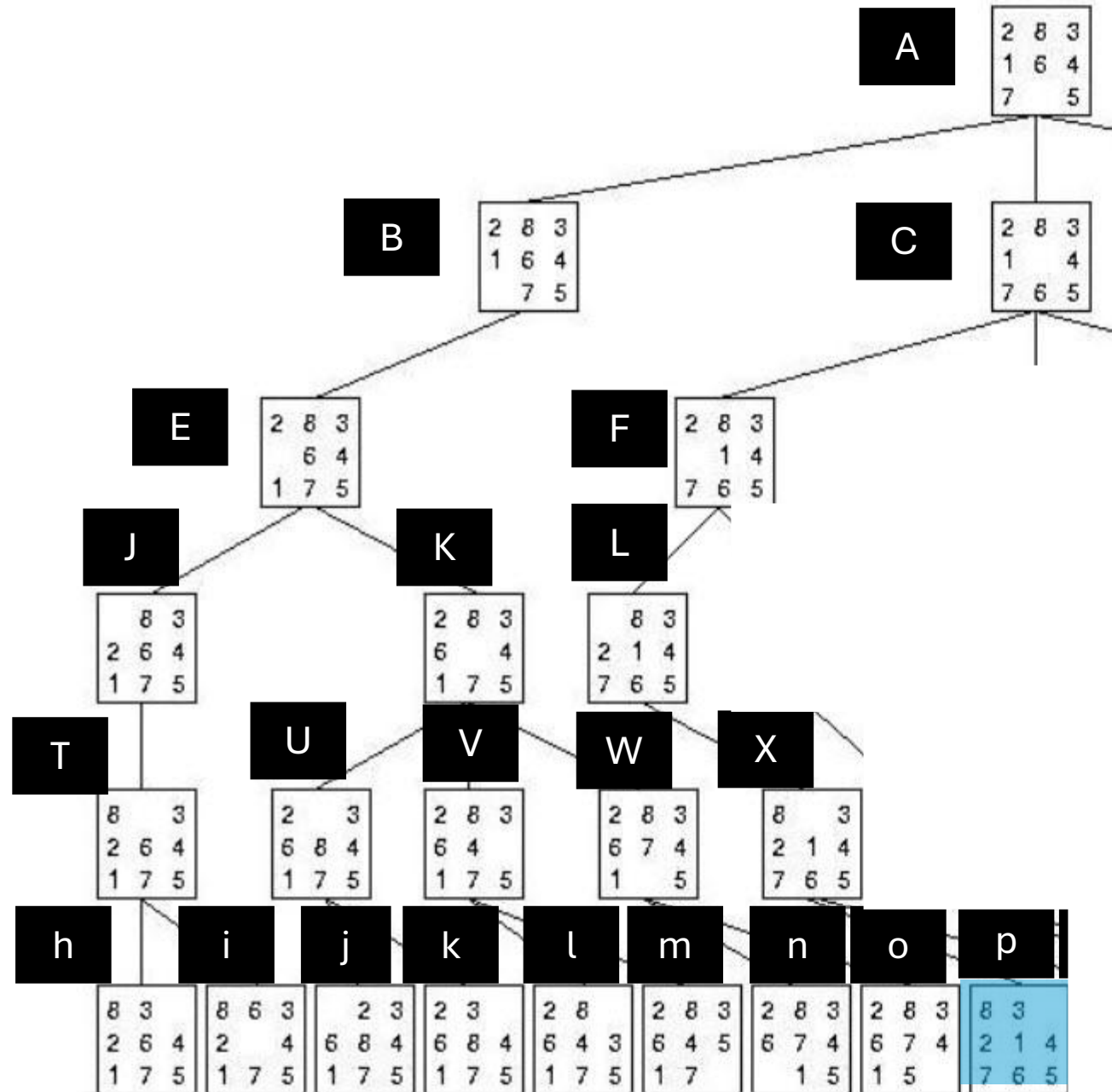
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F



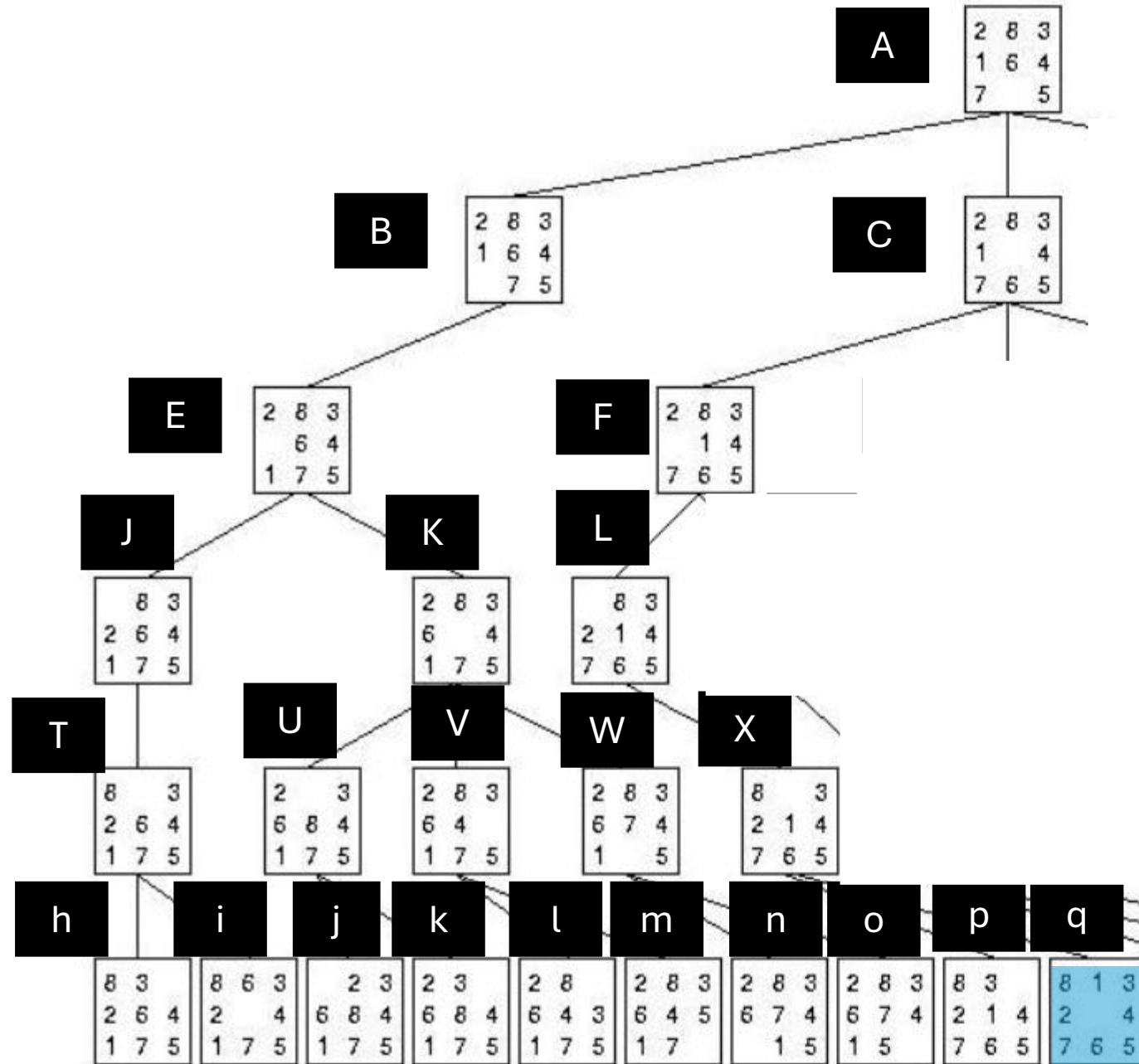
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L



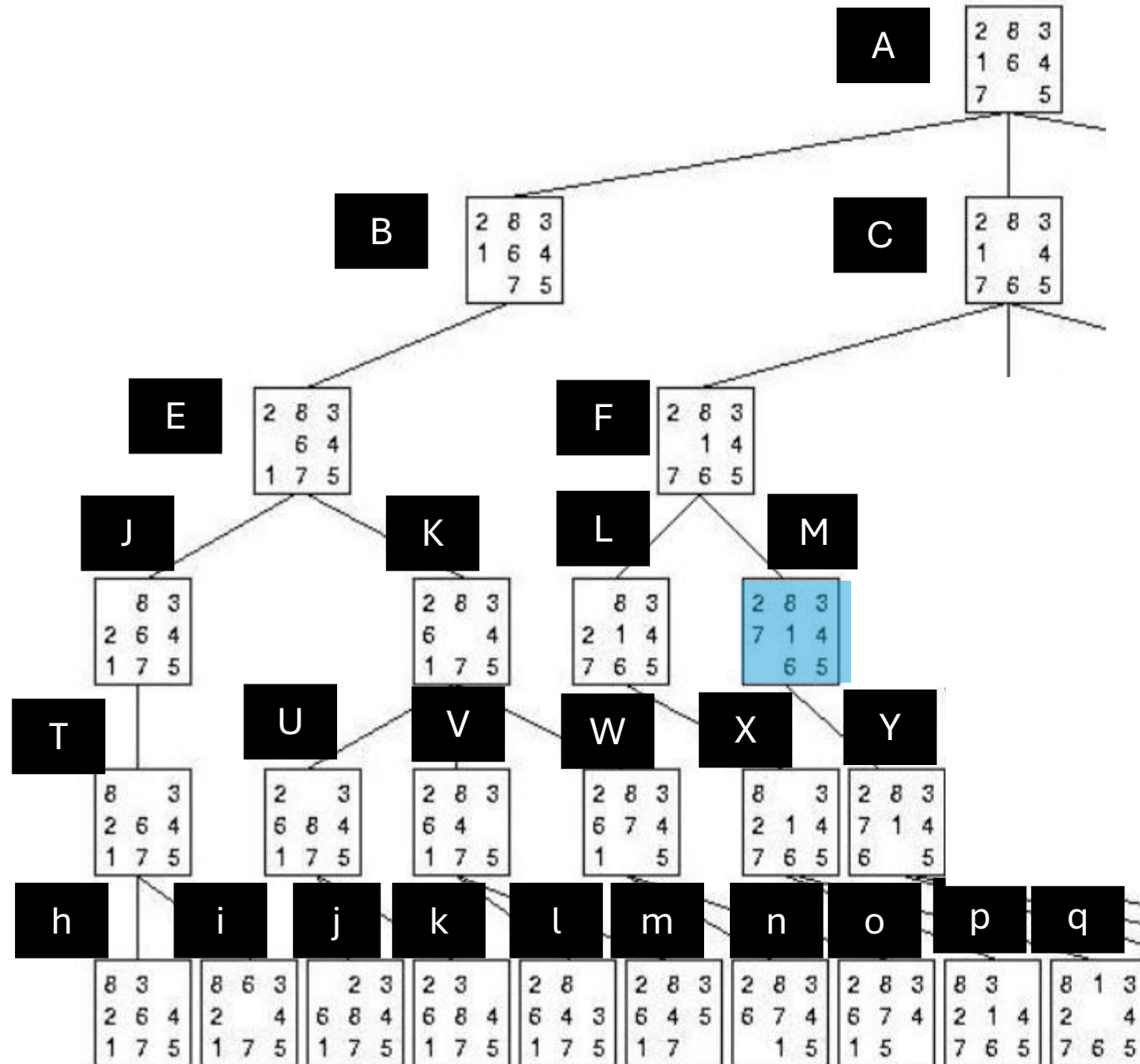
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X



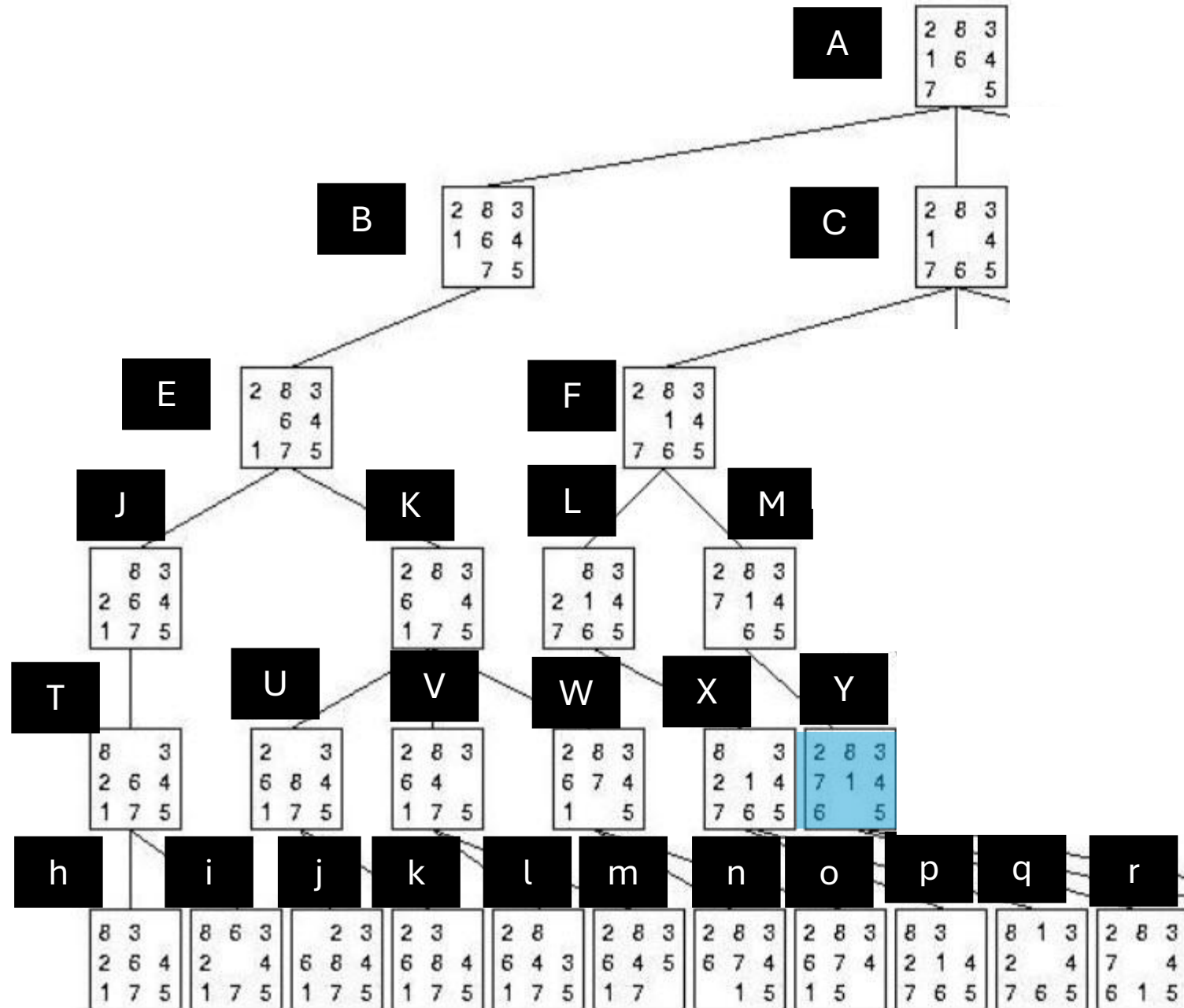
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p



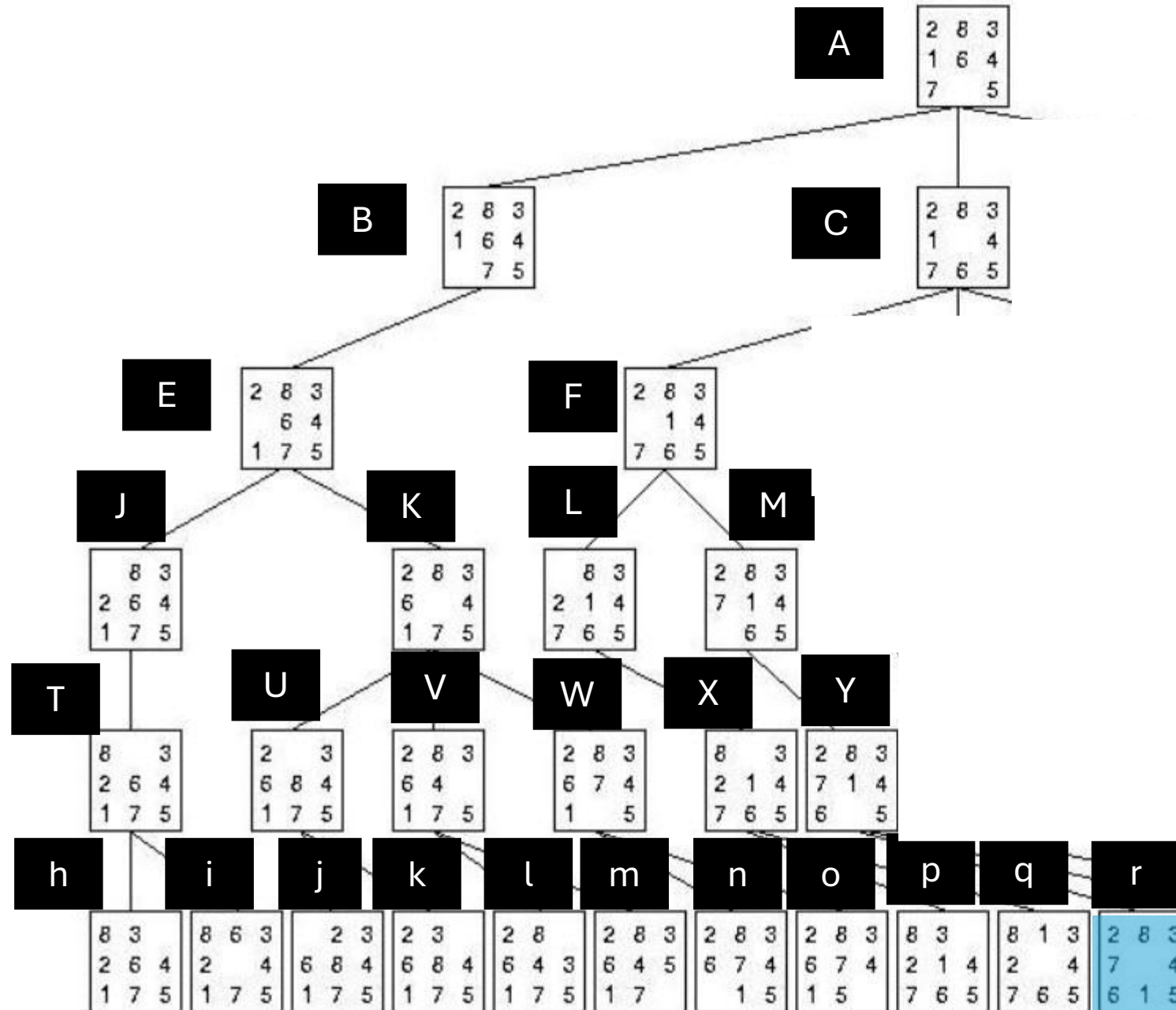
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p , q



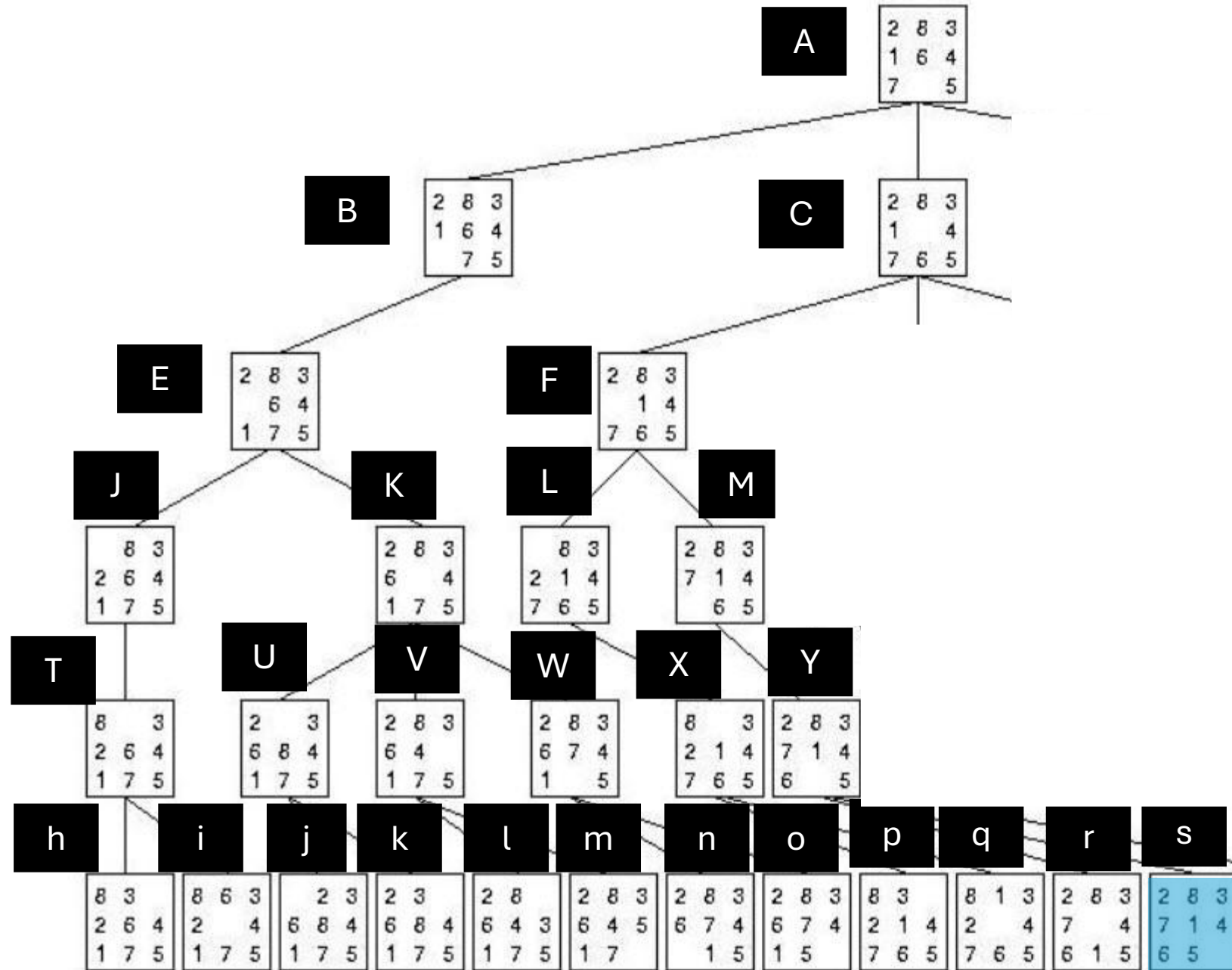
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p , q , M



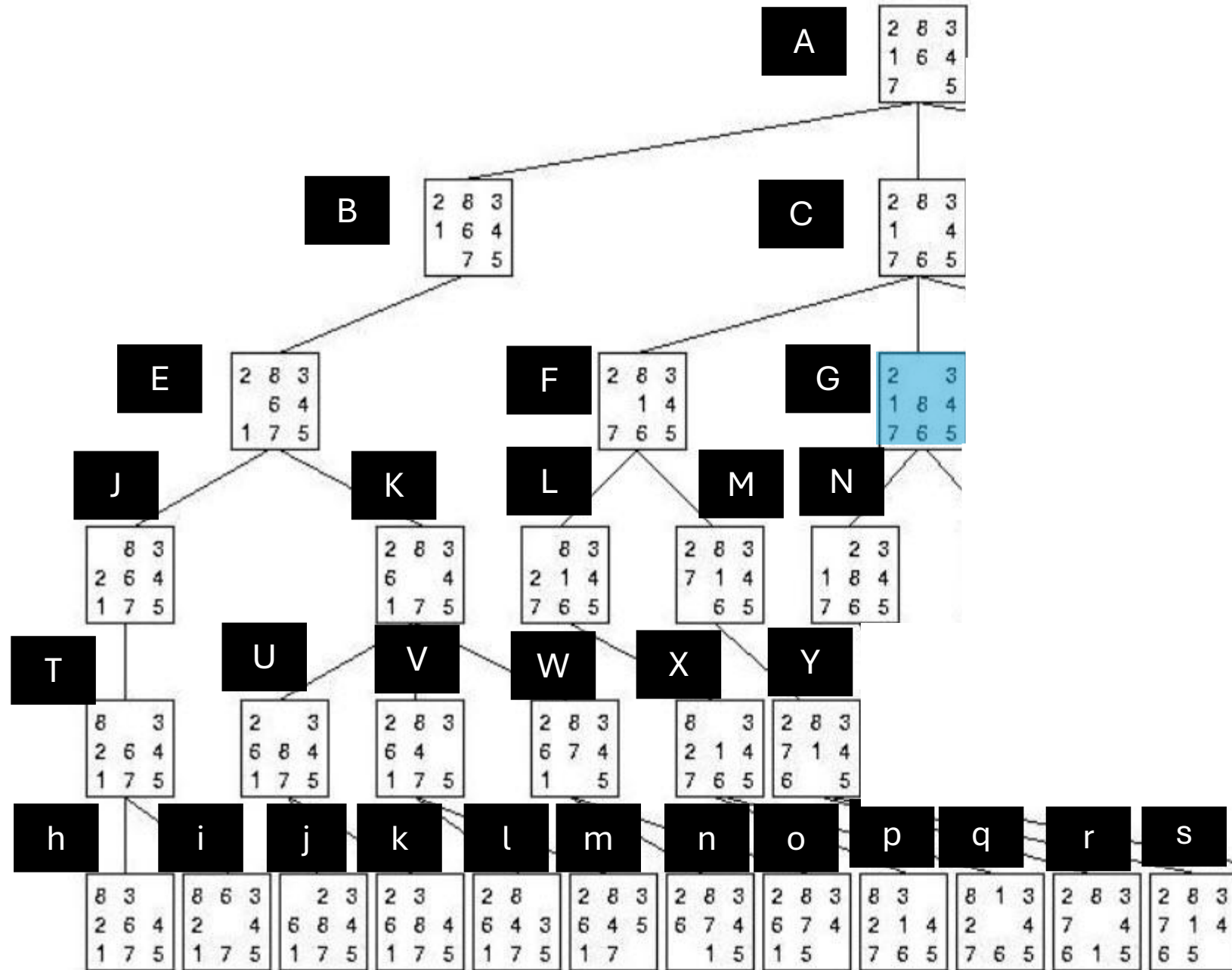
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p , q , M , Y



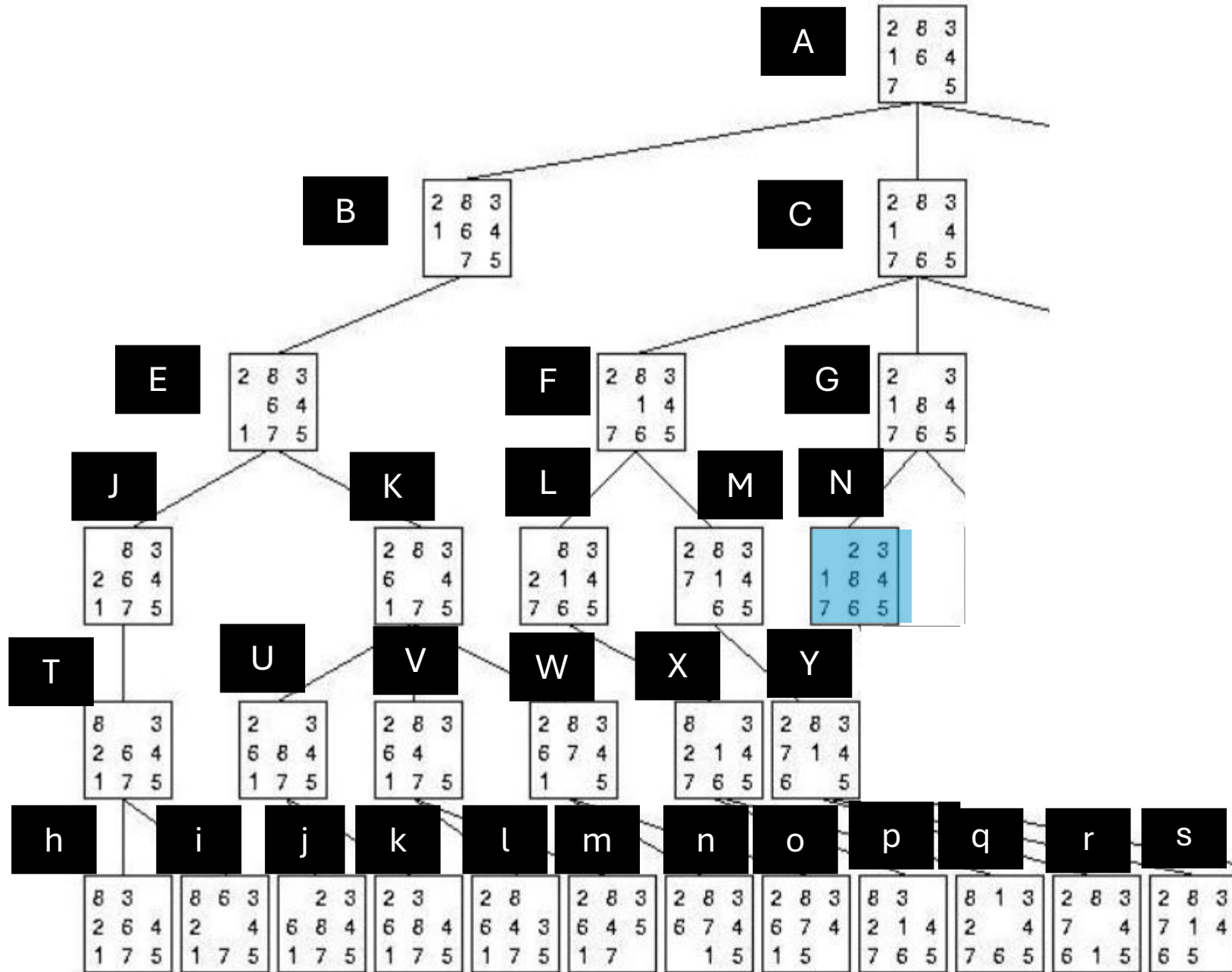
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p , q , M , Y , r



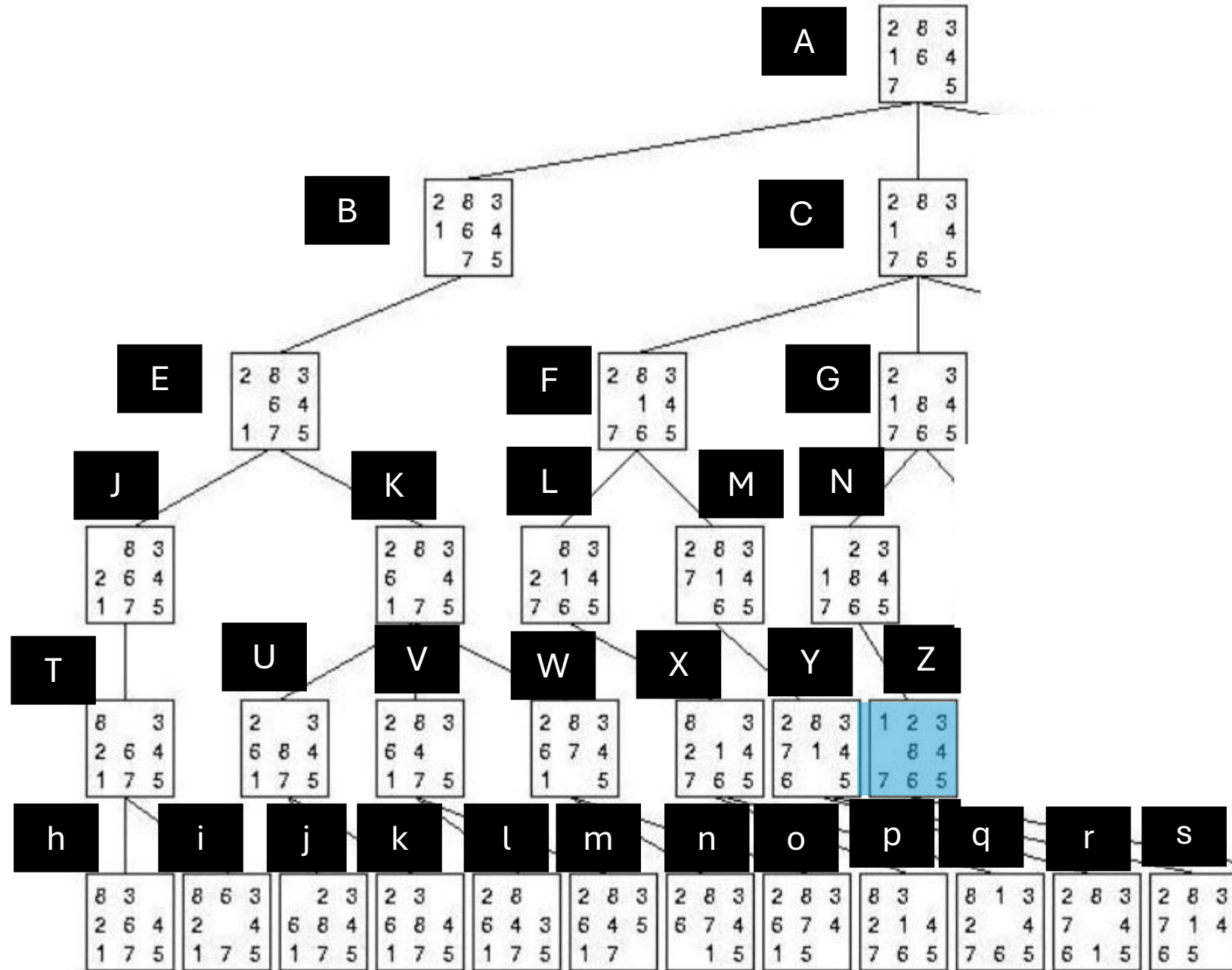
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p , q , M , Y , r , s



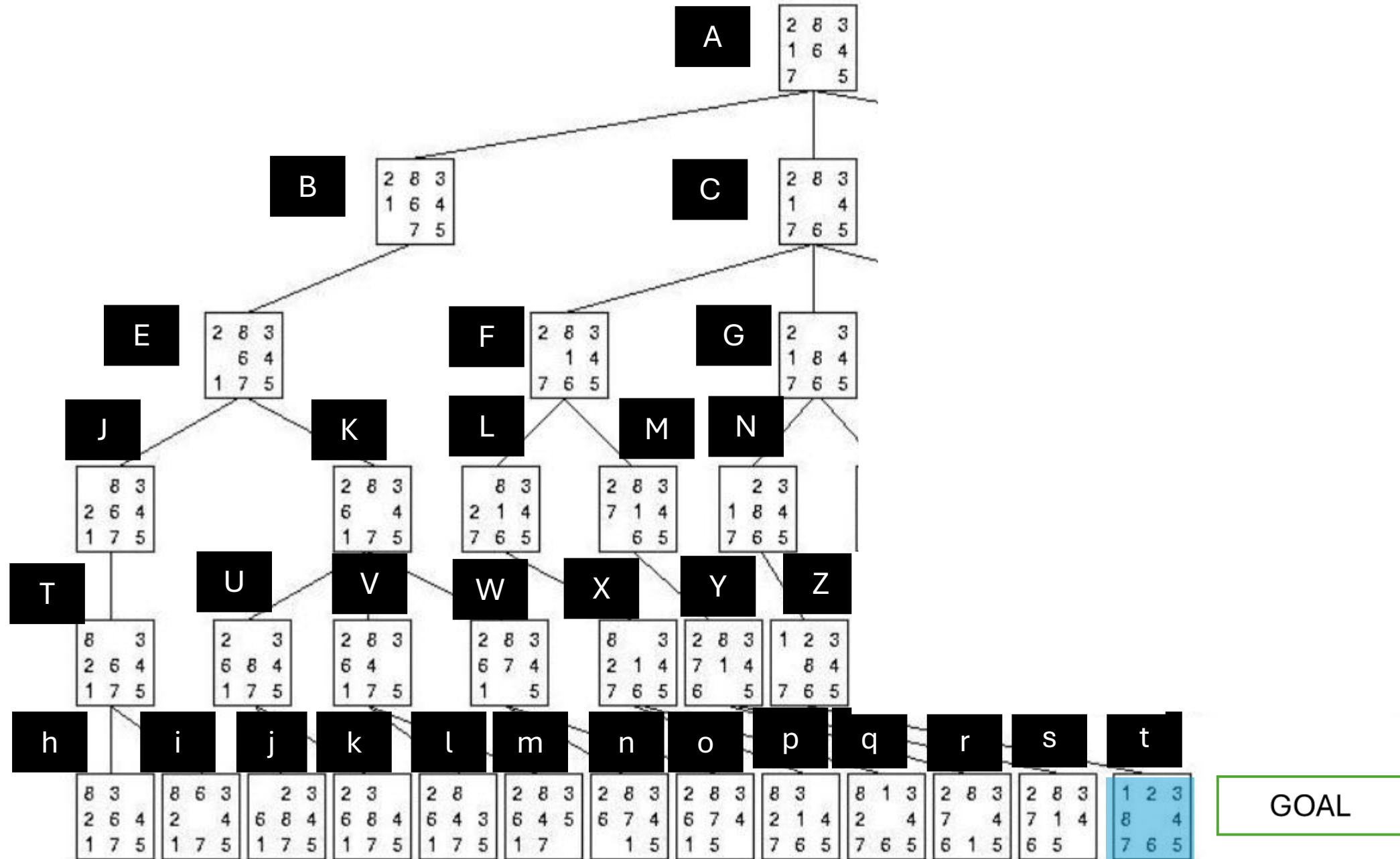
Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p , q , M , Y , r , s , G



Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p , q , M , Y , r , s , G , N



Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p , q , M , Y , r , s , G , N , Z



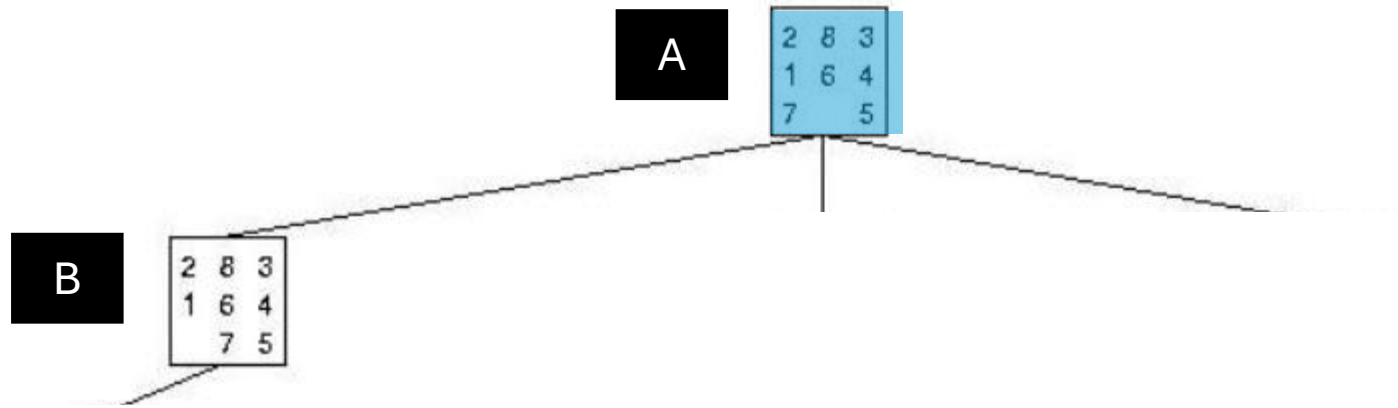
GOAL

Order : A , B , E , J , T , h , i , K , U , j , k , V , l , m , W , n , o , C , F , L , X , p , q , M , Y , r , s , G , N , Z , t

1- limited Depth-first

Same as Depth First Search, but with limited levels [$d = \text{max levels}$]

$d = 3$



Order : A ,

$d = 3$

A

2	8	3
1	6	4
7		5

B

2	8	3
1	6	4
7		5

E

2	8	3
	6	4
1	7	5

Order : A , B ,

d = 3

A

2	8	3
1	6	4
7		5

B

2	8	3
1	6	4
7		5

E

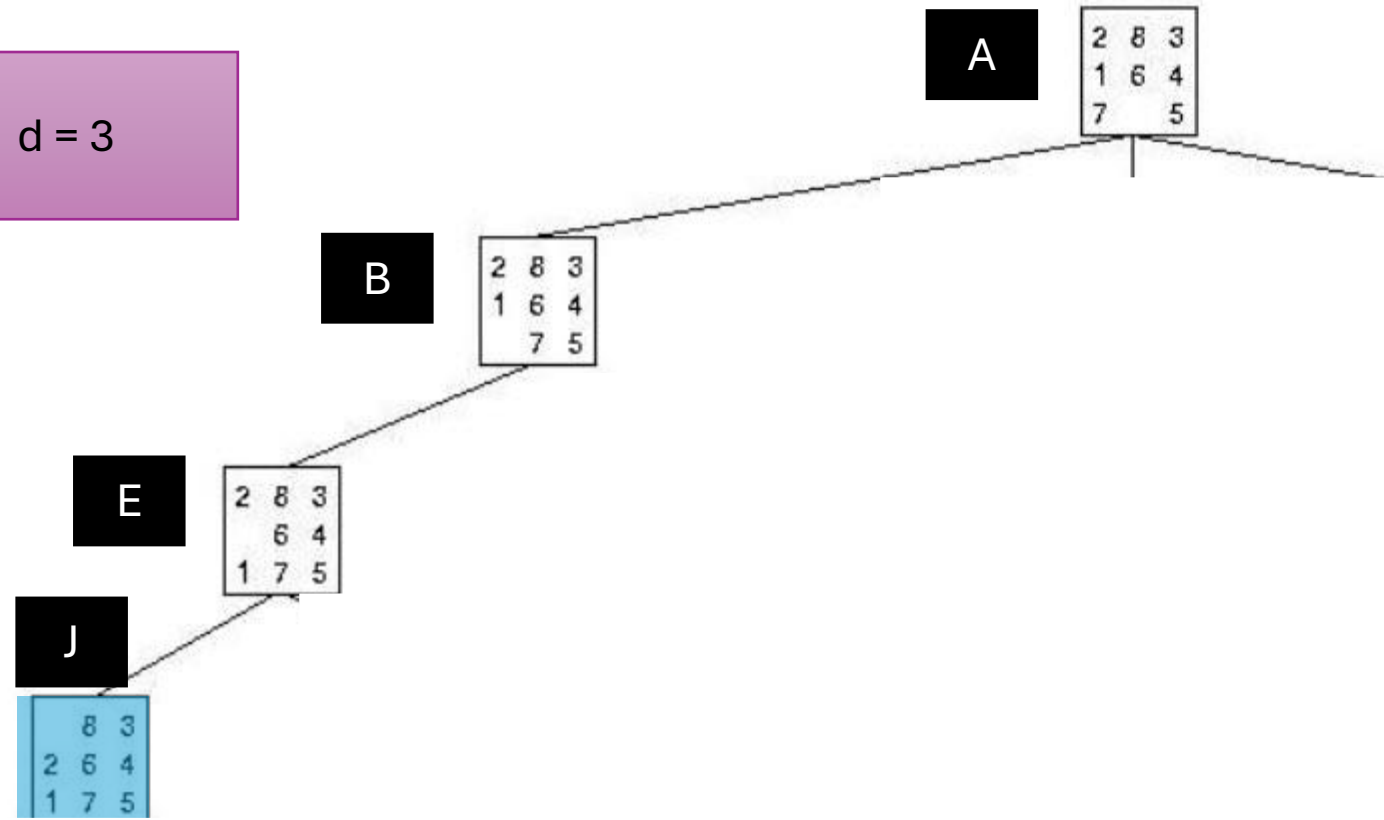
2	8	3
	6	4
1	7	5

J

	8	3
2	6	4
1	7	5

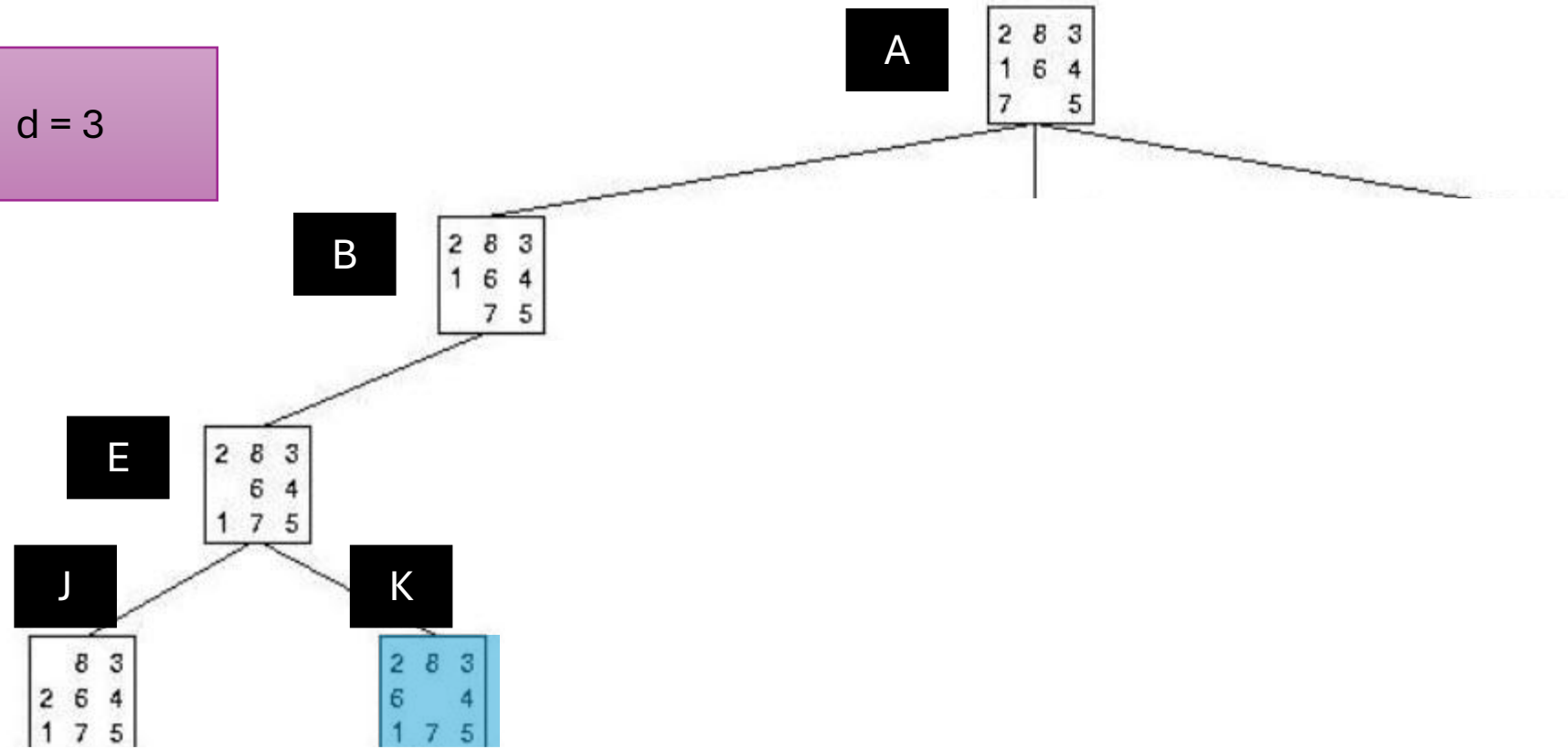
Order : A , B , E

d = 3



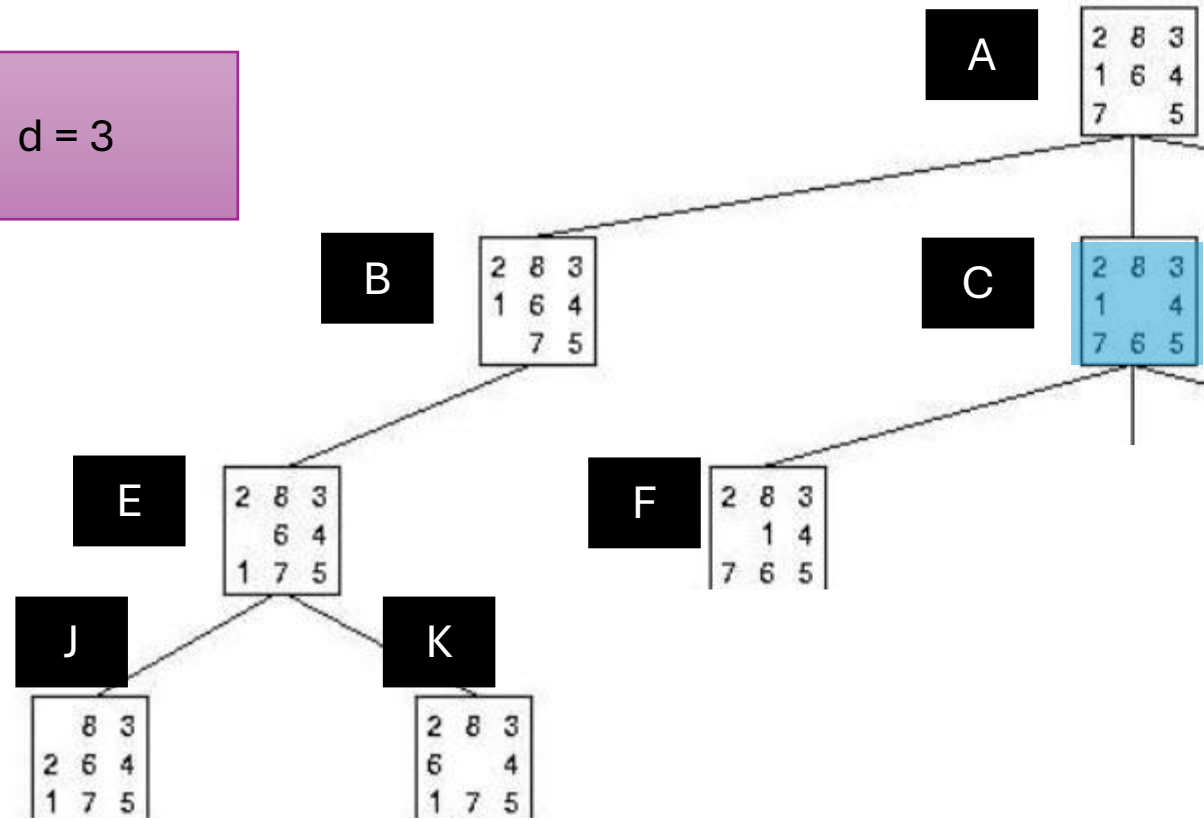
Order : A , B , E , J

d = 3



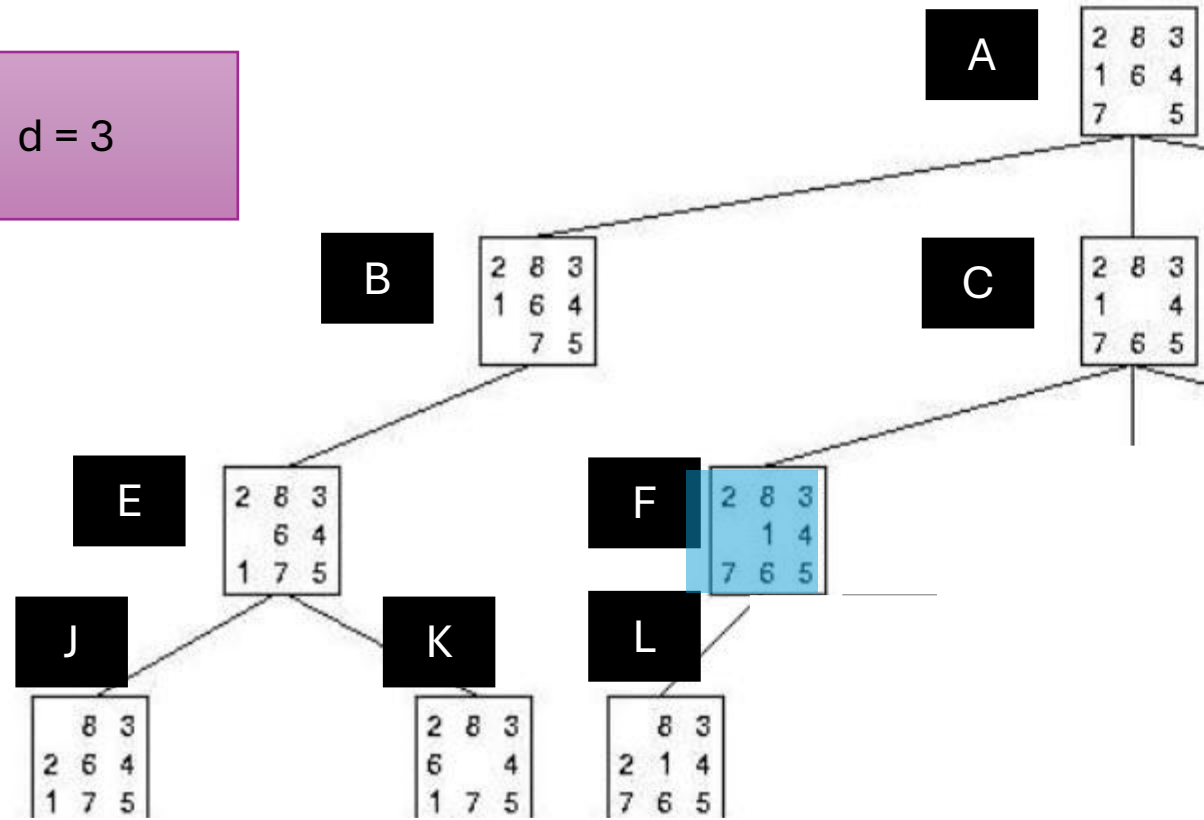
Order : A , B , E , J , K

d = 3



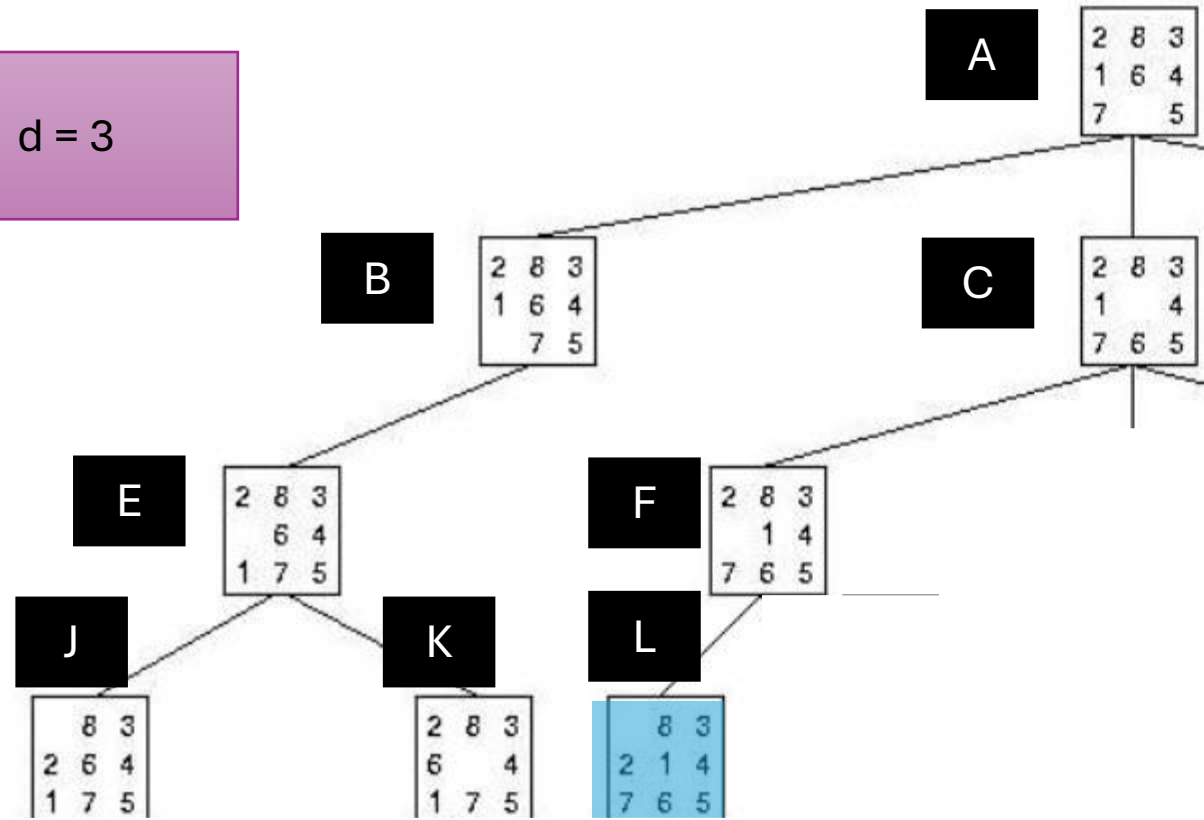
Order : A , B , E , J , K , C

d = 3



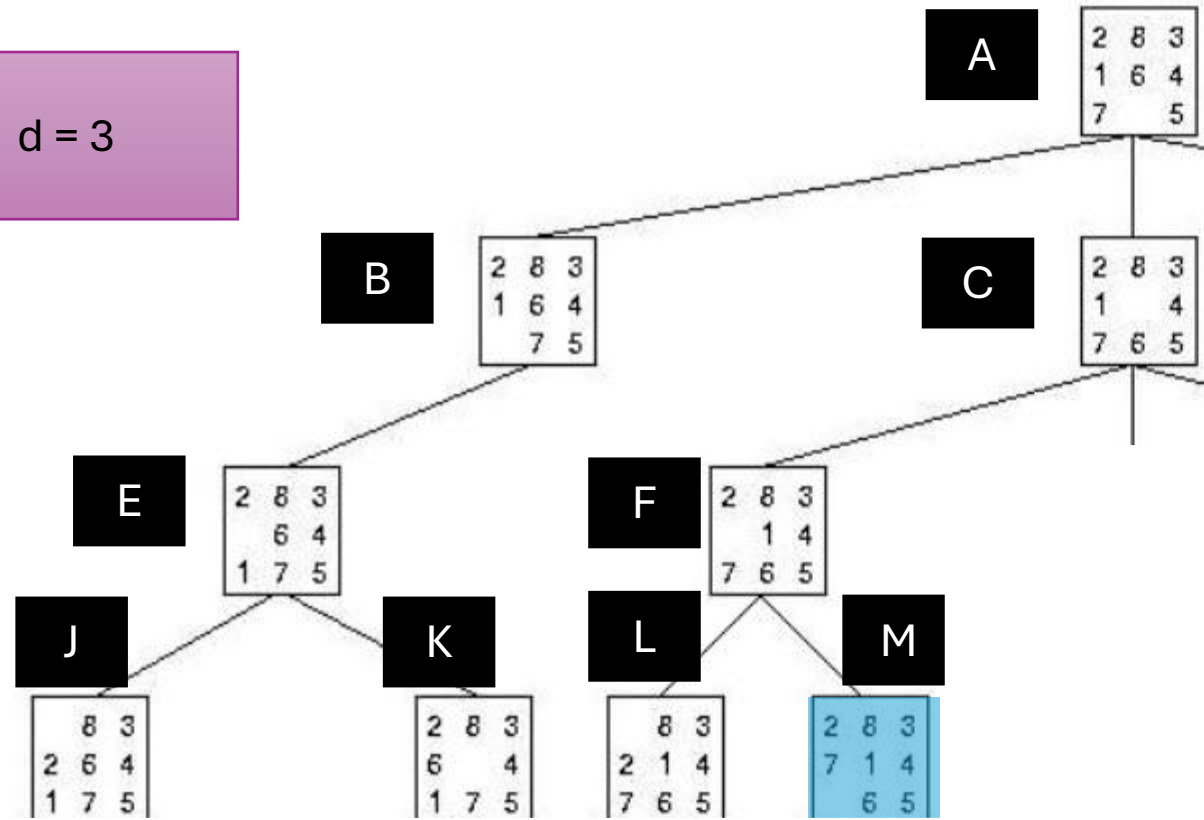
Order : A , B , E , J , K , C , F

d = 3



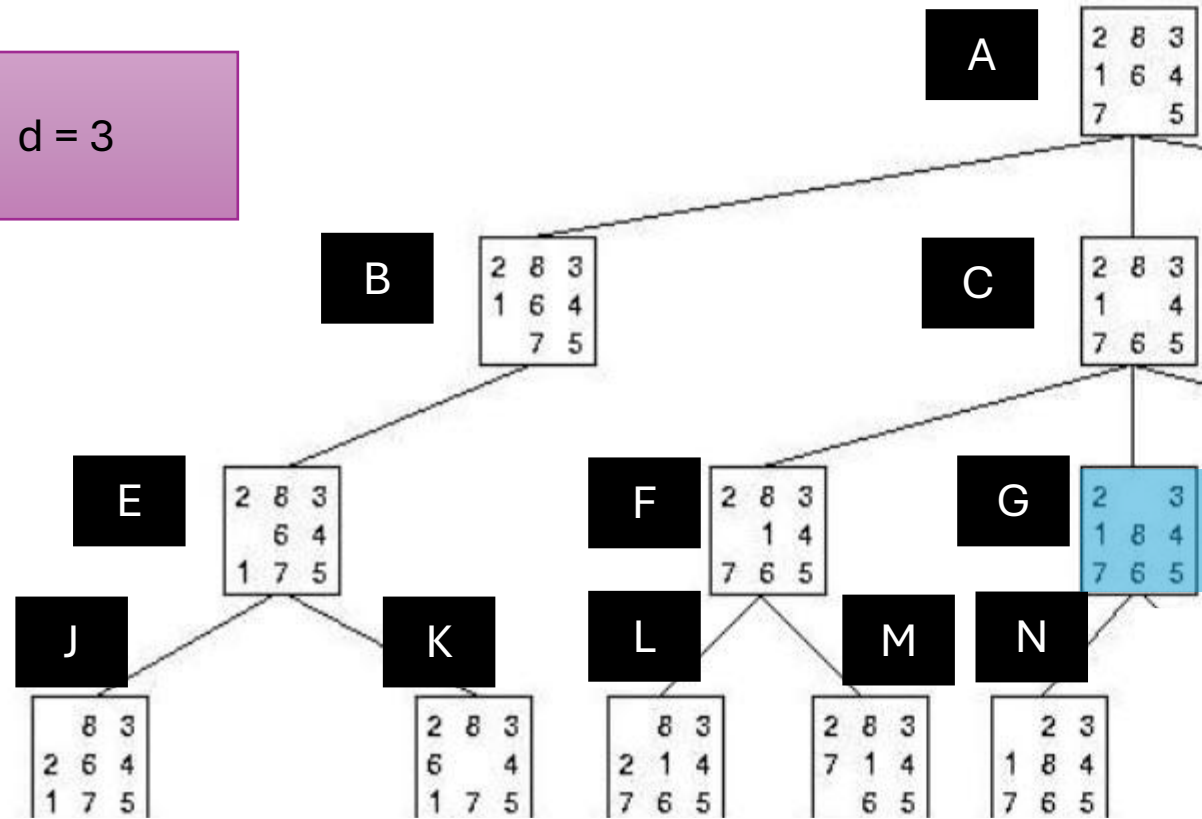
Order : A , B , E , J , K , C , F , L

d = 3



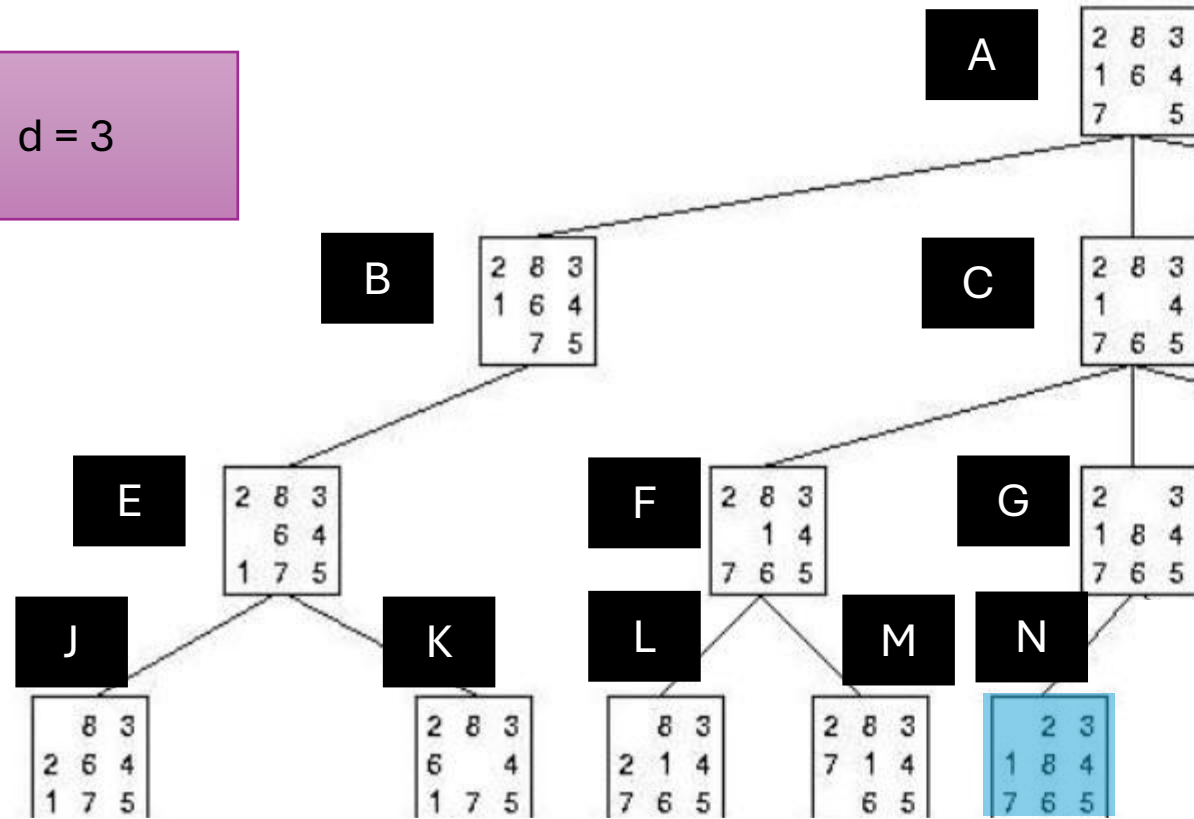
Order : A , B , E , J , K , C , F , L , M

$d = 3$



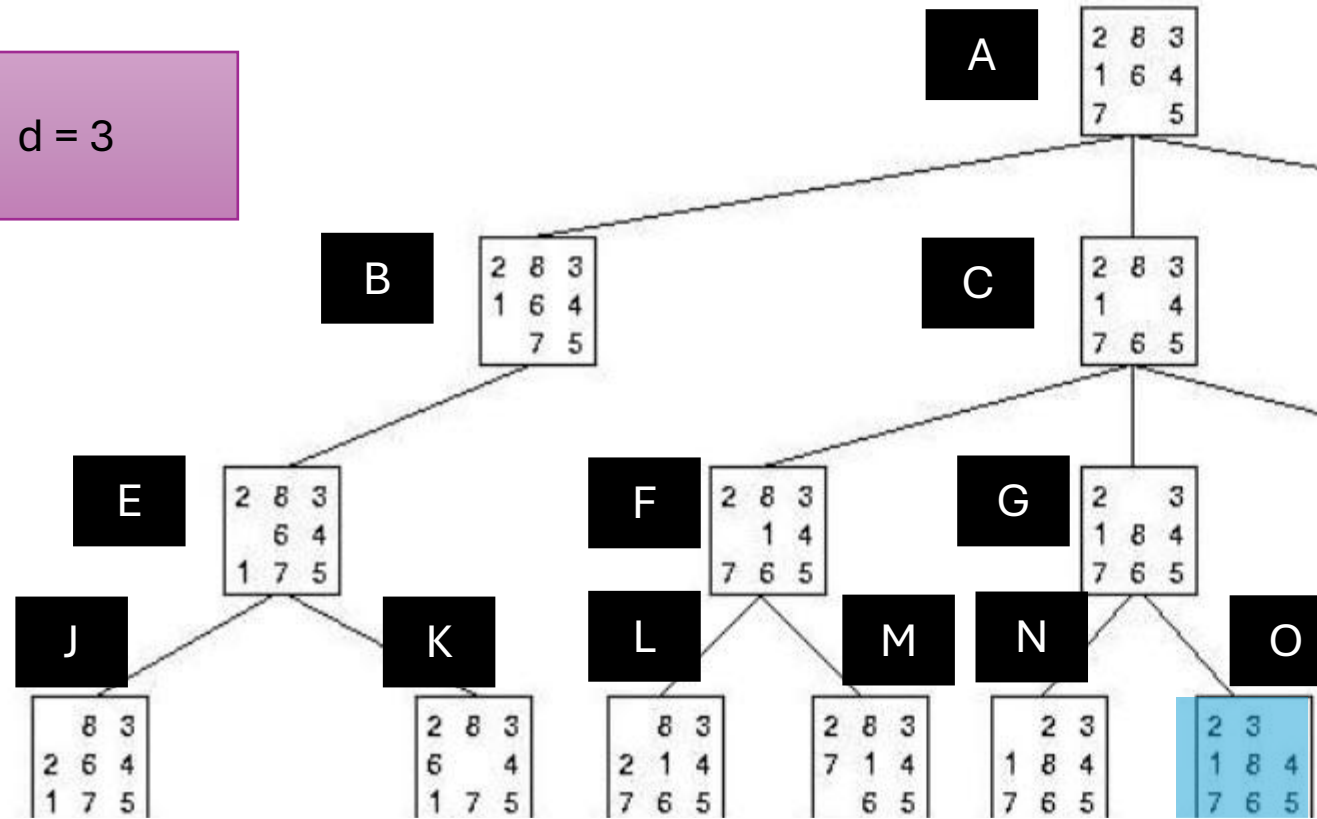
Order : A , B , E , J , K , C , F , L , M , G

$d = 3$



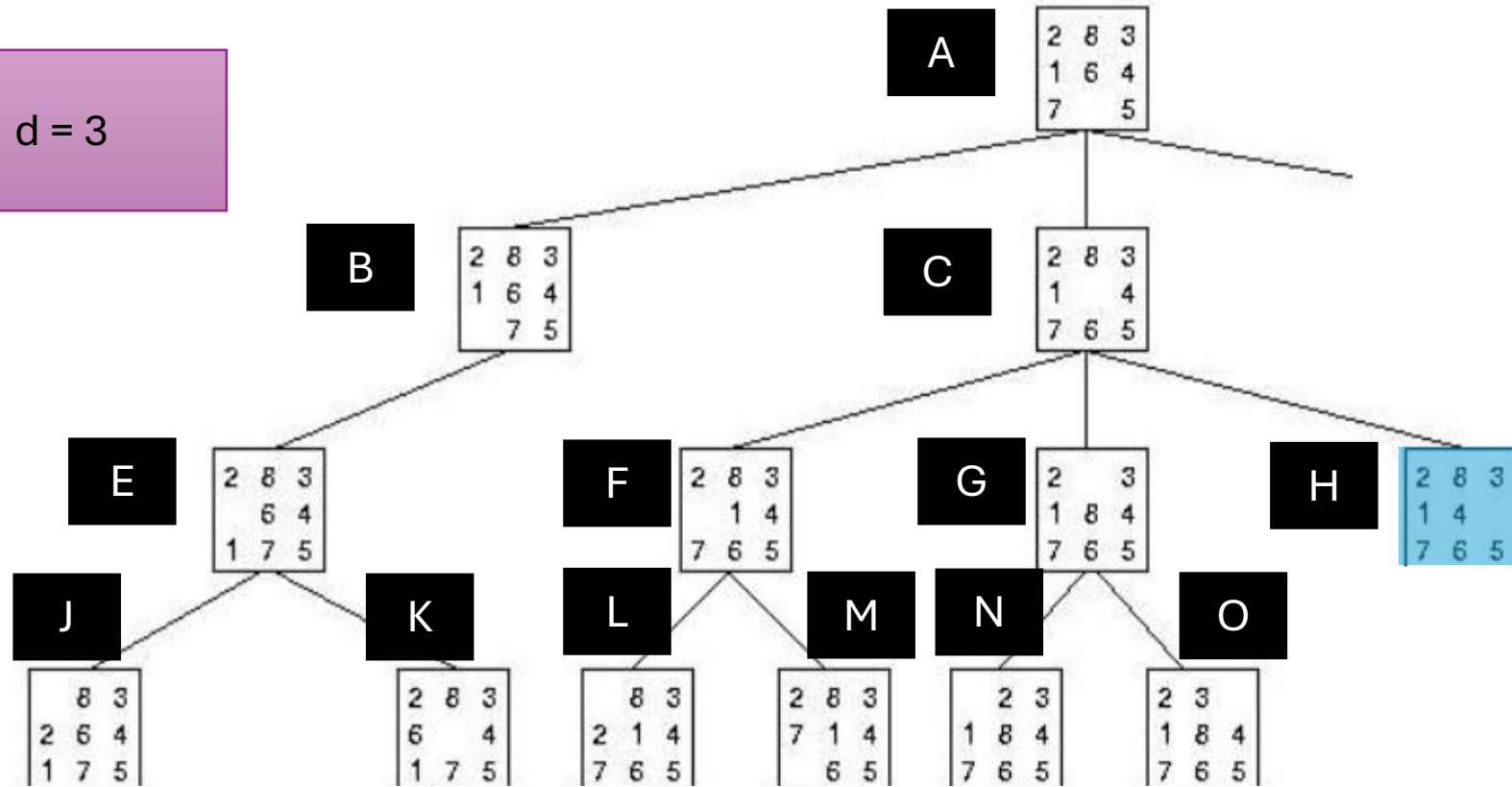
Order : A , B , E , J , K , C , F , L , M , G , N

$d = 3$



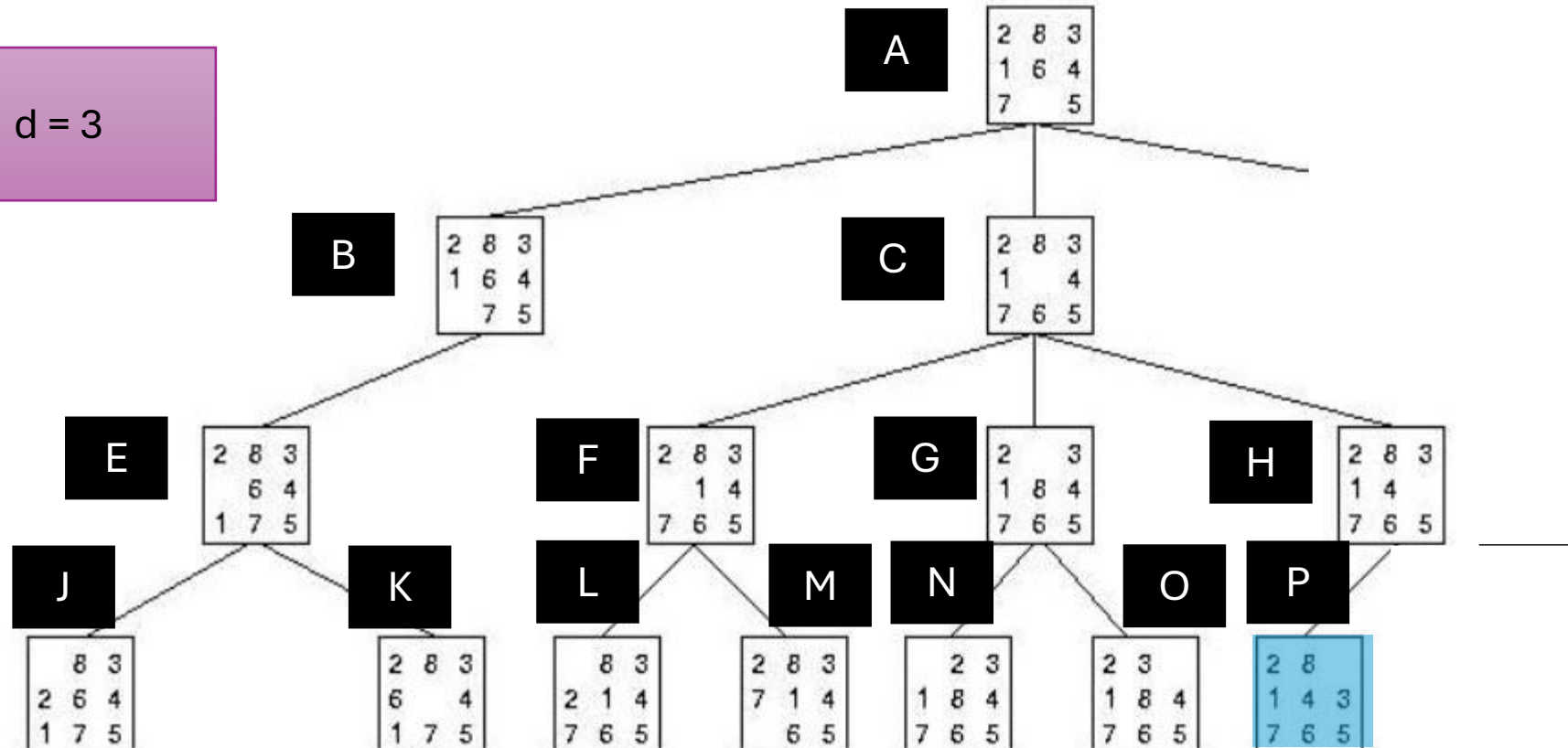
Order : A , B , E , J , K , C , F , L , M , G , N , O

$d = 3$



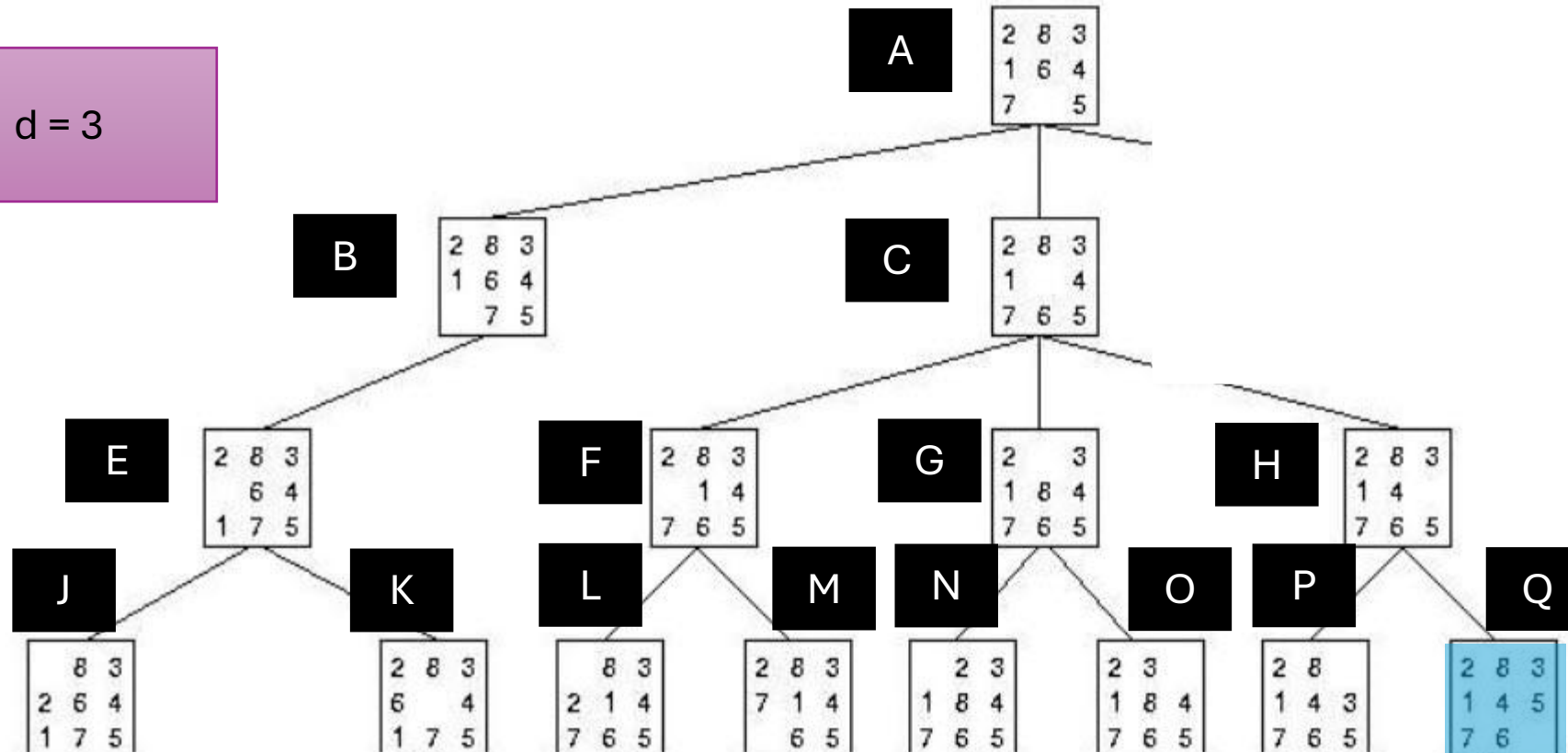
Order : A , B , E , J , K , C , F , L , M , G , N , O , H

d = 3



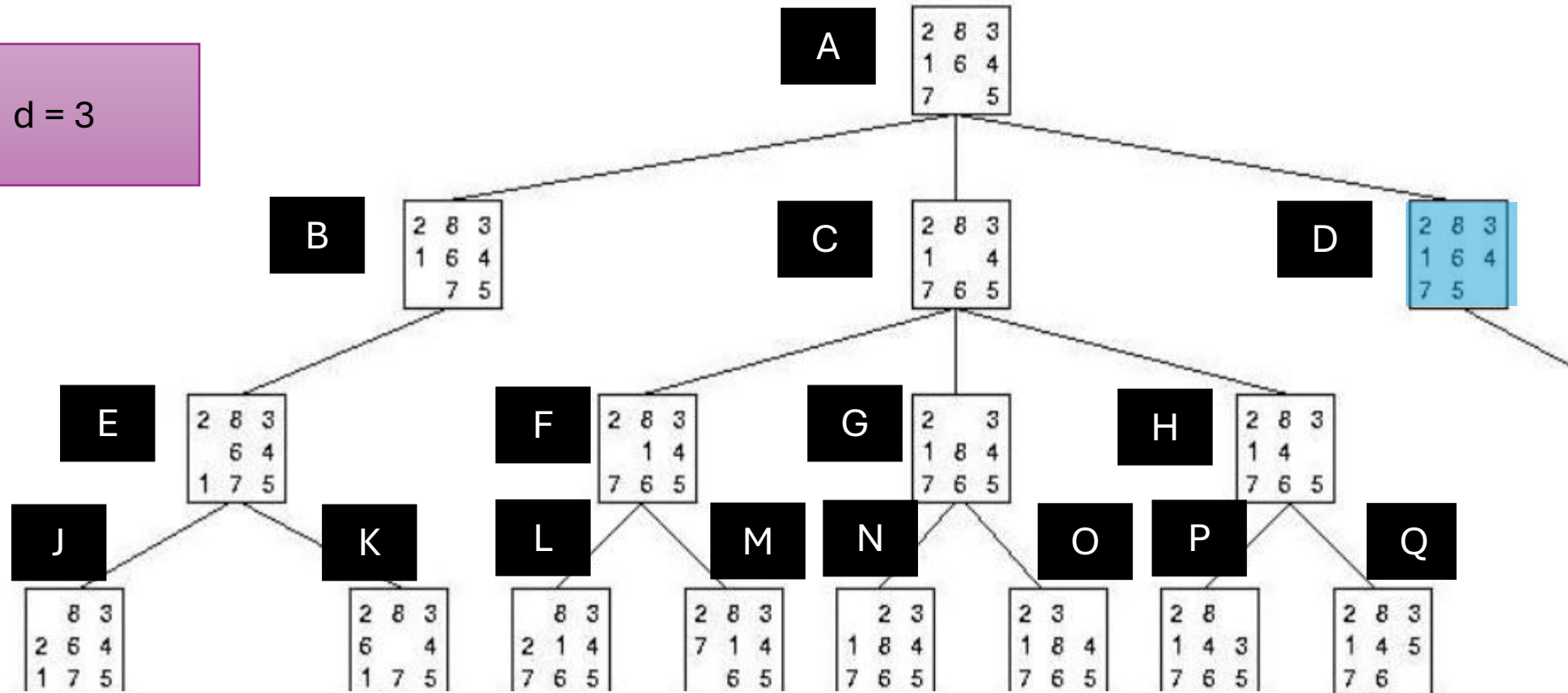
Order : A , B , E , J , K , C , F , L , M , G , N , O , H , P

d = 3



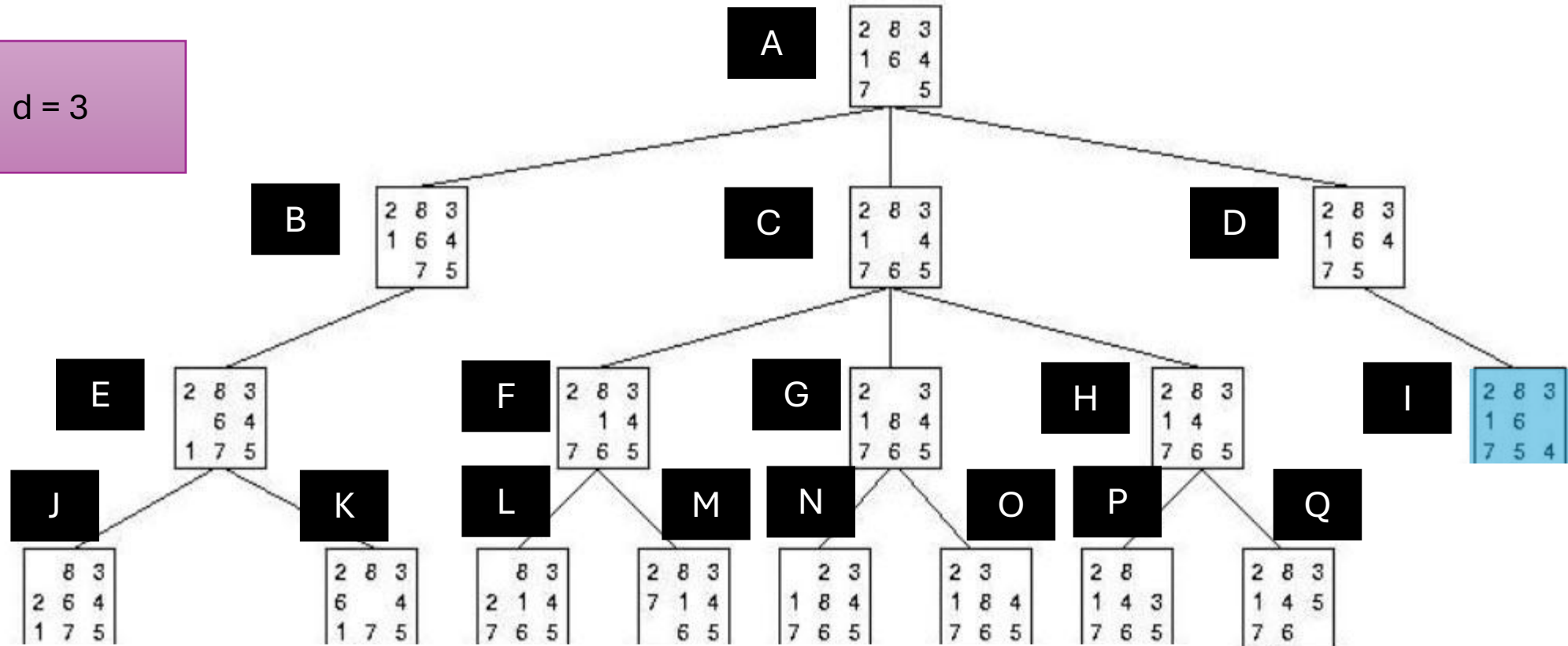
Order : A , B , E , J , K , C , F , L , M , G , N , O , H , P , Q

$d = 3$



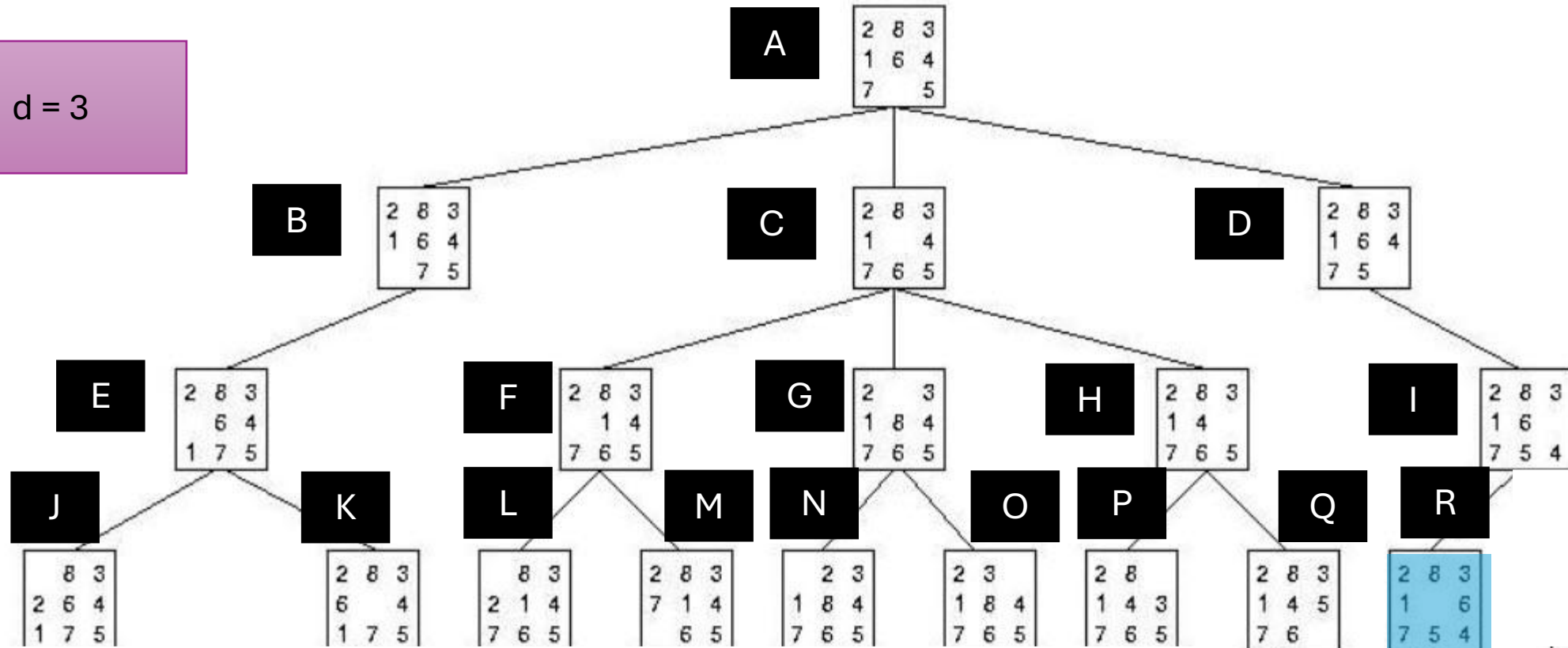
Order : A , B , E , J , K , C , F , L , M , G , N , O , H , P , Q , D

d = 3



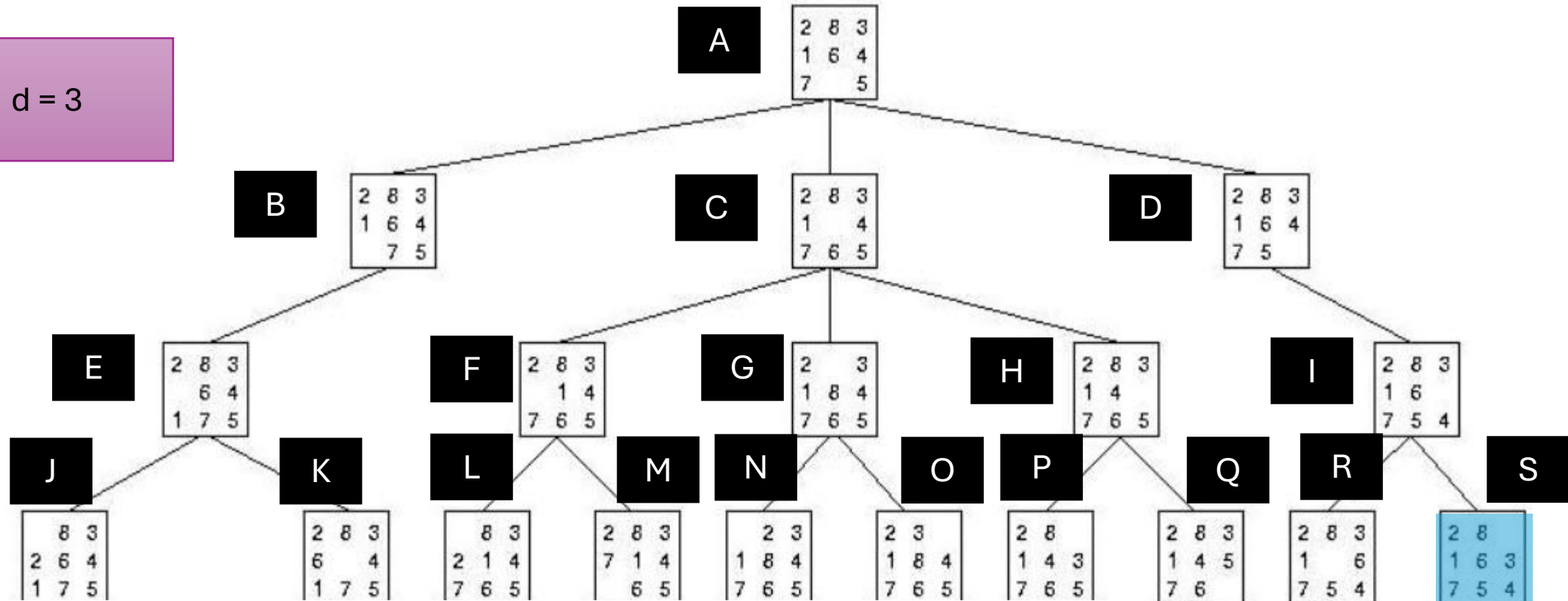
Order : A , B , E , J , K , C , F , L , M , G , N , O , H , P , Q , D , I

d = 3



Order : A , B , E , J , K , C , F , L , M , G , N , O , H , P , Q , D , I , R

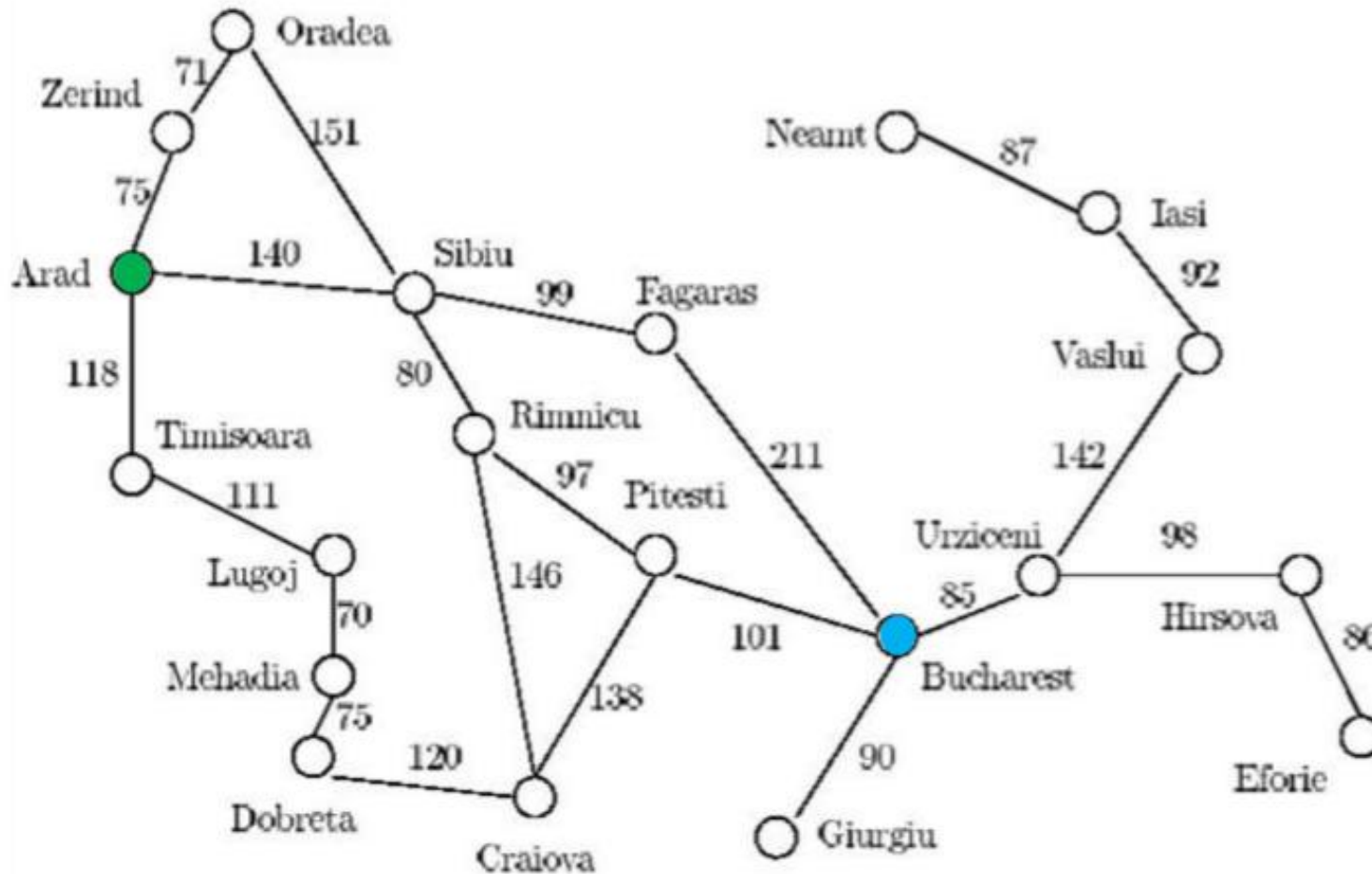
d = 3



Note : Limited Depth First can't always find the goal

Order : A , B , E , J , K , C , F , L , M , G , N , O , H , P , Q , D , I , R , S

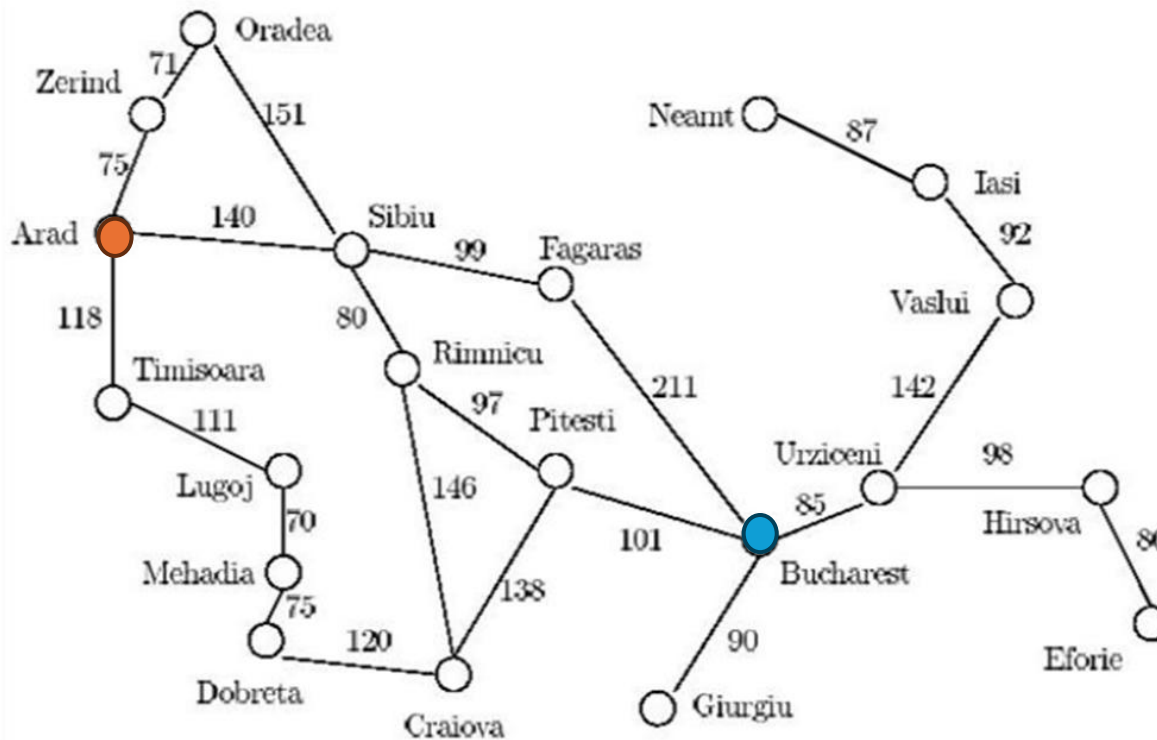
1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:



- 1 – Breadth first
- 2- Depth first

1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

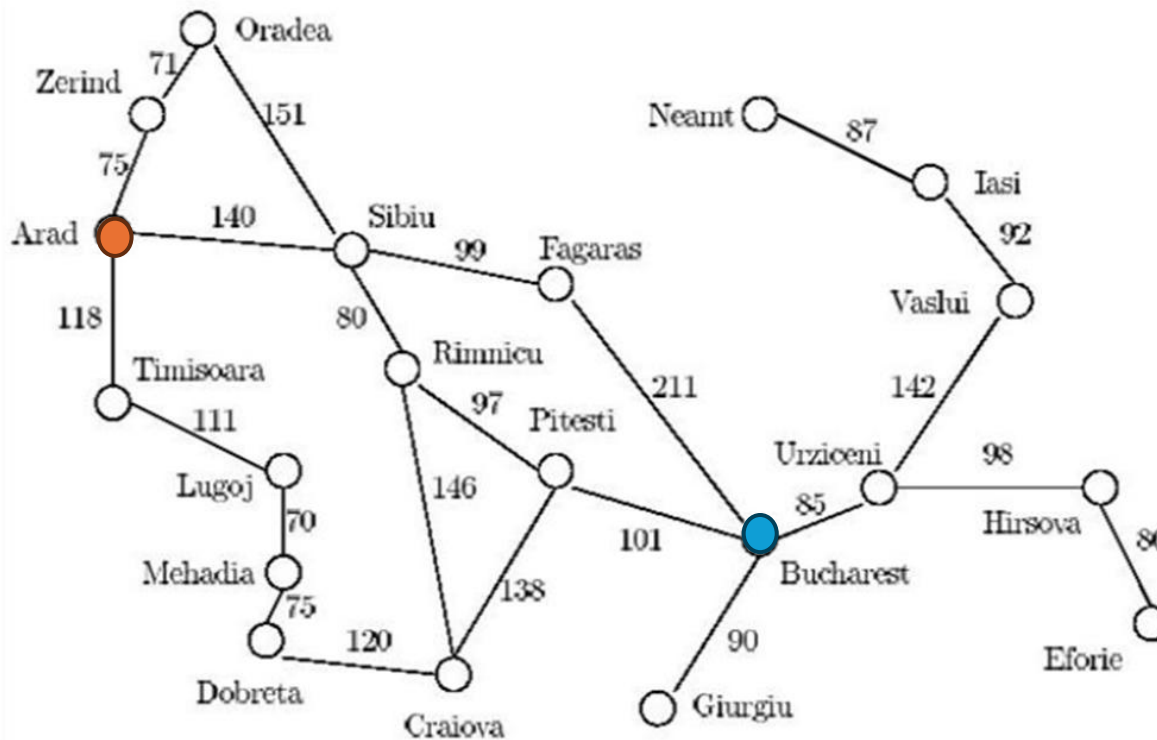


- 1 – Breadth first

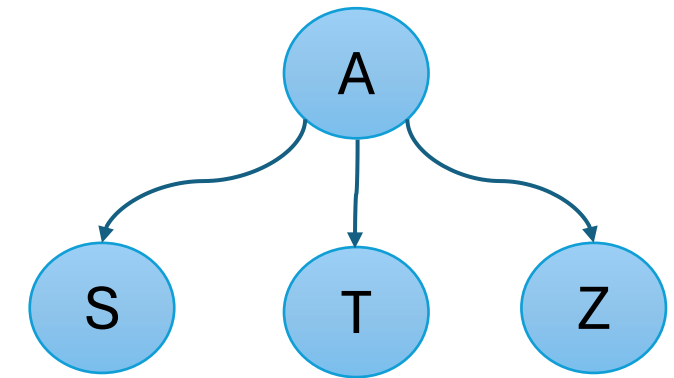
A

1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

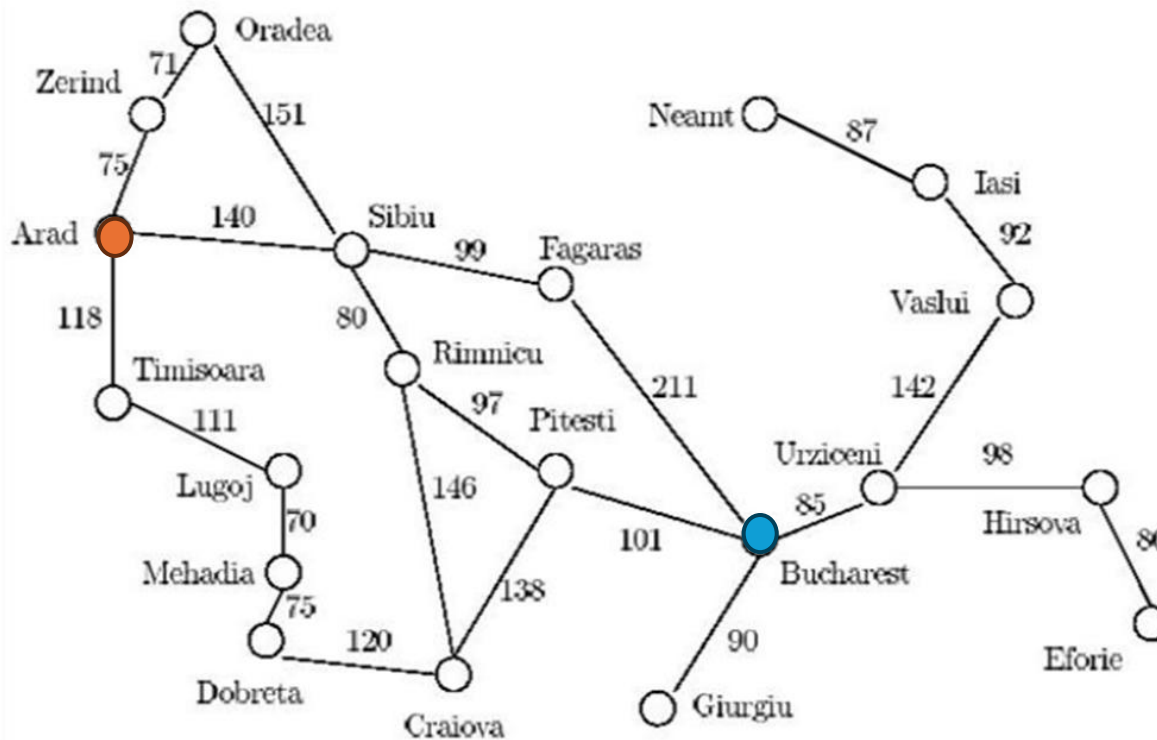


- 1 – Breadth first

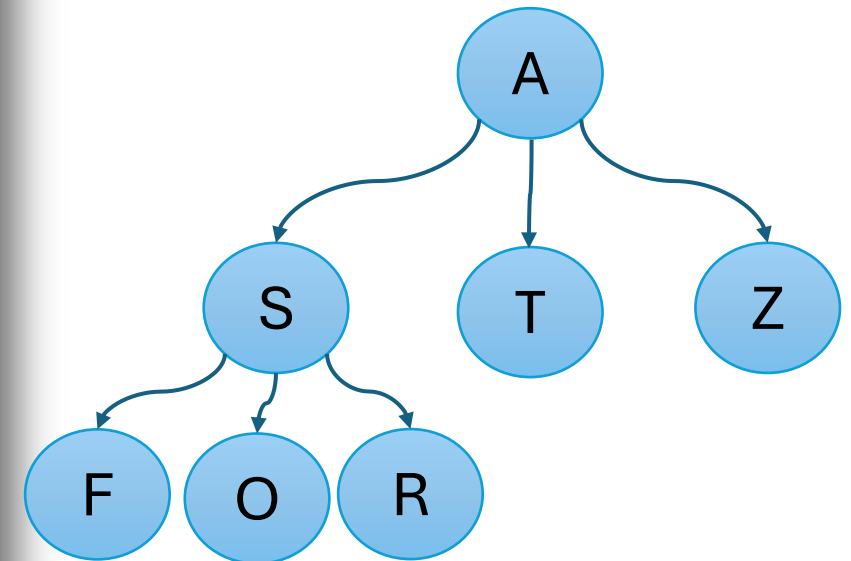


1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

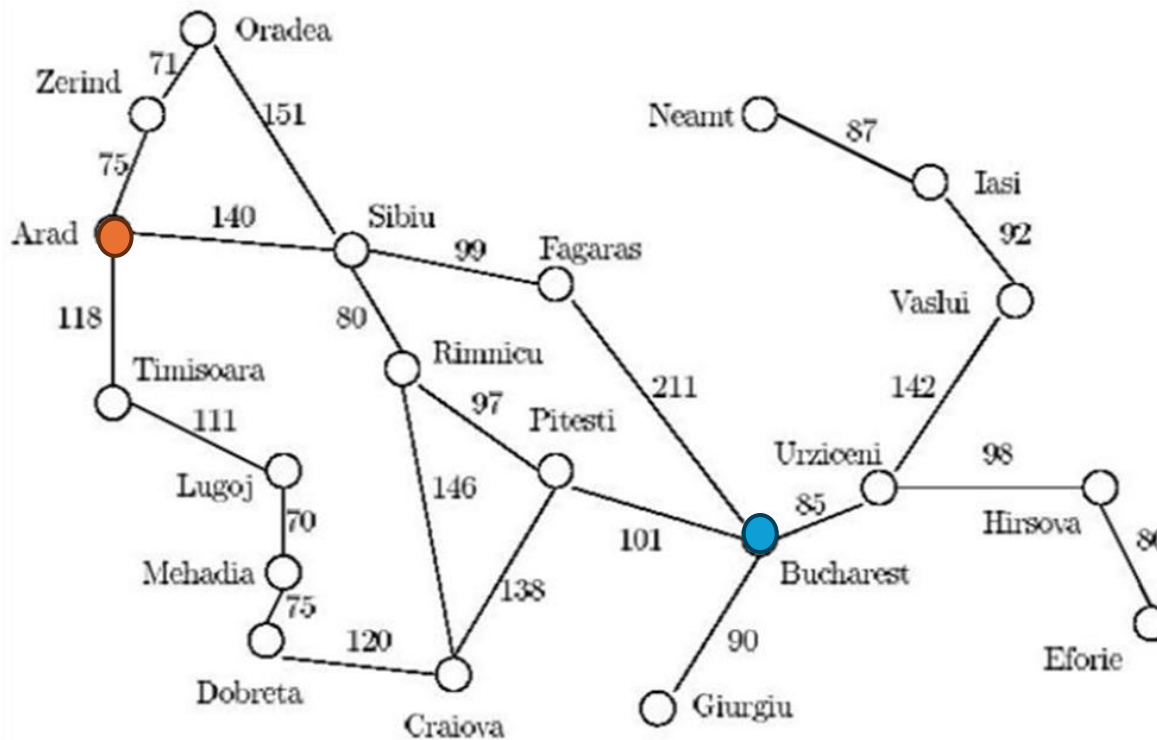


- 1 – Breadth first

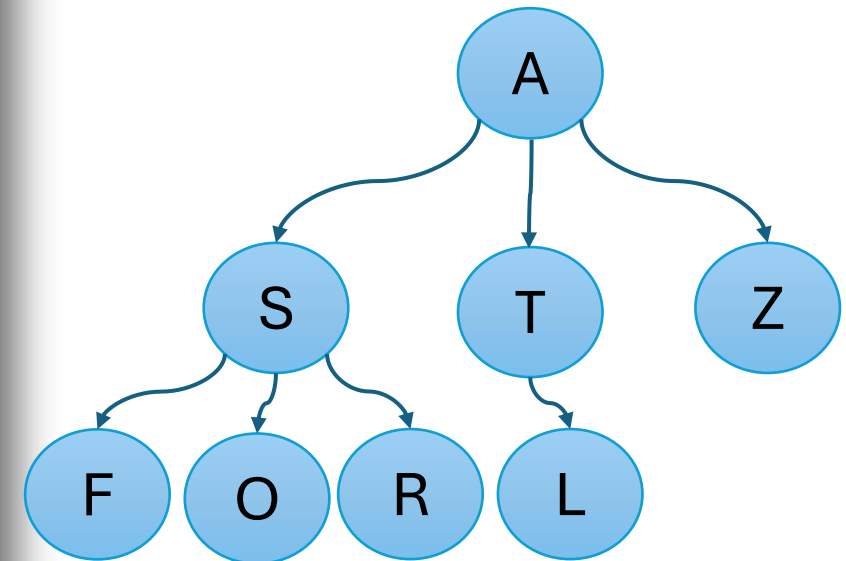


1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

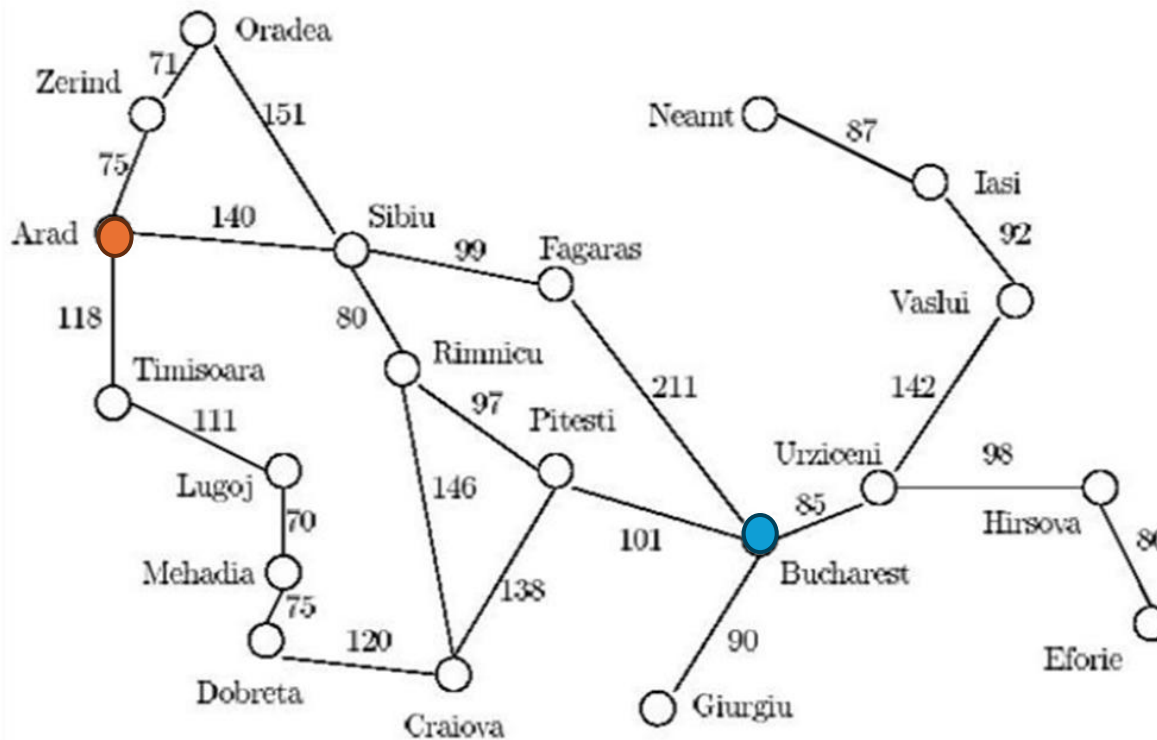


- 1 – Breadth first

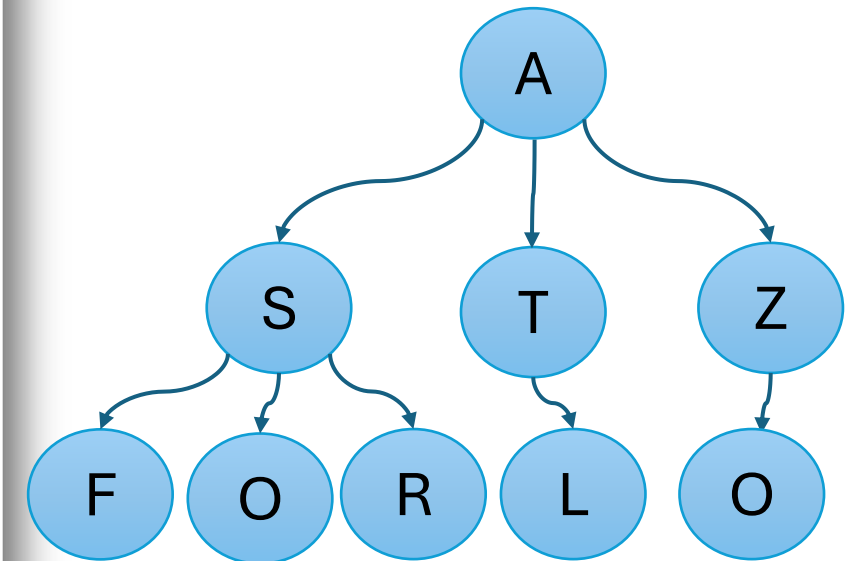


1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

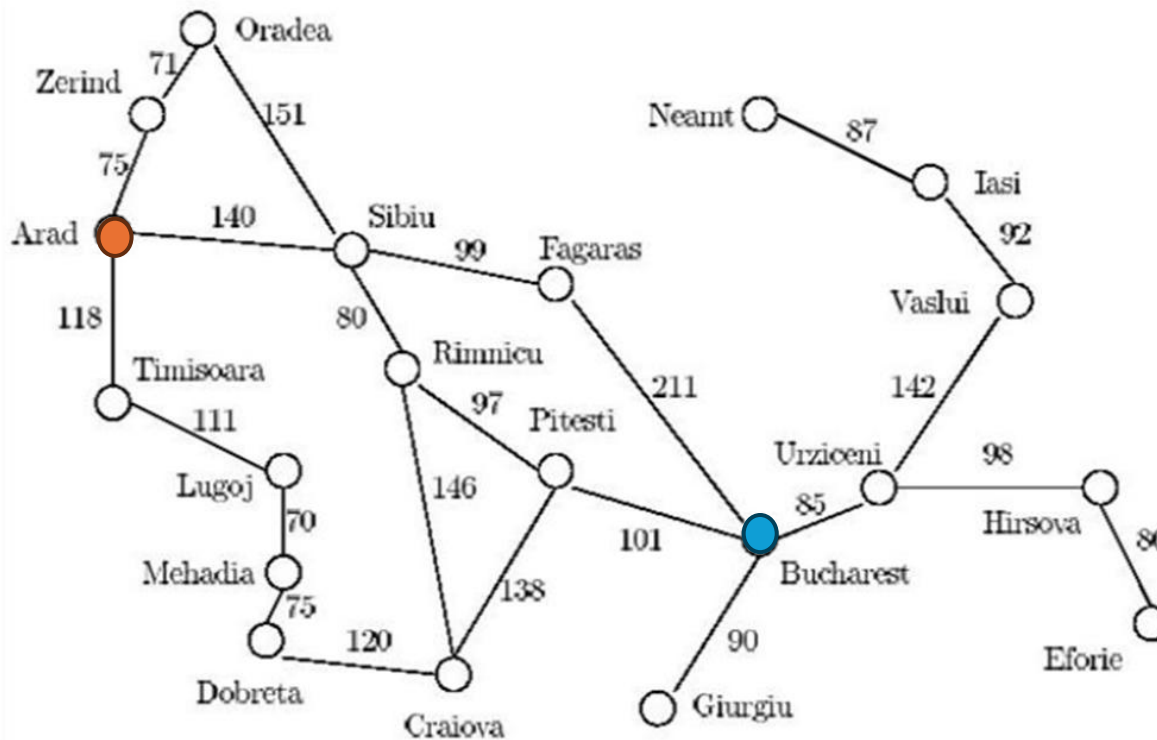


- 1 – Breadth first

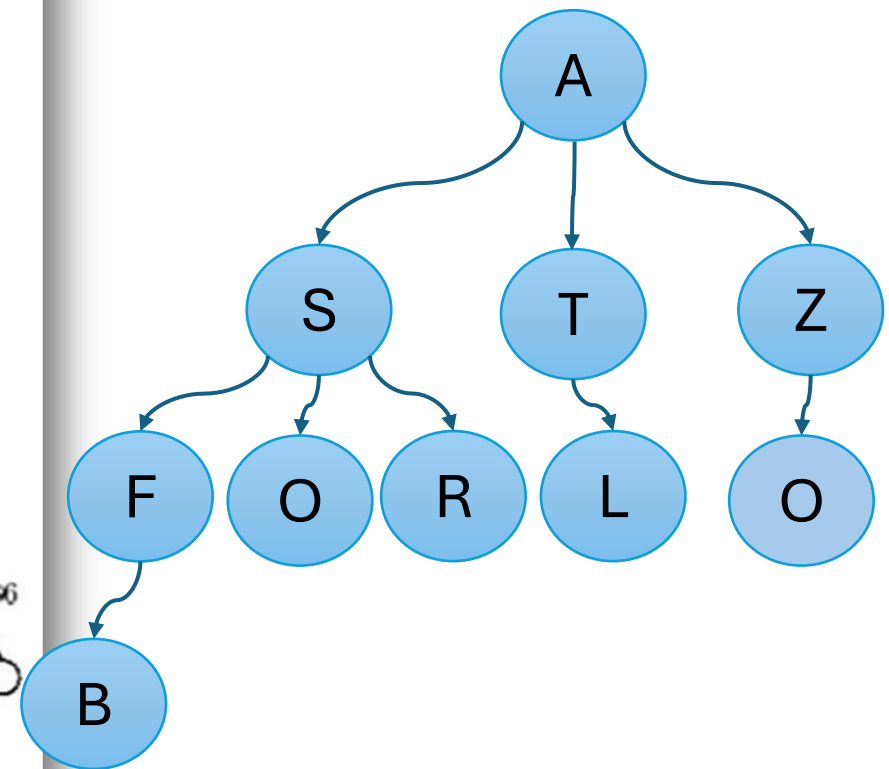


1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

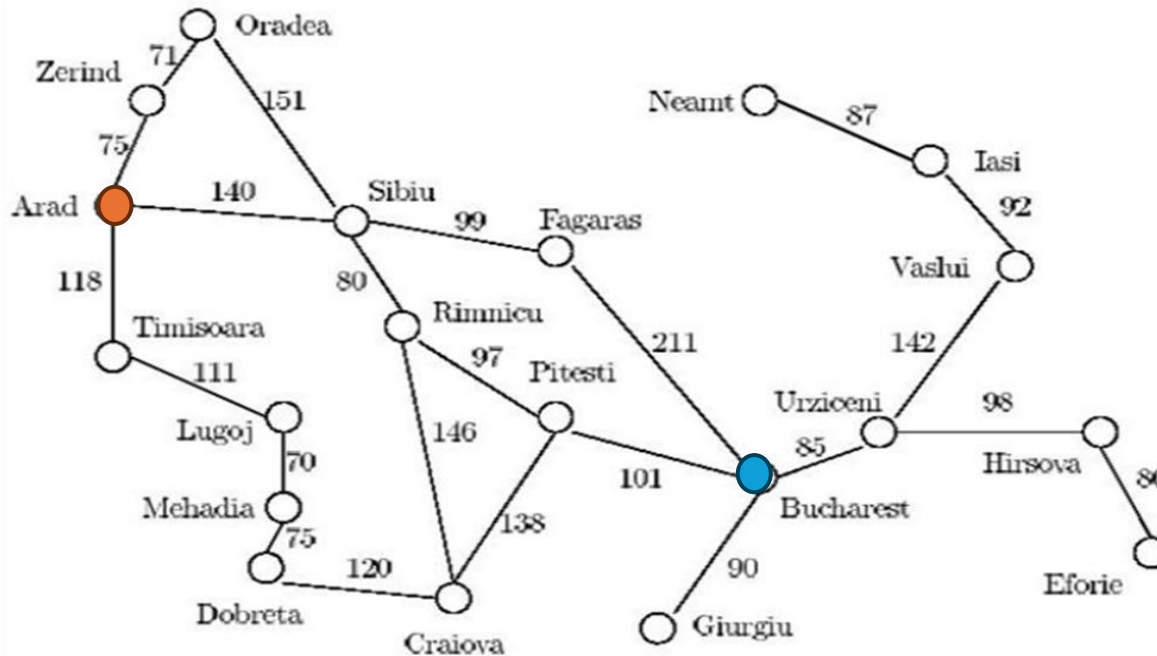


- 1 – Breadth first



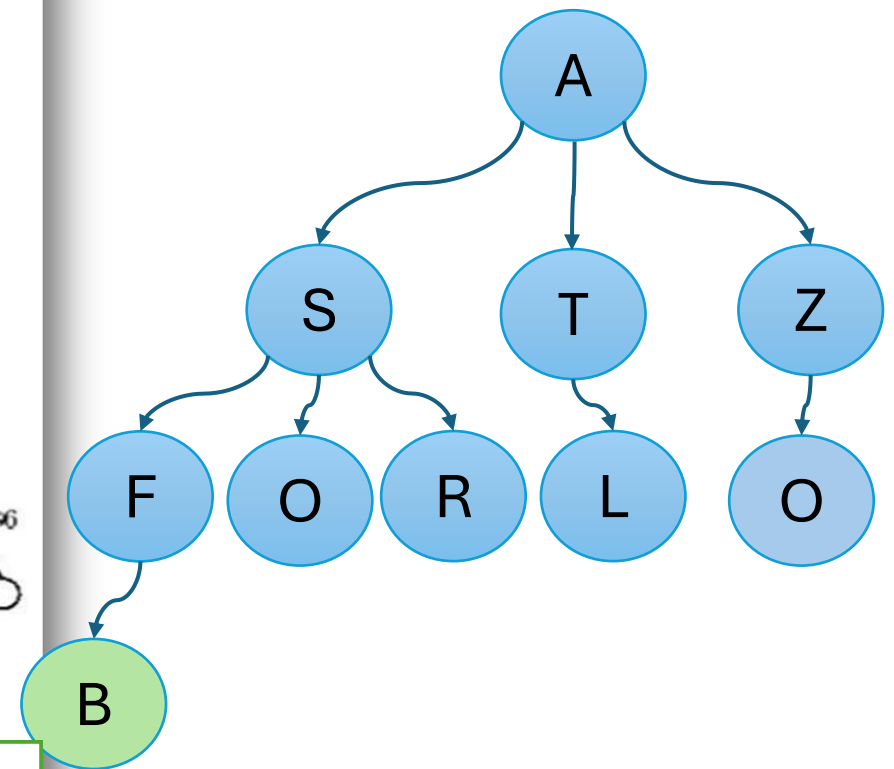
1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



GOAL

- 1 – Breadth first

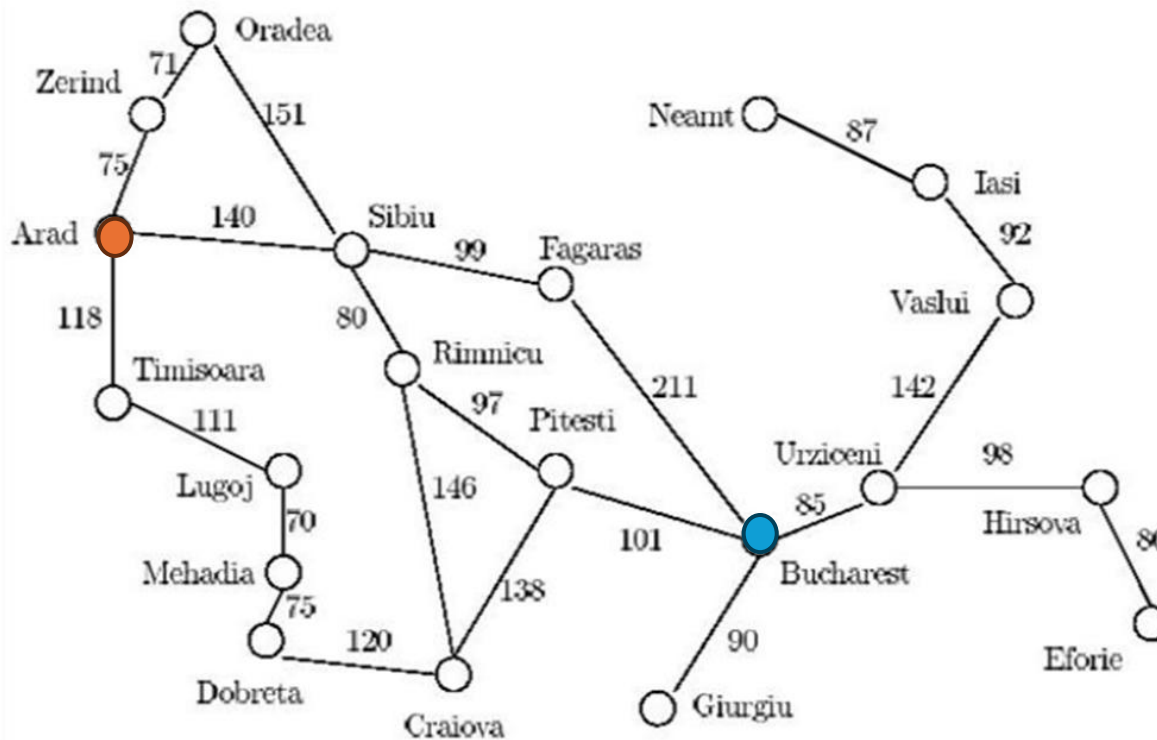


Order: A,S,T,Z,F,O,R,L,O,B

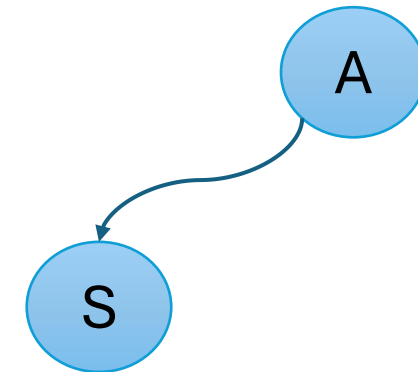
Path: A,S,F,B

1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

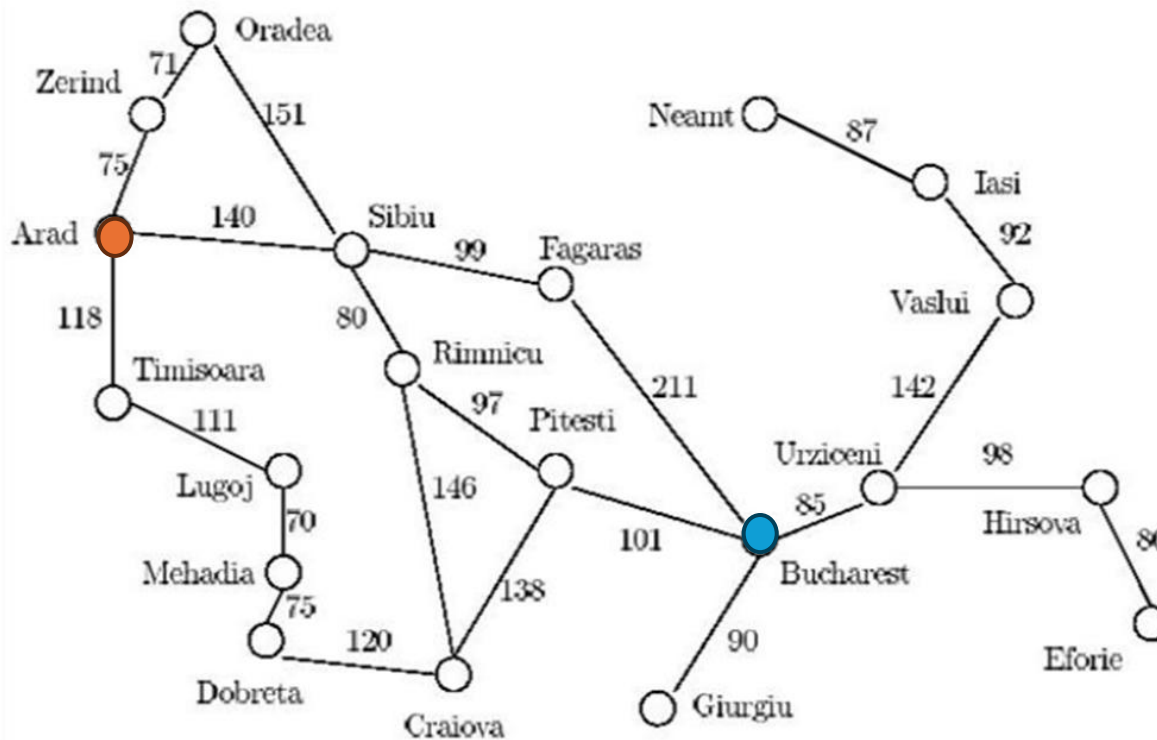


- 2 – Depth first

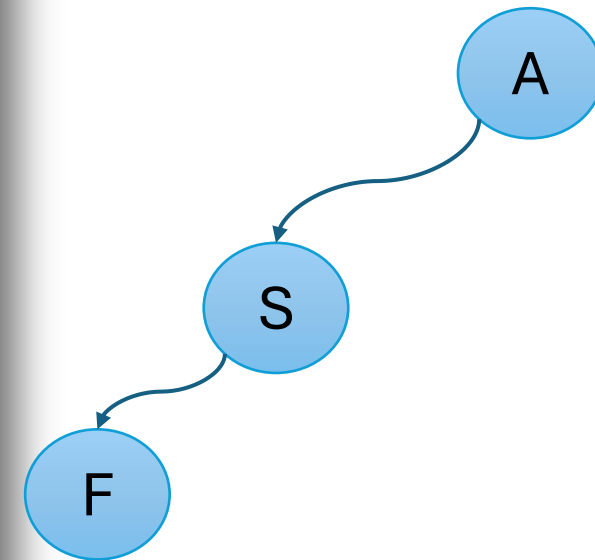


1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

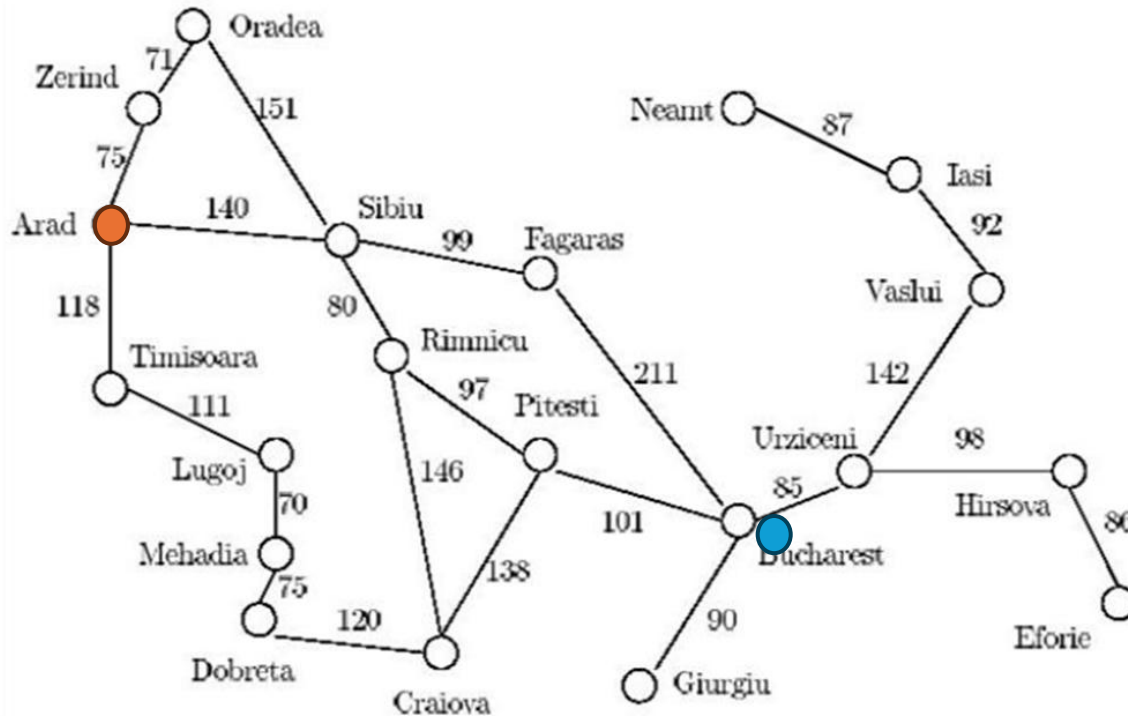


- 2 – Depth first

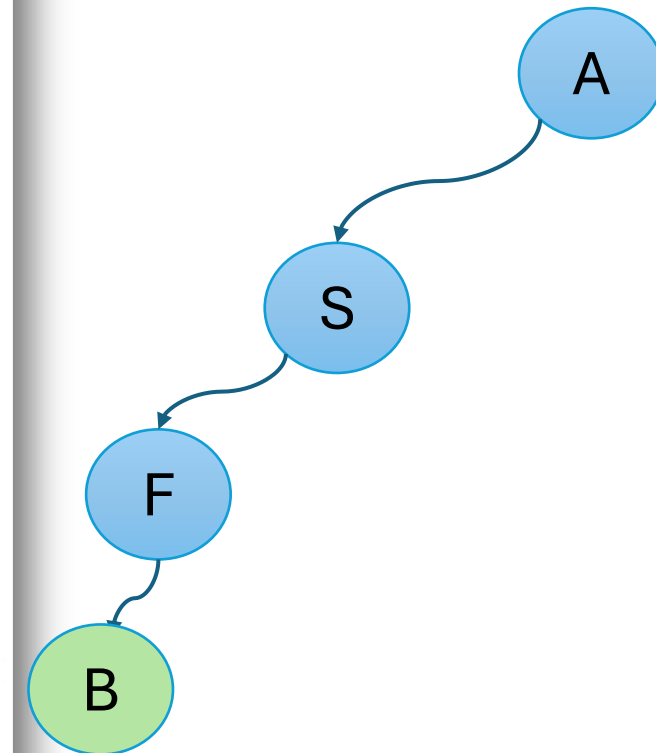


1. Given the **Arad/Bucharest** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

n	H(n)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



- 2 – Depth first

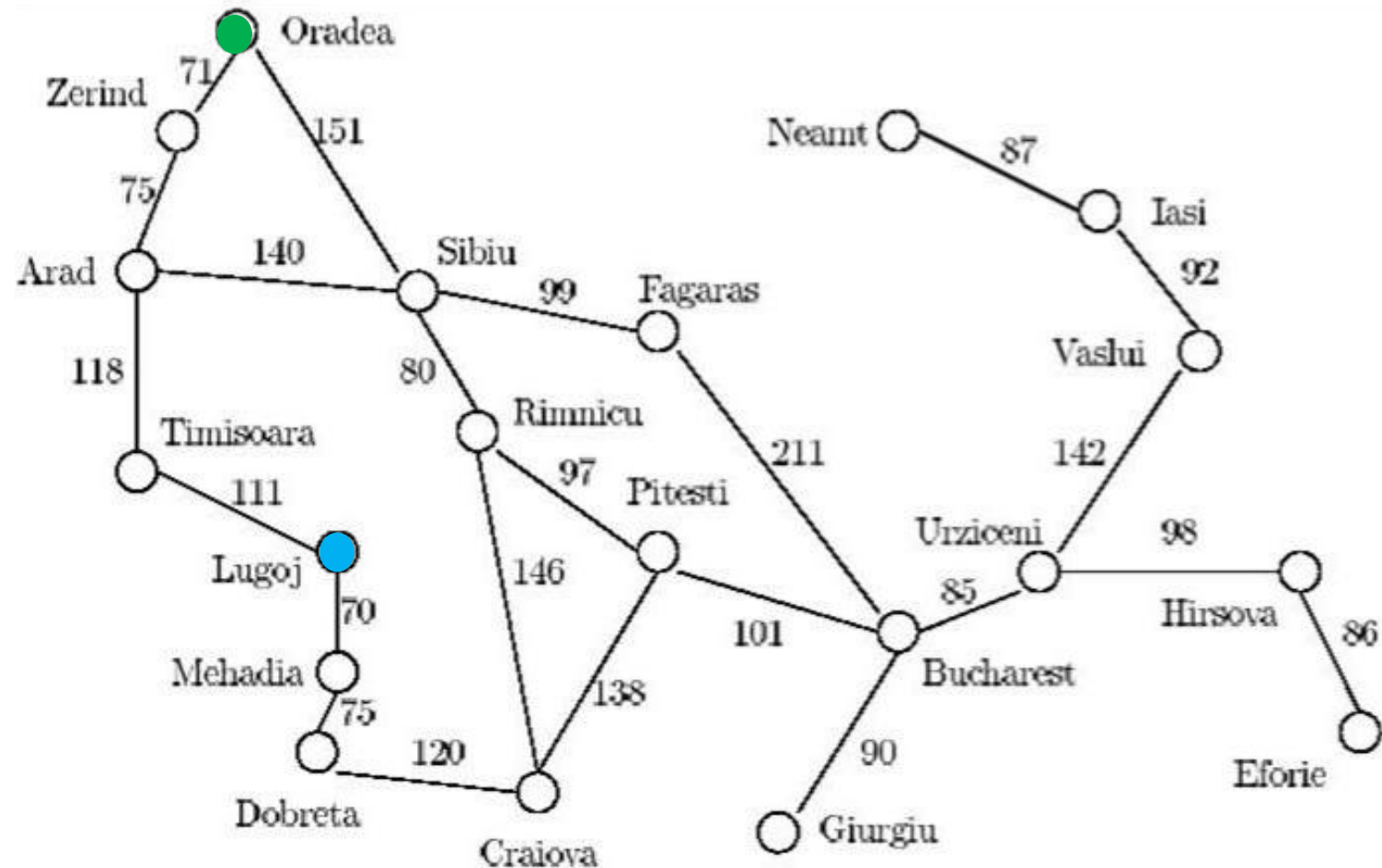


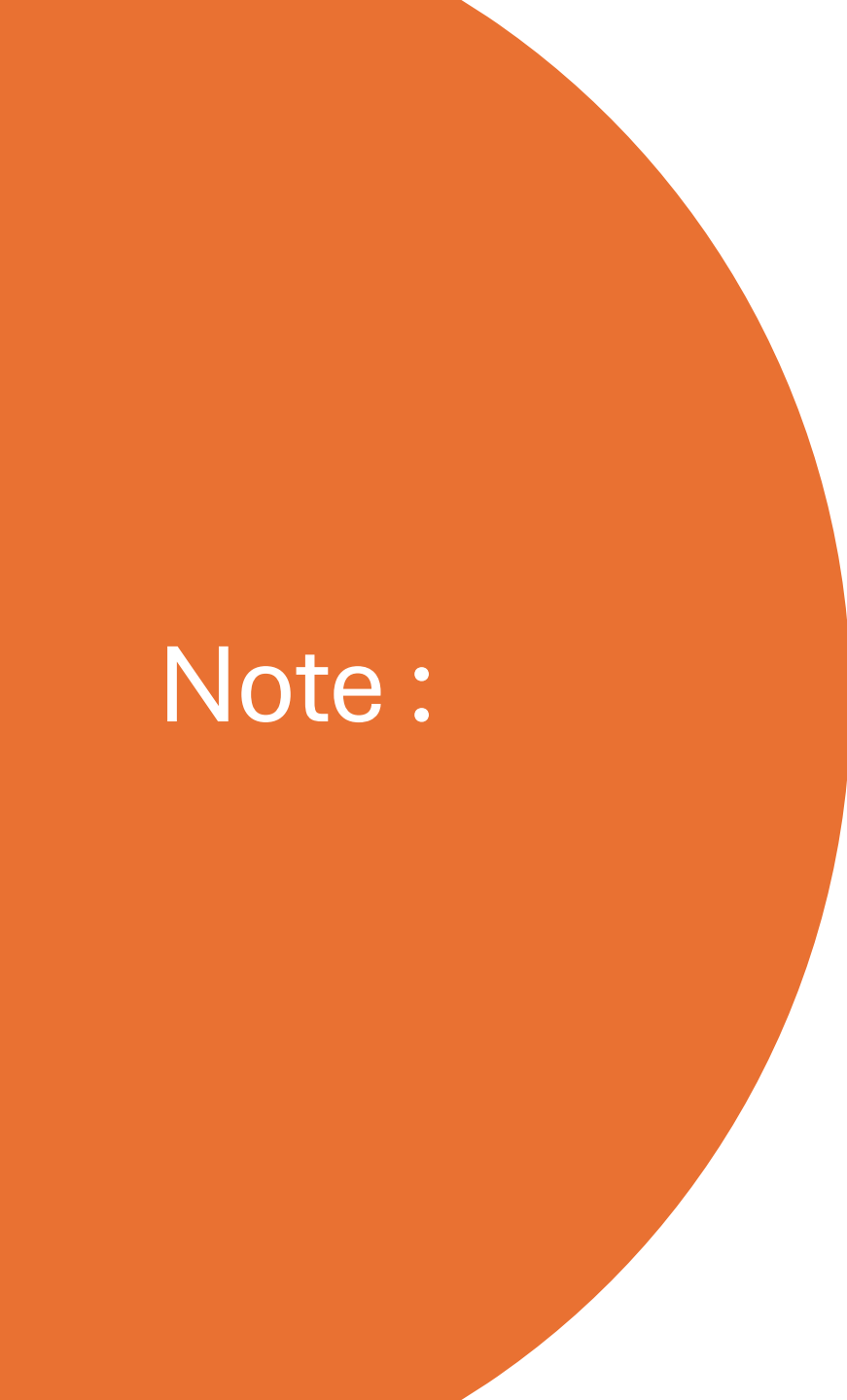
Order: A,S,F,B

Path: A,S,F,B

2. Given the **Oradea/Lugoj** route finding problem and the heuristics table below, show the list n of visited nodes using the following search techniques:

- a) Breadth first
- b) Depth First





Note :

- The **order** is the steps you took to reach the goal (the nodes you visited till you reached the goal)
- The **path** is the shortest way to the goal

